

Curso Profesional de JasperReports 6.20.0 Community

Módulo 2 — Diseño básico de informes

PRÁCTICAS

AUTOR: JAIME GALLO

JasperReports Library 6.20.0 Community · Jaspersoft Studio 6.20.0 Community Edition · Java 8 · Proyecto EditorialReports

Puntos incluidos

- 2.1 Bandas
- 2.2 Texto estático y campos de texto
- 2.3 Campos
- 2.4 Imágenes
- 2.5 Formato y estilos
- 2.6 Expresiones

Estado del proyecto al inicio del módulo

El alumno **no crea un proyecto nuevo**. Debe continuar con su propio resultado de M1.6. Para recuperación o incorporación directa puede copiar el checkpoint M1/1 . 6. El checkpoint M2/2 . 1 es la solución del primer punto, no su punto de partida.

Cadena pedagógica del módulo: A = construcción y modificación visual en Jaspersoft Studio; B = JRXML que representa y verifica lo construido; C = Java que compila, llena y exporta el mismo informe; D = comprobación del PDF y del estado del proyecto.

Ejecución reproducible. Las dependencias Java se resuelven mediante Maven. No se mantiene una lista manual de unos pocos JAR como sustituto del runtime completo.

Punto 2.1 — Bandas

PUNTO DE PARTIDA. Si vienes haciendo el curso, continúa con tu propio proyecto del punto anterior. Si te incorporas directamente aquí, usa M1/1.6 como estado inicial. El checkpoint M2/2.1 contiene la solución completa de este punto y no debe consultarse antes del ejercicio si quieres evitar spoilers.

Parte A — Práctica visual

Paso 1: Añadir la banda Last Page Footer VALIDADO

Acciones:

1. Hacer clic con el botón derecho sobre el nodo informe_concepto en el panel Outline (inferior izquierdo).
2. Hacer clic sobre la opción Add Band en el menú contextual.
3. Hacer clic sobre la opción Last Page Footer en el submenú.
4. Hacer clic sobre el borde inferior de la banda Last Page Footer en el editor central y arrastrarlo hasta que la altura sea de 30 unidades de informe.
5. Hacer clic sobre el campo Band height en el panel Properties (inferior derecho), pestaña Properties, escribir 30 y pulsar Enter.

Verificación visual: el panel Outline muestra un nuevo nodo Last Page Footer entre Page Footer y Summary. El editor central muestra la banda con 30 unidades de informe de altura.

Qué hace: añade la banda que sustituirá a Page Footer en la última página del informe.

Por qué: permite diferenciar el pie de la última página del resto de páginas.

Error común: añadir la banda Last Page Footer después de la banda Summary. El entorno la coloca automáticamente antes. Si se edita el XML manualmente y se coloca en el orden incorrecto, el editor muestra un subrayado amarillo.

Solución: eliminar la banda y volver a añadirla desde el panel Outline.

Analogía: es como reservar en el catálogo un pie de página distinto para la última página, con el colofón en lugar del número de página.

Paso 2: Añadir un mensaje en la banda Last Page Footer VALIDADO

Acciones:

1. Hacer clic sobre la pestaña Elements en el panel Palette (derecha del editor).
2. Hacer clic sobre el icono Static Text (una letra T mayúscula).
3. Arrastrar el icono Static Text y soltarlo dentro de la banda Last Page Footer, en la coordenada aproximada x=0, y=5.
4. Hacer doble clic sobre el Static Text creado en la acción anterior.
5. Escribir exactamente Documento generado en la última página.
6. Hacer clic sobre una zona vacía del editor central para confirmar el texto.
7. Hacer clic sobre el campo Width en el panel Properties, pestaña Properties, escribir 555 y pulsar Enter.
8. Hacer clic sobre el campo Height, escribir 20 y pulsar Enter.
9. Hacer clic sobre el campo X, escribir 0 y pulsar Enter.
10. Hacer clic sobre el campo Y, escribir 5 y pulsar Enter.

Verificación visual: la banda Last Page Footer muestra el texto Documento generado en la última página alineado a la izquierda.

Qué hace: inserta un texto que aparecerá únicamente en la última página del informe.

Por qué: permite diferenciar el cierre del documento en la última página.

Error común: dejar el texto alineado a la izquierda cuando se desea centrado.

Solución: hacer clic sobre el desplegable Horizontal Text Alignment en el panel Properties y seleccionar Center.

Analogía: es como escribir un mensaje de cierre en la última página del catálogo, distinto del pie de las páginas intermedias.

Paso 3: Crear la clase Libro en el proyecto Java VALIDADO

Acciones:

1. Hacer clic con el botón derecho sobre la carpeta src en el panel Project Explorer (superior izquierdo).
2. Hacer clic sobre la opción New en el menú contextual.
3. Hacer clic sobre la opción Class en el submenú.
4. Escribir exactamente Libro en el campo Name del diálogo New Java Class.
5. Hacer clic sobre el botón Finish.
6. En el editor central, escribir el código completo de la clase Libro que se muestra en la Parte C de este punto.
7. Pulsar Ctrl+S para guardar el archivo.

Verificación visual: el panel Project Explorer muestra el archivo Libro.java dentro de la carpeta src. El editor central muestra el código de la clase sin subrayados rojos.

Qué hace: crea la clase que representa un libro del catálogo con sus propiedades.

Por qué: la clase es el tipo de dato que alimentará la fuente de datos del informe.

Error común: escribir el nombre de la clase con minúscula inicial (libro). El compilador informa The public type libro must be defined in its own file.

Solución: renombrar la clase a Libro con mayúscula inicial.

Analogía: es como definir la ficha técnica que describe cada libro del catálogo.

Paso 4: Crear la clase CatalogoDataSource en el proyecto Java VALIDADO

Acciones:

1. Hacer clic con el botón derecho sobre la carpeta src en el panel Project Explorer.
2. Hacer clic sobre la opción New en el menú contextual.
3. Hacer clic sobre la opción Class en el submenú.
4. Escribir exactamente CatalogoDataSource en el campo Name del diálogo New Java Class.
5. Hacer clic sobre el botón Finish.
6. En el editor central, escribir el código completo de la clase CatalogoDataSource que se muestra en la Parte C de este punto.
7. Pulsar Ctrl+S para guardar el archivo.

Verificación visual: el panel Project Explorer muestra el archivo CatalogoDataSource.java dentro de la carpeta src.

Qué hace: crea la clase que implementa la interfaz JRDataSource y alimenta al motor con la lista de libros.

Por qué: la clase permite que la banda Detail del informe se emita una vez por cada libro.

Error común: olvidar implementar los métodos next() y getFieldValue() de la interfaz JRDataSource. El compilador informa CatalogoDataSource is not abstract and does not override abstract method next().

Solución: añadir los dos métodos con la anotación @Override.

Analogía: es como preparar la bandeja con las fichas de todos los libros que se van a maquetar en el catálogo.

Paso 5: Modificar el programa Java para usar la nueva fuente de datos VALIDADO

Acciones:

1. Hacer doble clic sobre el archivo GeneradorInformeConcepto.java en el panel Project Explorer.
2. Hacer clic sobre la línea que contiene `new JREmptyDataSource();` y seleccionarla completa.
3. Escribir exactamente `new CatalogoDataSource(Libro.listaEjemplo());` en su lugar.
4. Hacer clic sobre la línea que contiene `import net.sf.jasperreports.engine.JREmptyDataSource;` y pulsar `Ctrl+Mayús+/-` para comentarla.
5. Pulsar `Ctrl+S` para guardar el archivo.
6. Observar el panel Problems (inferior) y verificar que no hay errores.

Verificación visual: el editor central muestra la llamada a `new CatalogoDataSource(Libro.listaEjemplo())` en lugar de `new JREmptyDataSource()`. El panel Problems permanece vacío.

Qué hace: sustituye la fuente de datos vacía por una fuente que devuelve una lista de libros.

Por qué: la banda Detail se emitirá una vez por cada libro de la lista.

Error común: olvidar comentar la importación de `JREmptyDataSource` y obtener un aviso de importación no utilizada.

Solución: comentar la línea o eliminarla.

Analogía: es como cambiar la bandeja vacía por la bandeja con las fichas de los libros.

Paso 6: Declarar los campos en el JRXML VALIDADO

Acciones:

1. Hacer doble clic sobre el archivo `informe_concepto.jrxml` en el panel Project Explorer.
2. Hacer clic sobre la pestaña Source en la parte inferior del editor central.
3. Hacer clic al final de la línea que contiene `<style name="Sans_Normal" ...` y pulsar `Enter`.
4. Escribir exactamente `<field name="titulo" class="java.lang.String"/>` y pulsar `Enter`.
5. Escribir exactamente `<field name="precio" class="java.lang.Double"/>` y pulsar `Enter`.
6. Pulsar `Ctrl+S` para guardar el archivo.
7. Hacer clic sobre la pestaña Design en la parte inferior del editor central.
8. Expandir el nodo `informe_concepto` en el panel Outline y verificar que aparece un nodo Fields con los dos campos declarados.

Verificación visual: el panel Outline muestra un nodo Fields con dos entradas: `titulo` de tipo String y `precio` de tipo Double.

Qué hace: declara los campos que el motor resolverá para cada registro de la fuente de datos.

Por qué: las expresiones `$F{titulo}` y `$F{precio}` de la banda Detail necesitan que los campos estén declarados en el JRXML.

Error común: escribir el nombre del campo con mayúscula inicial (Titulo). El motor busca el campo en la fuente de datos por el nombre exacto y lanza `Field not found: Titulo`.

Solución: usar el nombre exacto en minúsculas que coincide con el campo de la clase Libro.

Analogía: es como declarar en el pliego qué datos del manuscrito se van a extraer para el catálogo.

Paso 7: Ajustar la banda Detail VALIDADO

Acciones:

1. Hacer clic sobre el nodo Detail 1 en el panel Outline (inferior izquierdo).

2. Hacer clic sobre el campo Band height en el panel Properties (inferior derecho), pestaña Properties, escribir 20 y pulsar Enter.
3. Hacer clic sobre el desplegable Split Type en el panel Properties y seleccionar Stretch.
4. Hacer clic sobre el primer textField de la banda Detail 1 en el editor central.
5. Verificar en el panel Properties que el campo Text Field Expression contiene $\$F\{\text{titulo}\}$.
6. Hacer clic sobre el segundo textField de la banda Detail 1 en el editor central.
7. Verificar en el panel Properties que el campo Text Field Expression contiene $\$F\{\text{precio}\}$.

Verificación visual: la banda Detail 1 aparece con 20 unidades de informe de altura y la propiedad Split Type ajustada a Stretch. Los dos campos tienen las expresiones correctas.

Qué hace: ajusta la banda Detail para que se emita una vez por cada registro con la altura adecuada.

Por qué: la banda Detail es la que repite el contenido variable del informe.

Error común: dejar la banda Detail sin campos. La banda se emite vacía y el informe no muestra ningún dato.

Solución: comprobar que los dos textField están dentro de la banda Detail 1.

Analogía: es como ajustar la altura de cada fila de la tabla del catálogo para que todos los libros quepan con claridad.

Paso 8: Añadir un contador de registros en la banda Summary VALIDADO

Acciones:

1. Hacer clic sobre el nodo Summary en el panel Outline (inferior izquierdo).
2. Hacer clic sobre el campo Band height en el panel Properties (inferior derecho), pestaña Properties, escribir 70 y pulsar Enter.
3. Hacer clic sobre la pestaña Elements en el panel Palette (derecha del editor).
4. Hacer clic sobre el icono Static Text (una letra T mayúscula).
5. Arrastrar el icono Static Text y soltarlo dentro de la banda Summary, en la coordenada aproximada $x=0, y=45$.
6. Hacer doble clic sobre el Static Text creado en la acción anterior.
7. Escribir exactamente Total de libros:.
8. Hacer clic sobre una zona vacía del editor central para confirmar el texto.
9. Hacer clic sobre el campo Width en el panel Properties, escribir 150 y pulsar Enter.
10. Hacer clic sobre el campo Height, escribir 20 y pulsar Enter.

Verificación visual: la banda Summary muestra el rótulo Total de libros: debajo de los elementos existentes.

Qué hace: añade un rótulo que precede al recuento total de libros.

Por qué: el recuento total de registros es un valor agregado que solo está disponible al final del llenado.

Error común: colocar el rótulo en la banda Page Footer. El rótulo aparecería repetido en cada página.

Solución: colocar el rótulo en la banda Summary.

Analogía: es como escribir en el colofón del catálogo el número total de libros que contiene.

Paso 9: Añadir el campo con el contador de registros VALIDADO

Acciones:

1. Hacer clic sobre la pestaña Elements en el panel Palette (derecha del editor).
2. Hacer clic sobre el icono Text Field (una letra F dentro de un cuadrado).
3. Arrastrar el icono Text Field y soltarlo dentro de la banda Summary, a la derecha del rótulo, en la coordenada aproximada $x=155, y=45$.
4. Hacer clic sobre el campo Text Field Expression en el panel Properties, pestaña Properties.

5. Escribir exactamente `$V{REPORT_COUNT}` y pulsar Enter.
6. Hacer clic sobre el campo Width, escribir 50 y pulsar Enter.
7. Hacer clic sobre el campo Height, escribir 20 y pulsar Enter.

Verificación visual: la banda Summary muestra el rótulo Total de libros: seguido de un campo con la expresión `$V{REPORT_COUNT}`.

Qué hace: inserta un campo que muestra el número total de registros procesados por el motor.

Por qué: la variable `REPORT_COUNT` es una variable incorporada que cuenta los registros procesados.

Error común: usar `$P{REPORT_COUNT}` en lugar de `$V{REPORT_COUNT}`. El compilador informa que el parámetro no existe.

Solución: cambiar el prefijo `$P{` por `$V{` porque `REPORT_COUNT` es una variable, no un parámetro.

Analogía: es como contar los libros que se han maquetado y escribir el total en el colofón.

Paso 10: Completar el Page Header con «Página N de M» VALIDADO

Acciones:

1. Hacer clic sobre el nodo Page Header en el panel Outline (inferior izquierdo).
2. Hacer clic sobre el Static Text que contiene Catálogo Editorial - Informe Conceptual y, en Properties, escribir 0 en X y 330 en Width. Mantener Y=5 y Height=15.
3. Hacer clic sobre la pestaña Elements del panel Palette y seleccionar Text Field.
4. Arrastrar el Text Field a Page Header; en Properties escribir X=330, Y=5, Width=170, Height=15.
5. Hacer clic sobre Text Field Expression, escribir exactamente "Página " + `$V{PAGE_NUMBER}` + " de" y pulsar Enter. Seleccionar alineación horizontal Right.
6. Volver a Palette > Elements y seleccionar otro Text Field.
7. Arrastrar el segundo campo a Page Header; escribir X=500, Y=5, Width=55, Height=15.
8. En Text Field Expression escribir exactamente `$V{PAGE_NUMBER}` y pulsar Enter.
9. En las propiedades del segundo campo localizar Evaluation Time y seleccionar Report; seleccionar también alineación horizontal Right.
10. Pulsar Ctrl+S y comprobar en Source que el segundo campo aparece como `<textField evaluationTime="Report">` y que su expresión contiene únicamente `$V{PAGE_NUMBER}`.

Verificación visual: el Page Header contiene tres elementos en la primera fila: el título abreviado, el campo "Página " + `$V{PAGE_NUMBER}` + " de" y, a su derecha, el campo `$V{PAGE_NUMBER}` evaluado al final del informe. En el PDF se obtiene Página N de M.

Qué hace: separa el número de página actual del total final de páginas.

Por qué: el primer `PAGE_NUMBER` se evalúa en el momento normal y muestra N; el segundo se evalúa con `evaluationTime="Report"` y muestra el valor final M.

Error común: escribir `evaluationTime="Report"` dentro de `textFieldExpression` o dejar el segundo campo con evaluación normal. **Solución:** `evaluationTime` es un atributo de `<textField>`, no parte de la expresión; seleccionar Report en la propiedad Evaluation Time.

Analogía: es como imprimir el número de la página que se está leyendo y, en una segunda casilla, el total definitivo de páginas de la edición.

Paso 11: Compilar y ejecutar el programa Java VALIDADO

Acciones:

1. Hacer clic sobre el archivo informe_concepto.jrxml en el panel Project Explorer.
2. Pulsar Ctrl+S para guardar el archivo.
3. Pulsar Ctrl+Mayús+B para compilar el informe.
4. Hacer clic sobre el panel Problems y verificar que no hay errores.
5. Hacer clic con el botón derecho sobre el archivo GeneradorInformeConcepto.java en el panel Project Explorer.

6. Hacer clic sobre la opción Run As en el menú contextual.
7. Hacer clic sobre la opción Java Application en el submenú.
8. Hacer clic sobre la vista Console en el panel inferior y observar el resultado.

Verificación visual: la vista Console muestra la línea Informe generado en: ... con la ruta absoluta del archivo PDF. El panel Problems permanece vacío.

Qué hace: compila el informe y ejecuta el programa Java con la nueva fuente de datos.

Por qué: el programa genera el PDF con la banda Detail emitida una vez por cada libro de la lista.

Error común: olvidar compilar el informe después de modificar el JRXML y obtener un PDF con la versión anterior.

Solución: pulsar Ctrl+Mayús+B antes de ejecutar el programa.

Analogía: es como imprimir la tirada del catálogo con todos los libros de la lista.

Paso 12: Documentar las bandas del informe en BANDAS.md VALIDADO

Acciones:

1. Hacer doble clic sobre el archivo BANDAS.md en el panel Project Explorer.
2. Hacer clic al final del archivo en el editor central.
3. Pulsar Enter y escribir exactamente ## Comportamiento con datos y pulsar Enter dos veces.
4. Escribir exactamente - La banda Detail se emite una vez por cada libro de la lista. y pulsar Enter.
5. Escribir exactamente - La banda Summary muestra el recuento total con \$V{REPORT_COUNT}. y pulsar Enter.
6. Escribir exactamente - La banda Page Header muestra "Página N de M". y pulsar Enter.
7. Escribir exactamente - La banda Last Page Footer sustituye a Page Footer en la última ocurrencia del pie de página; en este informe coincide con la página final. y pulsar Enter.
8. Pulsar Ctrl+S para guardar el archivo.

Verificación visual: el archivo BANDAS.md contiene la nueva sección con las cuatro anotaciones.

Qué hace: incorpora al proyecto la documentación del comportamiento de las bandas con datos.

Por qué: la documentación facilita el mantenimiento y la incorporación de nuevos desarrolladores.

Error común: olvidar guardar el archivo y perder los cambios al cerrar el editor.

Solución: pulsar Ctrl+S antes de cerrar el archivo.

Analogía: es como anotar en el manual del catálogo cómo se comporta cada sección cuando el manuscrito contiene varios capítulos.

Parte B — JRXML explicado y contrastado COMPLETADO

El siguiente JRXML representa **el estado tras los doce pasos de la Parte A y antes del reto resuelto**. Se ha contrastado con el checkpoint 2.1: mantiene el orden XSD correcto (background antes de title) y la paginación usa un segundo textField con evaluationTime="Report". El reto posterior añade TotalPrecios, amplía Summary y queda incorporado al checkpoint final.

XML

```
<?xml version="1.0" encoding="UTF-8"?>
<jasperReport xmlns="http://jasperreports.sourceforge.net/jasperreports"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://jasperreports.sourceforge.net/jasperreports http://jasperreports.sourceforge.net/xsd/
jasperreport.xsd"
  name="informe_concepto"
  language="java"
  pageWidth="595"
  pageHeight="842"
  columnWidth="555"
  leftMargin="20"
```

```

        rightMargin="20"
        topMargin="20"
        bottomMargin="20"
        uuid="8f2c1a4e-1d3b-4f5a-9c7e-2b6d8a0f1c33">
<property name="com.jaspersoft.studio.data.defaultdataadapter" value="EmptyDataSource"/>
<style name="Sans_Normal" isDefault="true" fontName="DejaVu Sans" fontSize="10"/>
<field name="titulo" class="java.lang.String"/>
<field name="precio" class="java.lang.Double"/>
<background>
    <band height="0"/>
</background>
<title>
    <band height="70">
        <staticText>
            <reportElement x="0" y="15" width="555" height="30" uuid="1a2b3c4d-5e6f-7a8b-9c0d-1e2f3a4b5c6d"/>
            <textElement textAlignment="Center" verticalAlignment="Middle"/>
            <text><![CDATA[Catálogo Editorial - Informe Conceptual]]></text>
        </staticText>
        <staticText>
            <reportElement x="0" y="45" width="120" height="20" uuid="2b3c4d5e-6f7a-8b9c-0d1e-2f3a4b5c6d7e"/>
            <text><![CDATA[Fecha de emisión:]]></text>
        </staticText>
        <textField pattern="dd/MM/yyyy">
            <reportElement x="125" y="45" width="150" height="20" uuid="3c4d5e6f-7a8b-9c0d-1e2f3a4b5c6d7e8f"/>
            <textFieldExpression><![CDATA[new java.util.Date()]]></textFieldExpression>
        </textField>
    </band>
</title>
<pageHeader>
    <band height="25">
        <staticText>
            <reportElement x="0" y="5" width="330" height="15" uuid="a1b2c3d4-e5f6-7a8b-9c0d-1e2f3a4b5c6d"/>
            <textElement verticalAlignment="Middle"><font size="9" isItalic="true"/></textElement>
            <text><![CDATA[Catálogo Editorial - Informe Conceptual]]></text>
        </staticText>
        <textField>
            <reportElement x="330" y="5" width="170" height="15" uuid="b2c3d4e5-f6a7-8b9c-0d1e-2f3a4b5c6d7e"/>
            <textElement textAlignment="Right" verticalAlignment="Middle"><font size="9" isItalic="true"/></textElement>
            <textFieldExpression><![CDATA["Página " + $V{PAGE_NUMBER} + " de"]></textFieldExpression>
        </textField>
        <textField evaluationTime="Report">
            <reportElement x="500" y="5" width="55" height="15" uuid="f9ab1a6c-59d2-5c71-a85f-72cee0efbcfb"/>
            <textElement textAlignment="Right" verticalAlignment="Middle"><font size="9" isItalic="true"/></textElement>
            <textFieldExpression><![CDATA[$V{PAGE_NUMBER}]]></textFieldExpression>
        </textField>
    </band>
</pageHeader>
<columnHeader>
    <band height="25">
        <staticText>
            <reportElement x="0" y="5" width="330" height="15" uuid="c3d4e5f6-a7b8-9c0d-1e2f-3a4b5c6d7e8f"/>
            <textElement verticalAlignment="Middle"><font size="10" isBold="true"/></textElement>
            <text><![CDATA[Título]]></text>
        </staticText>
        <staticText>
            <reportElement x="330" y="5" width="100" height="15" uuid="d4e5f6a7-b8c9-0d1e-2f3a-4b5c6d7e8f9a"/>
            <textElement verticalAlignment="Middle"><font size="10" isBold="true"/></textElement>
            <text><![CDATA[Precio]]></text>
        </staticText>
    </band>
</columnHeader>
<detail>
    <band height="20" splitType="Stretch">
        <textField>
            <reportElement x="0" y="0" width="330" height="20" uuid="e5f6a7b8-c9d0-1e2f-3a4b-5c6d7e8f9a0b"/>
            <textFieldExpression><![CDATA[{F{titulo}}]]></textFieldExpression>
        </textField>
        <textField pattern="#.##0.00">
            <reportElement x="330" y="0" width="100" height="20" uuid="f6a7b8c9-d0e1-2f3a-4b5c-6d7e8f9a0b1c"/>
            <textFieldExpression><![CDATA[{F{precio}}]]></textFieldExpression>
        </textField>
    </band>
</detail>
<columnFooter>
    <band height="25">
        <staticText>
            <reportElement x="0" y="5" width="555" height="15" uuid="a7b8c9d0-e1f2-3a4b-5c6d-7e8f9a0b1c2d"/>
            <textElement textAlignment="Center" verticalAlignment="Middle"><font size="9" isItalic="true"/></textElement>
            <text><![CDATA[--- Fin de la tabla de datos ---]]></text>
        </staticText>
    </band>
</columnFooter>
<pageFooter>
    <band height="30">
        <staticText>
            <reportElement x="0" y="5" width="555" height="20" uuid="b8c9d0e1-f2a3-4b5c-6d7e-8f9a0b1c2d3e"/>
            <textElement verticalAlignment="Middle"><font size="9"/></textElement>
            <text><![CDATA[EditorialReports - Documento generado con JasperReports 6.20.0]]></text>
        </staticText>
    </band>

```

```

</pageFooter>
<lastPageFooter>
  <band height="30">
    <staticText>
      <reportElement x="0" y="5" width="555" height="20" uuid="c9d0e1f2-a3b4-5c6d-7e8f-9a0b1c2d3e4f"/>
      <textElement textAlignment="Center" verticalAlignment="Middle"><font size="9" isItalic="true"/></textElement>
      <text><![CDATA[Documento generado en la última página]]></text>
    </staticText>
  </band>
</lastPageFooter>
<summary>
  <band height="70">
    <staticText>
      <reportElement x="0" y="5" width="150" height="20" uuid="d0e1f2a3-b4c5-6d7e-8f9a-0b1c2d3e4f5a"/>
      <text><![CDATA[Total de páginas:]]></text>
    </staticText>
    <textField evaluationTime="Report">
      <reportElement x="155" y="5" width="50" height="20" uuid="e1f2a3b4-c5d6-7e8f-9a0b-1c2d3e4f5a6b"/>
      <textFieldExpression><![CDATA[${PAGE_NUMBER}]]></textFieldExpression>
    </textField>
    <staticText>
      <reportElement x="0" y="25" width="555" height="20" uuid="f2a3b4c5-d6e7-8f9a-0b1c-2d3e4f5a6b7c"/>
      <textElement textAlignment="Center" verticalAlignment="Middle"/>
      <text><![CDATA[Fin del informe. EditorialReports.]]></text>
    </staticText>
    <staticText>
      <reportElement x="0" y="45" width="150" height="20" uuid="a3b4c5d6-e7f8-9a0b-1c2d-3e4f5a6b7c8d"/>
      <text><![CDATA[Total de libros:]]></text>
    </staticText>
    <textField>
      <reportElement x="155" y="45" width="80" height="20" uuid="b4c5d6e7-f8a9-0b1c-2d3e-4f5a6b7c8d9e"/>
      <textFieldExpression><![CDATA[${REPORT_COUNT}]]></textFieldExpression>
    </textField>
  </band>
</summary>
</jasperReport>

```

Explicación línea por línea

Línea 1	<?xml version="1.0" encoding="UTF-8"?> → declara la versión XML y la codificación UTF-8.
Línea 2	<jasperReport xmlns="http://jasperreports.sourceforge.net/jasperreports" → abre el elemento raíz del informe; los atributos siguientes fijan el espacio de nombres y la geometría.
Línea 3	xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" → continúa la configuración del elemento raíz.
Línea 4	xsi:schemaLocation="http://jasperreports.sourceforge.net/jasperreports http://jasperreports.sourceforge.net/xsd/jasperreport.xsd" → continúa la configuración del elemento raíz.
Línea 5	name="informe_concepto" → continúa la configuración del elemento raíz.
Línea 6	language="java" → continúa la configuración del elemento raíz.
Línea 7	pageWidth="595" → continúa la configuración del elemento raíz.
Línea 8	pageHeight="842" → continúa la configuración del elemento raíz.
Línea 9	columnWidth="555" → continúa la configuración del elemento raíz.
Línea 10	leftMargin="20" → continúa la configuración del elemento raíz.
Línea 11	rightMargin="20" → continúa la configuración del elemento raíz.
Línea 12	topMargin="20" → continúa la configuración del elemento raíz.
Línea 13	bottomMargin="20" → continúa la configuración del elemento raíz.
Línea 14	uuid="8f2c1a4e-1d3b-4f5a-9c7e-2b6d8a0f1c33"> → continúa la configuración del elemento raíz.
Línea 15	<property name="com.jaspersoft.studio.data.defaultdataadapter" value="EmptyDataSource"/> → declara una propiedad de Jaspersoft Studio usada por el diseño.
Línea 16	<style name="Sans_Normal" isDefault="true" fontName="DejaVu Sans" fontSize="10"/> → declara o aplica una definición de estilo del informe.
Línea 17	<field name="titulo" class="java.lang.String"/> → declara un campo JRXML y su tipo Java para que pueda resolverse desde la fuente de datos.

Línea 18	<code><field name="precio" class="java.lang.Double" /></code> → declara un campo JRXML y su tipo Java para que pueda resolverse desde la fuente de datos.
Línea 19	<code><background></code> → abre la sección de fondo; en el XSD se declara antes de las secciones de contenido.
Línea 20	<code><band height="0" /></code> → define la banda y su altura; si aparece <code>splitType</code> , establece la política de división.
Línea 21	<code></background></code> → cierra el elemento o sección abierto correspondiente.
Línea 22	<code><title></code> → abre la sección Title, generada una vez al comienzo del informe.
Línea 23	<code><band height="70"></code> → define la banda y su altura; si aparece <code>splitType</code> , establece la política de división.
Línea 24	<code><staticText></code> → abre un texto estático.
Línea 25	<code><reportElement x="0" y="15" width="555" height="30" uuid="1a2b3c4d-5e6f-7a8b-9c0d-1e2f3a4b5c6d" /></code> → fija coordenadas, tamaño, UUID y, cuando existe, el estilo del elemento.
Línea 26	<code><textElement textAlignment="Center" verticalAlignment="Middle" /></code> → configura alineación y/o marcado del texto.
Línea 27	<code><text><![CDATA[Catálogo Editorial - Informe Conceptual]]></text></code> → define el contenido literal del elemento estático.
Línea 28	<code></staticText></code> → cierra el elemento o sección abierto correspondiente.
Línea 29	<code><staticText></code> → abre un texto estático.
Línea 30	<code><reportElement x="0" y="45" width="120" height="20" uuid="2b3c4d5e-6f7a-8b9c-0d1e-2f3a4b5c6d7e" /></code> → fija coordenadas, tamaño, UUID y, cuando existe, el estilo del elemento.
Línea 31	<code><text><![CDATA[Fecha de emisión:]]></text></code> → define el contenido literal del elemento estático.
Línea 32	<code></staticText></code> → cierra el elemento o sección abierto correspondiente.
Línea 33	<code><textField pattern="dd/MM/yyyy"></code> → abre un campo de texto dinámico; sus atributos controlan evaluación, formato o nulos.
Línea 34	<code><reportElement x="125" y="45" width="150" height="20" uuid="3c4d5e6f-7a8b-9c0d-1e2f3a4b5c6d7e8f" /></code> → fija coordenadas, tamaño, UUID y, cuando existe, el estilo del elemento.
Línea 35	<code><textFieldExpression><![CDATA[new java.util.Date()]]></textFieldExpression></code> → abre un campo de texto dinámico; sus atributos controlan evaluación, formato o nulos.
Línea 36	<code></textField></code> → cierra el elemento o sección abierto correspondiente.
Línea 37	<code></band></code> → cierra el elemento o sección abierto correspondiente.
Línea 38	<code></title></code> → cierra el elemento o sección abierto correspondiente.
Línea 39	<code><pageHeader></code> → abre Page Header, generado al comienzo de cada página.
Línea 40	<code><band height="25"></code> → define la banda y su altura; si aparece <code>splitType</code> , establece la política de división.
Línea 41	<code><staticText></code> → abre un texto estático.
Línea 42	<code><reportElement x="0" y="5" width="330" height="15" uuid="a1b2c3d4-e5f6-7a8b-9c0d-1e2f3a4b5c6d" /></code> → fija coordenadas, tamaño, UUID y, cuando existe, el estilo del elemento.
Línea 43	<code><textElement verticalAlignment="Middle"></textElement></code> → configura alineación y/o marcado del texto.
Línea 44	<code><text><![CDATA[Catálogo Editorial - Informe Conceptual]]></text></code> → define el contenido literal del elemento estático.
Línea 45	<code></staticText></code> → cierra el elemento o sección abierto correspondiente.

Línea 46	<code><textField></code> → abre un campo de texto dinámico; sus atributos controlan evaluación, formato o nulos.
Línea 47	<code><reportElement x="330" y="5" width="170" height="15" uuid="b2c3d4e5-f6a7-8b9c-0d1e-2f3a4b5c6d7e"/></code> → fija coordenadas, tamaño, UUID y, cuando existe, el estilo del elemento.
Línea 48	<code><textElement textAlignment="Right" verticalAlignment="Middle"></textElement></code> → configura alineación y/o marcado del texto.
Línea 49	<code><textFieldExpression><![CDATA["Página " + \${PAGE_NUMBER} + " de"]]></textFieldExpression></code> → abre un campo de texto dinámico; sus atributos controlan evaluación, formato o nulos.
Línea 50	<code></textField></code> → cierra el elemento o sección abierto correspondiente.
Línea 51	<code><textField evaluationTime="Report"></code> → abre un campo de texto dinámico; sus atributos controlan evaluación, formato o nulos.
Línea 52	<code><reportElement x="500" y="5" width="55" height="15" uuid="f9ab1a6c-59d2-5c71-a85f-72cee0efbcfb"/></code> → fija coordenadas, tamaño, UUID y, cuando existe, el estilo del elemento.
Línea 53	<code><textElement textAlignment="Right" verticalAlignment="Middle"></textElement></code> → configura alineación y/o marcado del texto.
Línea 54	<code><textFieldExpression><![CDATA[\${PAGE_NUMBER}]]></textFieldExpression></code> → abre un campo de texto dinámico; sus atributos controlan evaluación, formato o nulos.
Línea 55	<code></textField></code> → cierra el elemento o sección abierto correspondiente.
Línea 56	<code></band></code> → cierra el elemento o sección abierto correspondiente.
Línea 57	<code></pageHeader></code> → cierra el elemento o sección abierto correspondiente.
Línea 58	<code><columnHeader></code> → abre Column Header, generado al comienzo de cada columna.
Línea 59	<code><band height="25"></code> → define la banda y su altura; si aparece <code>splitType</code> , establece la política de división.
Línea 60	<code><staticText></code> → abre un texto estático.
Línea 61	<code><reportElement x="0" y="5" width="330" height="15" uuid="c3d4e5f6-a7b8-9c0d-1e2f-3a4b5c6d7e8f"/></code> → fija coordenadas, tamaño, UUID y, cuando existe, el estilo del elemento.
Línea 62	<code><textElement verticalAlignment="Middle"></textElement></code> → configura alineación y/o marcado del texto.
Línea 63	<code><text><![CDATA[Título]]></text></code> → define el contenido literal del elemento estático.
Línea 64	<code></staticText></code> → cierra el elemento o sección abierto correspondiente.
Línea 65	<code><staticText></code> → abre un texto estático.
Línea 66	<code><reportElement x="330" y="5" width="100" height="15" uuid="d4e5f6a7-b8c9-0d1e-2f3a-4b5c6d7e8f9a"/></code> → fija coordenadas, tamaño, UUID y, cuando existe, el estilo del elemento.
Línea 67	<code><textElement verticalAlignment="Middle"></textElement></code> → configura alineación y/o marcado del texto.
Línea 68	<code><text><![CDATA[Precio]]></text></code> → define el contenido literal del elemento estático.
Línea 69	<code></staticText></code> → cierra el elemento o sección abierto correspondiente.
Línea 70	<code></band></code> → cierra el elemento o sección abierto correspondiente.
Línea 71	<code></columnHeader></code> → cierra el elemento o sección abierto correspondiente.
Línea 72	<code><detail></code> → abre Detail, que el motor intenta generar por cada registro.
Línea 73	<code><band height="20" splitType="Stretch"></code> → define la banda y su altura; si aparece <code>splitType</code> , establece la política de división.
Línea 74	<code><textField></code> → abre un campo de texto dinámico; sus atributos controlan evaluación, formato o nulos.

Línea 75	<code><reportElement x="0" y="0" width="330" height="20" uuid="e5f6a7b8-c9d0-1e2f-3a4b-5c6d7e8f9a0b" /></code> → fija coordenadas, tamaño, UUID y, cuando existe, el estilo del elemento.
Línea 76	<code><textFieldExpression><![CDATA[\$F{titulo}]]></textFieldExpression></code> → abre un campo de texto dinámico; sus atributos controlan evaluación, formato o nulos.
Línea 77	<code></textField></code> → cierra el elemento o sección abierto correspondiente.
Línea 78	<code><textField pattern="#.##0.00"></code> → abre un campo de texto dinámico; sus atributos controlan evaluación, formato o nulos.
Línea 79	<code><reportElement x="330" y="0" width="100" height="20" uuid="f6a7b8c9-d0e1-2f3a-4b5c-6d7e8f9a0b1c" /></code> → fija coordenadas, tamaño, UUID y, cuando existe, el estilo del elemento.
Línea 80	<code><textFieldExpression><![CDATA[\$F{precio}]]></textFieldExpression></code> → abre un campo de texto dinámico; sus atributos controlan evaluación, formato o nulos.
Línea 81	<code></textField></code> → cierra el elemento o sección abierto correspondiente.
Línea 82	<code></band></code> → cierra el elemento o sección abierto correspondiente.
Línea 83	<code></detail></code> → cierra el elemento o sección abierto correspondiente.
Línea 84	<code><columnFooter></code> → abre Column Footer.
Línea 85	<code><band height="25"></code> → define la banda y su altura; si aparece <code>splitType</code> , establece la política de división.
Línea 86	<code><staticText></code> → abre un texto estático.
Línea 87	<code><reportElement x="0" y="5" width="555" height="15" uuid="a7b8c9d0-e1f2-3a4b-5c6d-7e8f9a0b1c2d" /></code> → fija coordenadas, tamaño, UUID y, cuando existe, el estilo del elemento.
Línea 88	<code><textElement textAlignment="Center" verticalAlignment="Middle"></textElement></code> → configura alineación y/o marcado del texto.
Línea 89	<code><text><![CDATA[--- Fin de la tabla de datos ---]]></text></code> → define el contenido literal del elemento estático.
Línea 90	<code></staticText></code> → cierra el elemento o sección abierto correspondiente.
Línea 91	<code></band></code> → cierra el elemento o sección abierto correspondiente.
Línea 92	<code></columnFooter></code> → cierra el elemento o sección abierto correspondiente.
Línea 93	<code><pageFooter></code> → abre el pie normal de página.
Línea 94	<code><band height="30"></code> → define la banda y su altura; si aparece <code>splitType</code> , establece la política de división.
Línea 95	<code><staticText></code> → abre un texto estático.
Línea 96	<code><reportElement x="0" y="5" width="555" height="20" uuid="b8c9d0e1-f2a3-4b5c-6d7e-8f9a0b1c2d3e" /></code> → fija coordenadas, tamaño, UUID y, cuando existe, el estilo del elemento.
Línea 97	<code><textElement verticalAlignment="Middle"></textElement></code> → configura alineación y/o marcado del texto.
Línea 98	<code><text><![CDATA[EditorialReports - Documento generado con JasperReports 6.20.0]]></text></code> → define el contenido literal del elemento estático.
Línea 99	<code></staticText></code> → cierra el elemento o sección abierto correspondiente.
Línea 100	<code></band></code> → cierra el elemento o sección abierto correspondiente.
Línea 101	<code></pageFooter></code> → cierra el elemento o sección abierto correspondiente.
Línea 102	<code><lastPageFooter></code> → abre el pie que sustituye al <code>pageFooter</code> en su última ocurrencia.
Línea 103	<code><band height="30"></code> → define la banda y su altura; si aparece <code>splitType</code> , establece la política de división.
Línea 104	<code><staticText></code> → abre un texto estático.

Línea 105	<code><reportElement x="0" y="5" width="555" height="20" uuid="c9d0e1f2-a3b4-5c6d-7e8f-9a0b1c2d3e4f" /></code> → fija coordenadas, tamaño, UUID y, cuando existe, el estilo del elemento.
Línea 106	<code><textElement textAlignment="Center" verticalAlignment="Middle"></textElement></code> → configura alineación y/o marcado del texto.
Línea 107	<code><text><![CDATA[Documento generado en la última página]]></text></code> → define el contenido literal del elemento estático.
Línea 108	<code></staticText></code> → cierra el elemento o sección abierto correspondiente.
Línea 109	<code></band></code> → cierra el elemento o sección abierto correspondiente.
Línea 110	<code></lastPageFooter></code> → cierra el elemento o sección abierto correspondiente.
Línea 111	<code><summary></code> → abre Summary, generado una vez al final del contenido del informe.
Línea 112	<code><band height="70"></code> → define la banda y su altura; si aparece <code>splitType</code> , establece la política de división.
Línea 113	<code><staticText></code> → abre un texto estático.
Línea 114	<code><reportElement x="0" y="5" width="150" height="20" uuid="d0e1f2a3-b4c5-6d7e-8f9a-0b1c2d3e4f5a" /></code> → fija coordenadas, tamaño, UUID y, cuando existe, el estilo del elemento.
Línea 115	<code><text><![CDATA[Total de páginas:]]></text></code> → define el contenido literal del elemento estático.
Línea 116	<code></staticText></code> → cierra el elemento o sección abierto correspondiente.
Línea 117	<code><textField evaluationTime="Report"></code> → abre un campo de texto dinámico; sus atributos controlan evaluación, formato o nulos.
Línea 118	<code><reportElement x="155" y="5" width="50" height="20" uuid="e1f2a3b4-c5d6-7e8f-9a0b-1c2d3e4f5a6b" /></code> → fija coordenadas, tamaño, UUID y, cuando existe, el estilo del elemento.
Línea 119	<code><textFieldExpression><![CDATA[\${PAGE_NUMBER}]]></textFieldExpression></code> → abre un campo de texto dinámico; sus atributos controlan evaluación, formato o nulos.
Línea 120	<code></textField></code> → cierra el elemento o sección abierto correspondiente.
Línea 121	<code><staticText></code> → abre un texto estático.
Línea 122	<code><reportElement x="0" y="25" width="555" height="20" uuid="f2a3b4c5-d6e7-8f9a-0b1c-2d3e4f5a6b7c" /></code> → fija coordenadas, tamaño, UUID y, cuando existe, el estilo del elemento.
Línea 123	<code><textElement textAlignment="Center" verticalAlignment="Middle" /></code> → configura alineación y/o marcado del texto.
Línea 124	<code><text><![CDATA[Fin del informe. EditorialReports.]]></text></code> → define el contenido literal del elemento estático.
Línea 125	<code></staticText></code> → cierra el elemento o sección abierto correspondiente.
Línea 126	<code><staticText></code> → abre un texto estático.
Línea 127	<code><reportElement x="0" y="45" width="150" height="20" uuid="a3b4c5d6-e7f8-9a0b-1c2d-3e4f5a6b7c8d" /></code> → fija coordenadas, tamaño, UUID y, cuando existe, el estilo del elemento.
Línea 128	<code><text><![CDATA[Total de libros:]]></text></code> → define el contenido literal del elemento estático.
Línea 129	<code></staticText></code> → cierra el elemento o sección abierto correspondiente.
Línea 130	<code><textField></code> → abre un campo de texto dinámico; sus atributos controlan evaluación, formato o nulos.
Línea 131	<code><reportElement x="155" y="45" width="80" height="20" uuid="b4c5d6e7-f8a9-0b1c-2d3e-4f5a6b7c8d9e" /></code> → fija coordenadas, tamaño, UUID y, cuando existe, el estilo del elemento.
Línea 132	<code><textFieldExpression><![CDATA[\${REPORT_COUNT}]]></textFieldExpression></code> → abre un campo de texto dinámico; sus atributos controlan evaluación, formato o nulos.
Línea 133	<code></textField></code> → cierra el elemento o sección abierto correspondiente.

Línea 134	<code></band></code> → cierra el elemento o sección abierto correspondiente.
Línea 135	<code></summary></code> → cierra el elemento o sección abierto correspondiente.
Línea 136	<code></jasperReport></code> → cierra el elemento o sección abierto correspondiente.

Parte C — Código Java explicado línea por línea COMPLETADO

2.1 introduce el modelo Libro, la implementación CatalogoDataSource y sustituye la fuente vacía por datos reales.

Los tres archivos siguientes se reproducen **literalmente desde el checkpoint ejecutable M2/2.1**. De este modo, la Parte C coincide con el código que compila y se ejecuta en la validación end-to-end.

Clase Libro.java

JAVA

```
import java.util.ArrayList;
import java.util.List;

public class Libro {
    private final String titulo;
    private final Double precio;

    public Libro(String titulo, Double precio) {
        this.titulo = titulo;
        this.precio = precio;
    }

    public String getTitulo() {
        return titulo;
    }

    public Double getPrecio() {
        return precio;
    }

    public static List<Libro> listaEjemplo() {
        List<Libro> libros = new ArrayList<Libro>();
        libros.add(new Libro("Cien años de soledad", 19.95));
        libros.add(new Libro("Rayuela", 22.50));
        libros.add(new Libro("La ciudad y los perros", 18.75));
        libros.add(new Libro("Pedro Páramo", 15.90));
        libros.add(new Libro("Ficciones", 21.00));
        libros.add(new Libro("La casa de los espíritus", 23.40));
        libros.add(new Libro("El amor en los tiempos del cólera", 20.80));
        libros.add(new Libro("La muerte de Artemio Cruz", 17.60));
        libros.add(new Libro("Doña Bárbara", 16.95));
        libros.add(new Libro("Martín Fierro", 14.50));
        libros.add(new Libro("Comala", 19.20));
        libros.add(new Libro("Paradiso", 25.00));
        return libros;
    }
}
```

Explicación línea por línea

Línea 1	<code>import java.util.ArrayList;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 2	<code>import java.util.List;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 4	<code>public class Libro {</code> → declara la clase Java y, si aparece <code>implements</code> , establece el contrato que debe implementar.
Línea 5	<code>private final String titulo;</code> → declara un campo de instancia inmutable después de la construcción del objeto.
Línea 6	<code>private final Double precio;</code> → declara un campo de instancia inmutable después de la construcción del objeto.
Línea 8	<code>public Libro(String titulo, Double precio) {</code> → declara el constructor completo del modelo Libro con los valores que necesita el informe.
Línea 9	<code>this.titulo = titulo;</code> → asigna al campo del objeto el valor recibido o calculado.

Línea 10	<code>this.precio = precio;</code> → asigna al campo del objeto el valor recibido o calculado.
Línea 11	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.
Línea 13	<code>public String getTitulo()</code> { → devuelve el título del libro.
Línea 14	<code>return titulo;</code> → forma parte del bloque Java reproducido literalmente desde el checkpoint ejecutable.
Línea 15	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.
Línea 17	<code>public Double getPrecio()</code> { → devuelve el precio del libro.
Línea 18	<code>return precio;</code> → forma parte del bloque Java reproducido literalmente desde el checkpoint ejecutable.
Línea 19	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.
Línea 21	<code>public static List<Libro> listaEjemplo()</code> { → abre el método que construye los datos de ejemplo del curso.
Línea 22	<code>List<Libro> libros = new ArrayList<Libro>();</code> → crea una lista tipada compatible con Java 8 y con la baseline del proyecto.
Línea 23	<code>libros.add(new Libro("Cien años de soledad", 19.95));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.
Línea 24	<code>libros.add(new Libro("Rayuela", 22.50));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.
Línea 25	<code>libros.add(new Libro("La ciudad y los perros", 18.75));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.
Línea 26	<code>libros.add(new Libro("Pedro Páramo", 15.90));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.
Línea 27	<code>libros.add(new Libro("Ficciones", 21.00));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.
Línea 28	<code>libros.add(new Libro("La casa de los espíritus", 23.40));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.
Línea 29	<code>libros.add(new Libro("El amor en los tiempos del cólera", 20.80));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.
Línea 30	<code>libros.add(new Libro("La muerte de Artemio Cruz", 17.60));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.
Línea 31	<code>libros.add(new Libro("Doña Bárbara", 16.95));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.
Línea 32	<code>libros.add(new Libro("Martín Fierro", 14.50));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.
Línea 33	<code>libros.add(new Libro("Comala", 19.20));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.
Línea 34	<code>libros.add(new Libro("Paradiso", 25.00));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.
Línea 35	<code>return libros;</code> → devuelve la lista completa que alimentará la fuente de datos.
Línea 36	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.
Línea 37	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.

Clase CatalogoDataSource.java

JAVA

```
import java.util.List;
import net.sf.jasperreports.engine.JRDataSource;
```

```
import net.sf.jasperreports.engine.JRException;
import net.sf.jasperreports.engine.JRField;

public class CatalogoDataSource implements JRDataSource {
    private final List<Libro> libros;
    private int indice = -1;

    public CatalogoDataSource(List<Libro> libros) {
        this.libros = libros;
    }

    @Override
    public boolean next() throws JRException {
        indice++;
        return indice < libros.size();
    }

    @Override
    public Object getFieldValue(JRField campo) throws JRException {
        Libro actual = libros.get(indice);
        if ("titulo".equals(campo.getName())) {
            return actual.getTitulo();
        }
        if ("precio".equals(campo.getName())) {
            return actual.getPrecio();
        }
        throw new JRException("Campo no soportado por CatalogoDataSource: " + campo.getName());
    }
}
```

Explicación línea por línea

Línea 1	<code>import java.util.List;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 2	<code>import net.sf.jasperreports.engine.JRDataSource;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 3	<code>import net.sf.jasperreports.engine.JRException;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 4	<code>import net.sf.jasperreports.engine.JRField;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 6	<code>public class CatalogoDataSource implements JRDataSource {</code> → declara la clase Java y, si aparece <code>implements</code> , establece el contrato que debe implementar.
Línea 7	<code>private final List<Libro> libros;</code> → declara un campo de instancia inmutable después de la construcción del objeto.
Línea 8	<code>private int indice = -1;</code> → declara el índice interno de la fuente de datos; empieza en -1 porque <code>next()</code> se invoca antes de leer el primer registro.
Línea 10	<code>public CatalogoDataSource(List<Libro> libros) {</code> → declara el constructor de la fuente de datos y recibe la lista de libros.
Línea 11	<code>this.libros = libros;</code> → asigna al campo del objeto el valor recibido o calculado.
Línea 12	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.
Línea 14	<code>@Override</code> → indica que el método implementa un método del contrato <code>JRDataSource</code> .
Línea 15	<code>public boolean next() throws JRException {</code> → implementa <code>JRDataSource.next()</code> y declara <code>JRException</code> según el contrato de JasperReports.
Línea 16	<code>indice++;</code> → avanza al siguiente registro.
Línea 17	<code>return indice < libros.size();</code> → indica al motor si todavía existe un registro válido.
Línea 18	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.
Línea 20	<code>@Override</code> → indica que el método implementa un método del contrato <code>JRDataSource</code> .
Línea 21	<code>public Object getFieldValue(JRField campo) throws JRException {</code> → implementa la resolución de un campo JRXML para el registro actual.

Línea 22	<code>Libro actual = libros.get(indice);</code> → obtiene el libro correspondiente al índice actual.
Línea 23	<code>if ("titulo".equals(campo.getName())) {</code> → resuelve el campo titulo.
Línea 24	<code>return actual.getTitulo();</code> → devuelve el valor del campo solicitado para el libro actual.
Línea 25	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.
Línea 26	<code>if ("precio".equals(campo.getName())) {</code> → resuelve el campo precio.
Línea 27	<code>return actual.getPrecio();</code> → devuelve el valor del campo solicitado para el libro actual.
Línea 28	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.
Línea 29	<code>throw new JRException("Campo no soportado por CatalogoDataSource: " + campo.getName());</code> → falla explícitamente si el JRXML solicita un campo que la fuente no soporta, evitando devolver silenciosamente un valor incorrecto.
Línea 30	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.
Línea 31	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.

Clase GeneradorInformeConcepto.java

JAVA

```
import java.io.File;
import java.util.HashMap;
import java.util.Map;

import net.sf.jasperreports.engine.JasperCompileManager;
import net.sf.jasperreports.engine.JasperExportManager;
import net.sf.jasperreports.engine.JasperFillManager;
import net.sf.jasperreports.engine.JasperPrint;

public class GeneradorInformeConcepto {
    public static void main(String[] args) {
        try {
            String rutaJrxml = "reports/informe_concepto.jrxml";
            String rutaJasper = "reports/informe_concepto.jasper";
            String rutaPdf = "output/informe_concepto.pdf";

            JasperCompileManager.compileReportToFile(rutaJrxml, rutaJasper);

            Map<String, Object> parametros = new HashMap<String, Object>();

            JasperPrint documento = JasperFillManager.fillReport(
                rutaJasper,
                parametros,
                new CatalogoDataSource(Libro.listaEjemplo()));

            JasperExportManager.exportReportToPdfFile(documento, rutaPdf);

            System.out.println("Informe generado en: " + new File(rutaPdf).getAbsolutePath());
            System.out.println("Paginas del documento: " + documento.getPages().size());
            System.out.println("Registros de ejemplo: " + Libro.listaEjemplo().size());
        } catch (Exception e) {
            e.printStackTrace();
            System.exit(1);
        }
    }
}
```

Explicación línea por línea

Línea 1	<code>import java.io.File;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 2	<code>import java.util.HashMap;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 3	<code>import java.util.Map;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.

Línea 5	<code>import net.sf.jasperreports.engine.JasperCompileManager;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 6	<code>import net.sf.jasperreports.engine.JasperExportManager;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 7	<code>import net.sf.jasperreports.engine.JasperFillManager;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 8	<code>import net.sf.jasperreports.engine.JasperPrint;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 10	<code>public class GeneradorInformeConcepto {</code> → declara la clase Java y, si aparece <code>implements</code> , establece el contrato que debe implementar.
Línea 11	<code>public static void main(String[] args) {</code> → declara el punto de entrada de la aplicación.
Línea 12	<code>try {</code> → abre el bloque protegido de ejecución.
Línea 13	<code>String rutaJrxml = "reports/informe_concepto.jrxml";</code> → define la ruta relativa de la plantilla JRXML.
Línea 14	<code>String rutaJasper = "reports/informe_concepto.jasper";</code> → define la ruta del artefacto compilado <code>.jasper</code> .
Línea 15	<code>String rutaPdf = "output/informe_concepto.pdf";</code> → define la ruta del PDF de salida.
Línea 17	<code>JasperCompileManager.compileReportToFile(rutaJrxml, rutaJasper);</code> → compila el JRXML con JasperReports Library.
Línea 19	<code>Map<String, Object> parametros = new HashMap<String, Object>();</code> → crea el mapa tipado de parámetros.
Línea 21	<code>JasperPrint documento = JasperFillManager.fillReport(</code> → declara el <code>JasperPrint</code> resultante del llenado.
Línea 22	<code>rutaJasper,</code> → pasa al llenado el informe compilado.
Línea 23	<code>parametros,</code> → pasa el mapa de parámetros.
Línea 24	<code>new CatalogoDataSource(Libro.listaEjemplo());</code> → pasa la fuente de datos construida con los libros de ejemplo.
Línea 26	<code>JasperExportManager.exportReportToPdfFile(documento, rutaPdf);</code> → exporta el <code>JasperPrint</code> a un PDF real.
Línea 28	<code>System.out.println("Informe generado en: " + new File(rutaPdf).getAbsolutePath());</code> → escribe en consola la ruta absoluta del PDF generado.
Línea 29	<code>System.out.println("Paginas del documento: " + documento.getPages().size());</code> → escribe en consola el número real de páginas del <code>JasperPrint</code> .
Línea 30	<code>System.out.println("Registros de ejemplo: " + Libro.listaEjemplo().size());</code> → escribe en consola el número de registros de ejemplo; el workflow usa esta salida como evidencia de ejecución.
Línea 31	<code>} catch (Exception e) {</code> → captura cualquier fallo de compilación, llenado o exportación.
Línea 32	<code>e.printStackTrace();</code> → imprime la traza del error para diagnóstico.
Línea 33	<code>System.exit(1);</code> → termina con código distinto de cero para que GitHub Actions detecte el fallo.
Línea 34	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.
Línea 35	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.
Línea 36	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.

Traza de consola esperada tras la ejecución

TEXT

Informe generado en: C:\Users\<usuario>\Documents\JasperProjects\EditorialReports\output\informe_concepto.pdf
 Páginas del documento: 1
 Registros de ejemplo: 12

La ruta depende del equipo. Los valores anteriores están comprobados en el run E2E 36011315054: checkpoint 2.1 → 1 página y 12 registros.

Parte D — Validación del resultado y estructura del proyecto

D.1 — Vista de diseño en Jaspersoft Studio

TEXT

informe_concepto.jrxml	[Design]	[Source]
Ruler: 0 100 200 300 400 500 555		
Title h = 70 Catálogo Editorial - Informe Conceptual Fecha de emisión: [new java.util.Date()]		
Page Header h = 25 Catálogo Editorial (cursiva) ["Página "+\$V{PAGE_NUM...}]		
Column Header h = 25 Título Precio		
Detail 1 h = 20 [\$F{titulo}] [\$F{precio}] (se emite 12 veces con los 12 libros)		
Column Footer h = 25 --- Fin de la tabla de datos ---		
Page Footer h = 30 EditorialReports - Documento...		
Last Page Footer h = 30 Documento generado en la última página		
Summary h = 70 Total de páginas: [\$V{PAGE_NUMBER} [evaluationTime="Report"]] Fin del informe. EditorialReports. Total de libros: [\$V{REPORT_COUNT}]		
Background h = 0		

Qué representa: la disposición de las bandas en el editor central tras completar los doce pasos de la Parte A. La banda Detail se emite doce veces, una por cada libro.

Cómo verificarlo: comparar la presencia y la geometría de las secciones con este esquema. Para el orden XML válido, usar la Parte B: background se declara antes de las secciones de contenido aunque visualmente el diseñador pueda presentarlo en otra posición.

D.2 — Jerarquía del Outline

TEXT

```
informe_concepto
├── Properties
│   └── com.jaspersoft.studio.data.defaultdataadapter = EmptyDataSource
├── Styles
│   └── Sans_Normal [isDefault=true]
├── Fields
│   ├── titulo [java.lang.String]
│   └── precio [java.lang.Double]
```

```

└─ Title [band, height=70]
  └─ staticText "Catálogo Editorial - Informe Conceptual"
  └─ staticText "Fecha de emisión:"
  └─ textField [pattern=dd/MM/yyyy] new java.util.Date()

└─ Page Header [band, height=25]
  └─ staticText "Catálogo Editorial - Informe Conceptual" (italic)
  └─ textField [right] "Página " + ${PAGE_NUMBER} + " de"

└─ Column Header [band, height=25]
  └─ staticText "Título" (bold)
  └─ staticText "Precio" (bold)

└─ Detail 1 [band, height=20, splitType=Stretch]
  └─ textField ${F{titulo}}
  └─ textField [pattern=#,##0.00] ${F{precio}}

└─ Column Footer [band, height=25]
  └─ staticText "--- Fin de la tabla de datos ---"

└─ Page Footer [band, height=30]
  └─ staticText "EditorialReports - Documento..."

└─ Last Page Footer [band, height=30]
  └─ staticText "Documento generado en la última página"

└─ Summary [band, height=70]
  └─ staticText "Total de páginas:"
  └─ textField ${V{PAGE_NUMBER}} [evaluationTime="Report"]
  └─ staticText "Fin del informe. EditorialReports."
  └─ staticText "Total de libros:" (bold)
  └─ textField ${V{REPORT_COUNT}} (bold)

└─ Background [band, height=0]

```

Qué representa: el árbol de nodos del informe tal como aparece en el panel Outline. La novedad respecto al punto anterior es la sección Fields con los dos campos declarados y la banda Last Page Footer.

Cómo verificarlo: expandir el nodo informe_concepto en el panel Outline y comparar la estructura.

D.3 — Documento PDF resultante, página por página

TEXT

INFORME: informe_concepto.pdf
 PÁGINAS TOTALES: 1
 TAMAÑO DE PÁGINA: 595 × 842 unidades de informe (A4) (A4 vertical)
 MÁRGENES: izquierdo 20, derecho 20, superior 20, inferior 20
 REGISTROS PROCESADOS: 12
 BANDAS EMITIDAS:
 - Title (1 vez)
 - Page Header (1 vez)
 - Column Header (1 vez)
 - Detail (12 veces)
 - Column Footer (1 vez)
 - Last Page Footer (1 vez, sustituye a Page Footer)
 - Summary (1 vez)
 - Background (1 vez)

— Página 1 de 1 —

Catálogo Editorial - Informe Conceptual	
Fecha de emisión: 22/09/2026	
Catálogo Editorial (cursiva)	Página 1 de 1
Título	Precio
Cien años de soledad	19,95
Rayuela	22,50
La ciudad y los perros	18,75
Pedro Páramo	15,90
Ficciones	21,00
La casa de los espíritus	23,40
El amor en los tiempos del cólera	20,80
La muerte de Artemio Cruz	17,60
Doña Bárbara	16,95
Martín Fierro	14,50
Comala	19,20
Paradiso	25,00
--- Fin de la tabla de datos ---	
Documento generado en la última página	

```

Total de páginas: 1

Fin del informe. EditorialReports.

Total de libros: 12

```

Qué representa: la página única del PDF resultante con los doce libros emitidos en la banda Detail. El informe cabe en una sola página porque los doce registros ocupan 240 unidades de informe (12×20) más las bandas fijas. La banda Last Page Footer sustituye a Page Footer porque la única página es también la última.

Cómo verificarlo: abrir el archivo `output/informe_concepto.pdf` con un lector de PDF y comprobar que aparecen los doce libros en la tabla. Si solo aparece uno o ninguno, revisar la clase `CatalogoDataSource` y la declaración de los campos en el JRXML.

D.4 — Árbol de carpetas del proyecto tras completar el punto

TEXT

```

EditorialReports/
├── ECOSISTEMA.md                (documentación del ecosistema)
├── ENTORNO.md                  (documentación del entorno)
├── BANDAS.md                   (documentación de las bandas, ampliada)
├── JRXML.md                    (documentación del formato JRXML)
├── reports/
│   ├── informe_concepto.jrxml   (plantilla con 9 bandas)
│   └── informe_concepto.jasper  (artefacto compilado)
├── resources/
│   └── (vacía en este punto)
├── output/
│   └── informe_concepto.pdf      (documento con 12 registros)
└── EditorialReportsJava/
    ├── lib/
    │   └── README.md            (el runtime real se resuelve con Maven)
    └── src/
        ├── GeneradorInformeConcepto.java    (programa actualizado)
        ├── Libro.java                       (nueva clase)
        └── CatalogoDataSource.java          (nueva clase)

```

Qué representa: el estado de los dos proyectos tras completar los doce pasos de la Parte A. La novedad respecto al punto anterior es la ampliación de la banda Summary y del Page Header en el JRXML, y la incorporación de las clases `Libro` y `CatalogoDataSource` en el proyecto Java.

Cómo verificarlo: expandir los nodos del panel Project Explorer y comparar con este esquema. Si las clases `Libro` y `CatalogoDataSource` no aparecen, repetir los pasos 3 y 4.

Errores comunes del ejercicio completo

Error	Causa	Solución
Field not found: titulo al compilar	El campo no está declarado en el JRXML	Añadir <code><field name="titulo" class="java.lang.String"/></code> antes de las bandas
La banda Detail no emite ningún registro	La fuente de datos devuelve una lista vacía	Comprobar que <code>Libro.listaEjemplo()</code> devuelve la lista de doce libros
ClassCastException en getFieldValue	El tipo devuelto por el campo no coincide con el declarado	Verificar que precio devuelve un Double y no un String
El campo precio no se formatea con dos decimales	El campo no tiene el atributo pattern	Añadir <code>pattern="#,##0.00"</code> al elemento <code>textField</code> de precio
La banda Last Page Footer no aparece	La banda no se ha añadido al JRXML o está fuera del elemento raíz	Verificar en el panel Outline que el nodo Last Page Footer existe
El pie normal aparece donde se esperaba Last Page Footer	La banda Last Page Footer no está correctamente definida	Verificar el orden XSD del JRXML y que <code>lastPageFooter</code> esté definido antes de <code>summary</code>
El contador REPORT_COUNT muestra 0	Se usó <code>\$P{REPORT_COUNT}</code> en lugar de <code>\$V{REPORT_COUNT}</code>	Cambiar el prefijo a <code>\$V{</code>
	La banda Detail tiene una altura excesiva	

Error	Causa	Solución
El informe tiene más páginas de las esperadas		Reducir el campo Band height de la banda Detail a 20 unidades de informe
El campo de paginación del Page Header muestra Página N de N incorrecto	El campo del total no está configurado con Evaluation Time = Report	Configurar el segundo Text Field con \$V{PAGE_NUMBER} y Evaluation Time = Report
Los acentos de los títulos de los libros aparecen corruptos	El archivo JRXML no está guardado en UTF-8	Guardar el archivo como UTF-8 y recompilar

Reto resuelto paso a paso

Enunciado: añadir un subtotal de precios en la banda Summary que muestre la suma de los precios de todos los libros. Verificar que el valor aparece correctamente en el PDF.

- Paso 1. Hacer doble clic sobre el archivo informe_concepto.jrxml en el panel Project Explorer.
- Paso 2. Hacer clic sobre la pestaña Source en la parte inferior del editor central.
- Paso 3. Hacer clic al final de la línea que contiene `<field name="precio" class="java.lang.Double"/>` y pulsar Enter.
- Paso 4. Escribir exactamente `<variable name="TotalPrecios" class="java.lang.Double" calculation="Sum">` y pulsar Enter.
- Paso 5. Escribir exactamente `<variableExpression><![CDATA[$F{precio}]]></variableExpression>` y pulsar Enter.
- Paso 6. Escribir exactamente `</variable>` y pulsar Enter.
- Paso 7. Pulsar Ctrl+S para guardar el archivo.
- Paso 8. Hacer clic sobre la pestaña Design en la parte inferior del editor central.
- Paso 9. Hacer clic sobre el nodo Summary en el panel Outline.
- Paso 10. Hacer clic sobre el campo Band height en el panel Properties, pestaña Properties, escribir 95 y pulsar Enter.
- Paso 11. Hacer clic sobre la pestaña Elements en el panel Palette.
- Paso 12. Hacer clic sobre el icono Static Text y arrastrarlo hasta la banda Summary, en la coordenada aproximada x=0, y=70.
- Paso 13. Hacer doble clic sobre el Static Text creado en la acción anterior y escribir exactamente Subtotal precios:.
- Paso 14. Hacer clic sobre el campo Width en el panel Properties, escribir 150 y pulsar Enter.
- Paso 15. Hacer clic sobre la pestaña Elements en el panel Palette y hacer clic sobre el icono Text Field.
- Paso 16. Arrastrar el icono Text Field hasta la banda Summary, a la derecha del rótulo, en la coordenada aproximada x=155, y=70.
- Paso 17. Hacer clic sobre el campo Text Field Expression en el panel Properties y escribir exactamente `$V{TotalPrecios}`.
- Paso 18. Hacer clic sobre el campo Pattern y escribir exactamente `#,##0.00`.
- Paso 19. Hacer clic sobre el campo Width y escribir 100. Pulsar Enter.
- Paso 20. Pulsar Ctrl+S, después Ctrl+Mayús+B para compilar.
- Paso 21. Hacer clic con el botón derecho sobre el archivo GeneradorInformeConcepto.java y seleccionar Run As > Java Application.
- Paso 22. Abrir el archivo output/informe_concepto.pdf y verificar que en la banda Summary aparece el texto Subtotal precios: seguido del valor 252,55.

Simulación ASCII del PDF tras el reto

TEXT

```

Catálogo Editorial - Informe Conceptual

Fecha de emisión: 22/09/2026

Catálogo Editorial (cursiva)          Página 1 de 1

Título                                | Precio
-----|-----
Cien años de soledad                 | 19,95
Rayuela                             | 22,50
...
Paradiso                             | 25,00

--- Fin de la tabla de datos ---

Documento generado en la última página

Total de páginas: 1
Fin del informe. EditorialReports.
Total de libros: 12

```

Subtotal precios: 252,55

Resultado del reto: la variable `TotalPrecios` acumula los precios de los doce libros mediante la propiedad `calculation="Sum"`. El valor se muestra en la banda `Summary` con formato numérico. La suma de los doce precios del ejemplo es 252,55.

Analogía final con el contexto de la editorial

Las bandas son las secciones del catálogo. La banda `Title` es la portada con el título y la fecha. La banda `Page Header` es el encabezado que se repite en cada página con el número de página. La banda `Column Header` son los títulos de las columnas de la tabla. La banda `Detail` es cada fila de la tabla, una por cada libro. La banda `Column Footer` es la línea que cierra la tabla. La banda `Page Footer` es el pie de las páginas intermedias. La banda `Last Page Footer` es el pie especial de la última página. La banda `Summary` es el colofón con los totales. La banda `Background` es el papel continuo sobre el que se imprime todo. Comprender el comportamiento de cada banda con datos reales es comprender cómo se compone el catálogo completo.

Resultado esperado

- Al finalizar este punto, el alumno dispone de:
- El archivo `reports/informe_concepto.jrxml` con nueve bandas, dos campos declarados y una variable de suma.
- Las clases `Libro.java` y `CatalogoDataSource.java` en el proyecto Java.
- El programa `GeneradorInformeConcepto.java` modificado para usar `CatalogoDataSource` con doce libros.
- El archivo `output/informe_concepto.pdf` con doce registros emitidos y los totales calculados.
- Comprensión operativa del comportamiento de cada banda con una fuente de datos real.
- El archivo `BANDAS.md` ampliado con la documentación del comportamiento con datos.

Conclusión y enlace al siguiente punto

El punto 2.1 ha profundizado en el modelo de bandas del motor y ha demostrado el comportamiento de cada una con una fuente de datos que devuelve doce registros. Han quedado añadidas la banda `Last Page Footer` y la variable de suma `TotalPrecios`. El informe conceptual contiene ahora nueve bandas y muestra los doce libros del catálogo con sus precios y los totales al final.

El punto 2.2, «Texto estático y campos de texto», profundiza en los dos elementos textuales del informe, detalla sus propiedades avanzadas y explica cómo combinarlos para construir encabezados, etiquetas y valores dinámicos. El informe construido en este punto sirve como base para los nuevos elementos textuales.

Punto 2.2 — Texto estático y campos de texto

PUNTO DE PARTIDA. Si vienes haciendo el curso, continúa con tu propio proyecto del punto anterior. Si te incorporas directamente aquí, usa M2/2.1 como estado inicial. El checkpoint M2/2.2 contiene la solución completa de este punto y no debe consultarse antes del ejercicio si quieres evitar spoilers.

Parte A — Práctica visual

Paso 1: Abrir el informe y la banda Detail VALIDADO

Acciones:

1. Hacer clic con el botón derecho sobre el nodo EditorialReports en el panel Project Explorer (superior izquierdo).
2. Hacer clic sobre la opción Refresh en el menú contextual.
3. Hacer clic con el botón derecho sobre la carpeta reports en el panel Project Explorer.
4. Hacer clic sobre la opción Refresh en el menú contextual.
5. Hacer doble clic sobre el archivo informe_concepto.jrxml en el panel Project Explorer.
6. Hacer clic sobre la pestaña Design en la parte inferior del editor central.
7. Expandir el nodo informe_concepto en el panel Outline (inferior izquierdo).
8. Hacer clic sobre el nodo Detail 1 en el panel Outline.

Verificación visual: el editor central muestra la banda Detail 1 seleccionada con sus dos textField. El panel Properties (inferior derecho) muestra las propiedades del nodo Detail 1.

Qué hace: abre el informe y selecciona la banda Detail para trabajar sobre ella.

Por qué: la banda Detail es la que contiene los campos variables del informe y es donde se aplican las propiedades específicas del campo de texto.

Error común: abrir el archivo en la vista Source en lugar de Design.

Solución: hacer clic sobre la pestaña Design en la parte inferior del editor central.

Analogía: es como abrir el pliego del catálogo en la sección de la tabla de datos para ajustar la presentación de cada fila.

Paso 2: Aplicar formato numérico con símbolo de euro al campo precio VALIDADO

Acciones:

1. Hacer clic sobre el segundo textField de la banda Detail 1 en el editor central.
2. Hacer clic sobre el campo Pattern en el panel Properties (inferior derecho), pestaña Properties.
3. Seleccionar el contenido actual del campo y eliminarlo con la tecla Suprimir.
4. Escribir exactamente ###0.00 € y pulsar Enter.
5. Hacer clic sobre la pestaña Source en la parte inferior del editor central.
6. Hacer clic sobre el atributo pattern del textField del campo precio y verificar que contiene el valor ###0.00 €.

Verificación visual: el editor central muestra el campo con el patrón actualizado. En la vista Source, el atributo pattern contiene ###0.00 €.

Qué hace: aplica un patrón numérico que incluye el símbolo del euro al campo de precio.

Por qué: el formato del precio es más legible cuando incluye el símbolo de la moneda.

Error común: escribir el patrón sin el espacio entre 0.00 y €. El resultado es 19,95€ sin espacio.

Solución: escribir el patrón con el espacio incluido: #,##0.00 €.

Analogía: es como añadir el símbolo de la moneda a los precios del catálogo para que el lector los identifique de inmediato.

Paso 3: Activar la propiedad isBlankWhenNull en el campo precio VALIDADO

Acciones:

1. Hacer clic sobre la pestaña Design en la parte inferior del editor central.
2. Hacer clic sobre el segundo textField de la banda Detail 1 en el editor central.
3. Expandir la sección Text Field en el panel Properties (inferior derecho), pestaña Properties.
4. Marcar la casilla Blank When Null.
5. Hacer clic sobre la pestaña Source en la parte inferior del editor central.
6. Hacer clic sobre la etiqueta <textField> del campo precio y verificar que contiene el atributo isBlankWhenNull="true".

Verificación visual: el editor central muestra el campo con la propiedad activada. En la vista Source, la etiqueta <textField> incluye el atributo isBlankWhenNull="true".

Qué hace: activa la propiedad que hace que el campo se muestre vacío cuando la expresión devuelve null.

Por qué: si un libro no tiene precio asignado, el campo debe aparecer vacío en lugar de mostrar el texto null o producir un error de formato.

Error común: dejar la propiedad desactivada y obtener una representación no deseada de un valor nulo, según el tipo y el formateador.

Solución: marcar la casilla Blank When Null.

Analogía: es como dejar en blanco la casilla del precio de un libro sin precio asignado en lugar de imprimir un guion o un error.

Paso 4: Configurar el ajuste de texto del campo título VALIDADO

Acciones:

1. Hacer clic sobre el primer textField de la banda Detail 1 en el editor central.
2. Expandir la sección Text Field en el panel Properties (inferior derecho), pestaña Properties.
3. Seleccionar StretchHeight en la propiedad de ajuste de texto. Si la interfaz 6.20.0 muestra la opción heredada Stretch With Overflow, activarla produce el mismo comportamiento; el JRXML validado usa textAdjust="StretchHeight".
4. Hacer clic sobre la pestaña Source en la parte inferior del editor central.
5. Hacer clic sobre la etiqueta <textField> del campo título y verificar que contiene el atributo textAdjust="StretchHeight".

Verificación visual: el editor central muestra el campo con la propiedad activada. En la vista Source, la etiqueta <textField> incluye textAdjust="StretchHeight".

Qué hace: configura el ajuste de texto para que el campo pueda aumentar de altura cuando el contenido no cabe.

Por qué: los títulos de los libros pueden ser largos y es necesario que se ajusten en varias líneas sin recortarse.

Error común: dejar la propiedad desactivada y provocar que los títulos largos se recorten.

Solución: marcar la casilla Stretch With Overflow.

Analogía: es como permitir que el título de un libro ocupe varias líneas en la tabla del catálogo cuando es largo.

Paso 5: Compactar Título/Precio y añadir el rótulo del número de registro VALIDADO

Acciones:

1. Hacer clic sobre el campo \$F{título} de Detail y escribir X=0, Y=0, Width=330, Height=20.

2. Hacer clic sobre el campo `$F{precio}` y escribir `X=330, Y=0, Width=100, Height=20`.
3. Hacer clic sobre `Palette > Elements > Static Text`.
4. Arrastrar el `Static Text` a `Detail` y escribir exactamente `#`.
5. En `Properties` escribir `X=440, Y=0, Width=20, Height=20`.
6. Seleccionar alineación horizontal `Right`.
7. Pulsar `Ctrl+S`.

Verificación visual: título ocupa 0-330, precio 330-430 y el rótulo `#` ocupa 440-460, dejando espacio a la derecha para el contador.

Qué hace: compacta la fila para reservar una zona estable al número de registro.

Por qué: el checkpoint 2.2 debe permanecer dentro de `columnWidth="555"` y servir de base geométrica para 2.3.

Error común: conservar Título con 300 y colocar `#` en `X=400`, solapándolo con Precio. **Solución:** usar exactamente las posiciones 0/330, 330/100 y 440/20.

Analogía: es como redistribuir las columnas de una tabla antes de añadir una columna final de numeración.

Paso 6: Añadir el campo con el número de registro VALIDADO

Acciones:

1. Hacer clic sobre `Palette > Elements > Text Field`.
2. Arrastrar el campo a `Detail`, a la derecha del rótulo `#`.
3. En `Properties` escribir `X=465, Y=0, Width=40, Height=20`.
4. En `Text Field Expression` escribir exactamente `$V{REPORT_COUNT}`.
5. Seleccionar alineación horizontal `Right`.
6. Pulsar `Ctrl+S`.

Verificación visual: el contador ocupa la franja 465-505 y no se solapa con el precio ni con el rótulo `#`.

Qué hace: muestra el número correlativo del registro actual.

Por qué: `REPORT_COUNT` se incrementa a medida que el motor procesa los registros.

Error común: usar `X=430, Width=30` heredados de un borrador anterior. **Solución:** usar `X=465, Width=40`, que son las coordenadas del checkpoint validado.

Analogía: es como numerar las filas del catálogo en una columna reservada al margen derecho.

Paso 7: Añadir un rótulo estático con estilo en el Page Header VALIDADO

Acciones:

1. Hacer clic sobre el nodo `Page Header` en el panel `Outline` (inferior izquierdo).
2. Hacer clic sobre la pestaña `Elements` en el panel `Palette` (derecha del editor central).
3. Hacer clic sobre el icono `Static Text` (una letra `T` mayúscula).
4. Arrastrar el icono `Static Text` y soltarlo dentro de la banda `Page Header`, en la coordenada aproximada `x=0, y=20`.
5. Hacer doble clic sobre el `Static Text` creado en la acción anterior.
6. Escribir exactamente Precio en `<b>euros</b>` con IVA incluido.
7. Hacer clic sobre una zona vacía del editor central para confirmar el texto.
8. Hacer clic sobre el campo `Width` en el panel `Properties`, pestaña `Properties`, escribir 555 y pulsar `Enter`.
9. Hacer clic sobre el campo `Height`, escribir 15 y pulsar `Enter`.
10. Hacer clic sobre el campo `X`, escribir 0 y pulsar `Enter`.
11. Hacer clic sobre el campo `Y`, escribir 20 y pulsar `Enter`.
12. Hacer clic sobre la pestaña `Properties` y marcar la casilla `Styled Text`.
13. Hacer clic sobre el campo `Font size` y escribir 8. Pulsar `Enter`.

Verificación visual: la banda Page Header muestra un segundo rótulo con el texto Precio en euros con IVA incluido con la palabra euros en negrita.

Qué hace: inserta un texto con etiquetas de estilo interpretadas por el motor.

Por qué: el estilo permite destacar una palabra dentro de un texto más largo.

Error común: olvidar marcar la casilla Styled Text y provocar que las etiquetas se impriman como texto literal.

Solución: marcar la casilla Styled Text en el panel Properties.

Analogía: es como usar la negrita en una nota del catálogo para destacar una palabra importante.

Paso 8: Ajustar la altura de la banda Page Header VALIDADO

Acciones:

1. Hacer clic sobre el nodo Page Header en el panel Outline (inferior izquierdo).
2. Hacer clic sobre el campo Band height en el panel Properties, pestaña Properties.
3. Escribir 40 y pulsar Enter.
4. Hacer clic sobre el campo Split Type en el panel Properties y seleccionar Prevent.

Verificación visual: la banda Page Header aparece con 40 unidades de informe de altura y la propiedad Split Type ajustada a Prevent.

Qué hace: amplía la altura de la banda de cabecera y configura Prevent, que intenta evitar el primer corte de la banda cuando no cabe en el espacio restante.

Por qué: el rótulo adicional ocupa espacio y la banda debe crecer para alojarlo sin recortar el contenido existente.

Error común: olvidar ajustar la altura y provocar que el rótulo se solape con la banda Column Header.

Solución: ampliar la altura a 40 unidades de informe.

Analogía: es como ampliar la franja del encabezado del catálogo para que quepan la nota y el título abreviado.

Paso 9: Añadir un pie de tabla con el número total de registros VALIDADO

Acciones:

1. Hacer clic sobre el nodo Column Footer en el panel Outline (inferior izquierdo).
2. Hacer clic sobre el campo Band height en el panel Properties, pestaña Properties, escribir 40 y pulsar Enter.
3. Hacer clic sobre la pestaña Elements en el panel Palette (derecha del editor central).
4. Hacer clic sobre el icono Static Text (una letra T mayúscula).
5. Arrastrar el icono Static Text y soltarlo dentro de la banda Column Footer, en la coordenada aproximada x=0, y=20.
6. Hacer doble clic sobre el Static Text creado en la acción anterior.
7. Escribir exactamente Registros procesados: (con espacio al final).
8. Hacer clic sobre una zona vacía del editor central para confirmar el texto.
9. Hacer clic sobre el campo Width en el panel Properties, pestaña Properties, escribir 150 y pulsar Enter.
10. Hacer clic sobre el campo Height, escribir 15 y pulsar Enter.
11. Hacer clic sobre el campo X, escribir 0 y pulsar Enter.
12. Hacer clic sobre el campo Y, escribir 20 y pulsar Enter.

Verificación visual: la banda Column Footer muestra un segundo rótulo Registros procesados: debajo del texto separador.

Qué hace: inserta un rótulo que precede al número total de registros procesados.

Por qué: el recuento total de registros es un dato agregado que se muestra al final de la tabla.

Error común: dejar el rótulo sin el espacio al final y provocar que el número quede pegado.

Solución: incluir el espacio en el texto del rótulo.

Analogía: es como anotar en la tabla del catálogo cuántos libros se han maquetado.

Paso 10: Añadir el campo con el recuento total de registros VALIDADO

Acciones:

1. Hacer clic sobre la pestaña Elements en el panel Palette (derecha del editor central).
2. Hacer clic sobre el icono Text Field (una letra F dentro de un cuadrado).
3. Arrastrar el icono Text Field y soltarlo dentro de la banda Column Footer, en la coordenada aproximada x=150, y=20.
4. Hacer clic sobre el campo Text Field Expression en el panel Properties, pestaña Properties.
5. Escribir exactamente `$V{REPORT_COUNT}` y pulsar Enter.
6. Hacer clic sobre el campo Width, escribir 100 y pulsar Enter.
7. Hacer clic sobre el campo Height, escribir 15 y pulsar Enter.
8. Hacer clic sobre el campo X, escribir 150 y pulsar Enter.
9. Hacer clic sobre el campo Y, escribir 20 y pulsar Enter.
10. Hacer clic sobre el campo Font size y escribir 9. Pulsar Enter.
11. Marcar la casilla Bold.

Verificación visual: la banda Column Footer muestra el rótulo Registros procesados: seguido de un campo con la expresión `$V{REPORT_COUNT}` en negrita.

Qué hace: inserta un campo que muestra el número total de registros procesados.

Por qué: el recuento total es un valor agregado que se calcula al final del llenado.

Error común: usar `$P{REPORT_COUNT}` en lugar de `$V{REPORT_COUNT}`. El compilador informa Parameter not found: REPORT_COUNT.

Solución: cambiar el prefijo a `$V{`.

Analogía: es como escribir el número total de libros en el pie de la tabla del catálogo.

Paso 11: Compilar y ejecutar el programa Java VALIDADO

Acciones:

1. Pulsar Ctrl+S para guardar el archivo JRXML.
2. Pulsar Ctrl+Mayús+B para compilar el informe.
3. Hacer clic sobre el panel Problems (inferior) y verificar que no hay errores.
4. Hacer clic con el botón derecho sobre el archivo GeneradorInformeConcepto.java en el panel Project Explorer.
5. Hacer clic sobre la opción Run As en el menú contextual.
6. Hacer clic sobre la opción Java Application en el submenú.
7. Hacer clic sobre la vista Console en el panel inferior y observar el resultado.

Verificación visual: la vista Console muestra la línea Informe generado en: ... con la ruta absoluta del archivo PDF. El panel Problems permanece vacío.

Qué hace: compila el informe y ejecuta el programa Java con la fuente de datos de los doce libros.

Por qué: la ejecución confirma que las propiedades de los elementos textuales funcionan correctamente.

Error común: olvidar compilar el informe después de modificar el JRXML y obtener un PDF con la versión anterior.

Solución: pulsar Ctrl+Mayús+B antes de ejecutar el programa.

Analogía: es como imprimir la tirada del catálogo con los ajustes tipográficos aplicados.

Paso 12: Documentar las propiedades de los elementos textuales VALIDADO

Acciones:

1. Hacer clic con el botón derecho sobre el nodo EditorialReports en el panel Project Explorer (superior izquierdo).
2. Hacer clic sobre la opción New en el menú contextual.
3. Hacer clic sobre la opción File en el submenú.
4. Escribir exactamente TEXTO.md en el campo File name del diálogo.
5. Hacer clic sobre el botón Finish.
6. En el editor central, escribir exactamente # Propiedades de elementos textuales y pulsar Enter dos veces.
7. Escribir exactamente ## Propiedades comunes y pulsar Enter dos veces.
8. Escribir exactamente - reportElement: x, y, width, height, forecolor, backcolor, mode e isRemoveLineWhenBlank. y pulsar Enter.
9. Escribir exactamente - textElement: textAlignment, verticalAlignment, rotation, markup y font. y pulsar Enter dos veces.
10. Escribir exactamente ## Propiedades específicas de textField y pulsar Enter dos veces.
11. Escribir exactamente - textAdjust="StretchHeight": permite que el campo aumente de altura para mostrar todo el contenido. y pulsar Enter.
12. Escribir exactamente - isBlankWhenNull: muestra el campo vacío cuando la expresión devuelve null. y pulsar Enter.
13. Escribir exactamente - pattern: aplica formato a fechas y números. y pulsar Enter.
14. Pulsar Ctrl+S para guardar el archivo.

Verificación visual: el panel Project Explorer muestra el archivo TEXTO.md en la raíz del proyecto EditorialReports con las propiedades documentadas.

Qué hace: incorpora al proyecto un documento que registra las propiedades de los elementos textuales.

Por qué: la documentación de las propiedades facilita el mantenimiento y la consulta rápida.

Error común: escribir el nombre del archivo con extensión distinta a .md.

Solución: usar exactamente TEXTO.md.

Analogía: es como dejar en la editorial una ficha técnica con las propiedades tipográficas del catálogo.

Parte B — JRXML explicado y contrastado COMPLETADO

Se reproducen las tres secciones modificadas por 2.2 **exactamente como quedan en el checkpoint ejecutable**. El resto del JRXML se hereda de 2.1.

XML

```
<pageHeader>
  <band height="40" splitType="Prevent">
    <staticText>
      <reportElement x="0" y="5" width="330" height="15" uuid="a1b2c3d4-e5f6-7a8b-9c0d-1e2f3a4b5c6d"/>
      <textElement verticalAlignment="Middle"><font size="9" isItalic="true"/></textElement>
      <text><![CDATA[Catálogo Editorial - Informe Conceptual]]></text>
    </staticText>
    <textField>
      <reportElement x="330" y="5" width="170" height="15" uuid="b2c3d4e5-f6a7-8b9c-0d1e-2f3a4b5c6d7e"/>
      <textElement textAlignment="Right" verticalAlignment="Middle"><font size="9" isItalic="true"/></textElement>
      <textFieldExpression><![CDATA["Página " + ${PAGE_NUMBER} + " de"]></textFieldExpression>
    </textField>
    <textField evaluationTime="Report">
      <reportElement x="500" y="5" width="55" height="15" uuid="f9ab1a6c-59d2-5c71-a85f-72cee0efbcfb"/>
      <textElement textAlignment="Right" verticalAlignment="Middle"><font size="9" isItalic="true"/></textElement>
      <textFieldExpression><![CDATA[${PAGE_NUMBER}]]></textFieldExpression>
    </textField>
  </band>
</pageHeader>
```

```

        </textField>
        <staticText>
            <reportElement x="0" y="20" width="555" height="15" uuid="c3d4e5f6-a7b8-9c0d-1e2f-3a4b5c6d7e8f"/>
            <textElement verticalAlignment="Middle" markup="styled"><font size="8"/></textElement>
            <text><![CDATA[Precio en <b>euros</b> con IVA incluido]]></text>
        </staticText>
    </band>
</pageHeader>

<detail>
    <band height="20" splitType="Stretch">
        <textField textAdjust="StretchHeight">
            <reportElement x="0" y="0" width="330" height="20" uuid="d4e5f6a7-b8c9-0d1e-2f3a-4b5c6d7e8f9a"/>
            <textFieldExpression><![CDATA[${titulo}]]></textFieldExpression>
        </textField>
        <textField pattern="#,##0.00 €" isBlankWhenNull="true">
            <reportElement x="330" y="0" width="100" height="20" uuid="e5f6a7b8-c9d0-1e2f-3a4b-5c6d7e8f9a0b"/>
            <textFieldExpression><![CDATA[${precio}]]></textFieldExpression>
        </textField>
        <staticText>
            <reportElement x="440" y="0" width="20" height="20" uuid="f6a7b8c9-d0e1-2f3a-4b5c-6d7e8f9a0b1c"/>
            <textElement textAlignment="Right" verticalAlignment="Middle"><font size="9"/></textElement>
            <text><![CDATA[#]]></text>
        </staticText>
        <textField>
            <reportElement x="465" y="0" width="40" height="20" uuid="a7b8c9d0-e1f2-3a4b-5c6d-7e8f9a0b1c2d"/>
            <textElement textAlignment="Right" verticalAlignment="Middle"><font size="9"/></textElement>
            <textFieldExpression><![CDATA[${REPORT_COUNT}]]></textFieldExpression>
        </textField>
    </band>
</detail>

<columnFooter>
    <band height="40">
        <staticText>
            <reportElement x="0" y="5" width="555" height="15" uuid="b8c9d0e1-f2a3-4b5c-6d7e-8f9a0b1c2d3e"/>
            <textElement textAlignment="Center" verticalAlignment="Middle"><font size="9" isItalic="true"/></textElement>
            <text><![CDATA[--- Fin de la tabla de datos ---]]></text>
        </staticText>
        <staticText>
            <reportElement x="0" y="20" width="150" height="15" uuid="c9d0e1f2-a3b4-5c6d-7e8f-9a0b1c2d3e4f"/>
            <textElement verticalAlignment="Middle"><font size="9"/></textElement>
            <text><![CDATA[Registros procesados:]]></text>
        </staticText>
        <textField>
            <reportElement x="150" y="20" width="100" height="15" uuid="d0e1f2a3-b4c5-6d7e-8f9a-0b1c2d3e4f5a"/>
            <textElement verticalAlignment="Middle"><font size="9" isBold="true"/></textElement>
            <textFieldExpression><![CDATA[${REPORT_COUNT}]]></textFieldExpression>
        </textField>
    </band>
</columnFooter>

```

Explicación línea por línea

Línea 1	<pageHeader> → abre Page Header, generado al comienzo de cada página.
Línea 2	<band height="40" splitType="Prevent"> → define la banda y su altura; si aparece splitType, establece la política de división.
Línea 3	<staticText> → abre un texto estático.
Línea 4	<reportElement x="0" y="5" width="330" height="15" uuid="a1b2c3d4-e5f6-7a8b-9c0d-1e2f3a4b5c6d"/> → fija coordenadas, tamaño, UUID y, cuando existe, el estilo del elemento.
Línea 5	<textElement verticalAlignment="Middle"></textElement> → configura alineación y/o marcado del texto.
Línea 6	<text><![CDATA[Catálogo Editorial - Informe Conceptual]]></text> → define el contenido literal del elemento estático.
Línea 7	</staticText> → cierra el elemento o sección abierto correspondiente.
Línea 8	<textField> → abre un campo de texto dinámico; sus atributos controlan evaluación, formato o nulos.
Línea 9	<reportElement x="330" y="5" width="170" height="15" uuid="b2c3d4e5-f6a7-8b9c-0d1e-2f3a4b5c6d7e"/> → fija coordenadas, tamaño, UUID y, cuando existe, el estilo del elemento.
Línea 10	<textElement textAlignment="Right" verticalAlignment="Middle"></textElement> → configura alineación y/o marcado del texto.

Línea 11	<code><textFieldExpression><![CDATA["Página " + \${PAGE_NUMBER} + " de"]]></textFieldExpression></code> → abre un campo de texto dinámico; sus atributos controlan evaluación, formato o nulos.
Línea 12	<code></textField></code> → cierra el elemento o sección abierto correspondiente.
Línea 13	<code><textField evaluationTime="Report"></code> → abre un campo de texto dinámico; sus atributos controlan evaluación, formato o nulos.
Línea 14	<code><reportElement x="500" y="5" width="55" height="15" uuid="f9ab1a6c-59d2-5c71-a85f-72cee0efbcfb"/></code> → fija coordenadas, tamaño, UUID y, cuando existe, el estilo del elemento.
Línea 15	<code><textElement textAlignment="Right" verticalAlignment="Middle"></textElement></code> → configura alineación y/o marcado del texto.
Línea 16	<code><textFieldExpression><![CDATA[\${PAGE_NUMBER}]]></textFieldExpression></code> → abre un campo de texto dinámico; sus atributos controlan evaluación, formato o nulos.
Línea 17	<code></textField></code> → cierra el elemento o sección abierto correspondiente.
Línea 18	<code><staticText></code> → abre un texto estático.
Línea 19	<code><reportElement x="0" y="20" width="555" height="15" uuid="c3d4e5f6-a7b8-9c0d-1e2f-3a4b5c6d7e8f"/></code> → fija coordenadas, tamaño, UUID y, cuando existe, el estilo del elemento.
Línea 20	<code><textElement verticalAlignment="Middle" markup="styled"></textElement></code> → configura alineación y/o marcado del texto.
Línea 21	<code><text><![CDATA[Precio en euros con IVA incluido]]></text></code> → define el contenido literal del elemento estático.
Línea 22	<code></staticText></code> → cierra el elemento o sección abierto correspondiente.
Línea 23	<code></band></code> → cierra el elemento o sección abierto correspondiente.
Línea 24	<code></pageHeader></code> → cierra el elemento o sección abierto correspondiente.
Línea 26	<code><detail></code> → abre Detail, que el motor intenta generar por cada registro.
Línea 27	<code><band height="20" splitType="Stretch"></code> → define la banda y su altura; si aparece splitType, establece la política de división.
Línea 28	<code><textField textAdjust="StretchHeight"></code> → abre un campo de texto dinámico; sus atributos controlan evaluación, formato o nulos.
Línea 29	<code><reportElement x="0" y="0" width="330" height="20" uuid="d4e5f6a7-b8c9-0d1e-2f3a-4b5c6d7e8f9a"/></code> → fija coordenadas, tamaño, UUID y, cuando existe, el estilo del elemento.
Línea 30	<code><textFieldExpression><![CDATA[\${F{titulo}}]]></textFieldExpression></code> → abre un campo de texto dinámico; sus atributos controlan evaluación, formato o nulos.
Línea 31	<code></textField></code> → cierra el elemento o sección abierto correspondiente.
Línea 32	<code><textField pattern="# ,##0.00 €" isBlankWhenNull="true"></code> → abre un campo de texto dinámico; sus atributos controlan evaluación, formato o nulos.
Línea 33	<code><reportElement x="330" y="0" width="100" height="20" uuid="e5f6a7b8-c9d0-1e2f-3a4b-5c6d7e8f9a0b"/></code> → fija coordenadas, tamaño, UUID y, cuando existe, el estilo del elemento.
Línea 34	<code><textFieldExpression><![CDATA[\${F{precio}}]]></textFieldExpression></code> → abre un campo de texto dinámico; sus atributos controlan evaluación, formato o nulos.
Línea 35	<code></textField></code> → cierra el elemento o sección abierto correspondiente.
Línea 36	<code><staticText></code> → abre un texto estático.
Línea 37	<code><reportElement x="440" y="0" width="20" height="20" uuid="f6a7b8c9-d0e1-2f3a-4b5c-6d7e8f9a0b1c"/></code> → fija coordenadas, tamaño, UUID y, cuando existe, el estilo del elemento.
Línea 38	<code><textElement textAlignment="Right" verticalAlignment="Middle"></textElement></code> → configura alineación y/o marcado del texto.

Línea 39	<code><text><![CDATA[#]]></text></code> → define el contenido literal del elemento estático.
Línea 40	<code></staticText></code> → cierra el elemento o sección abierto correspondiente.
Línea 41	<code><textField></code> → abre un campo de texto dinámico; sus atributos controlan evaluación, formato o nulos.
Línea 42	<code><reportElement x="465" y="0" width="40" height="20" uuid="a7b8c9d0-e1f2-3a4b-5c6d-7e8f9a0b1c2d"/></code> → fija coordenadas, tamaño, UUID y, cuando existe, el estilo del elemento.
Línea 43	<code><textElement textAlignment="Right" verticalAlignment="Middle"></textElement></code> → configura alineación y/o marcado del texto.
Línea 44	<code><textFieldExpression><![CDATA[\${REPORT_COUNT}]]></textFieldExpression></code> → abre un campo de texto dinámico; sus atributos controlan evaluación, formato o nulos.
Línea 45	<code></textField></code> → cierra el elemento o sección abierto correspondiente.
Línea 46	<code></band></code> → cierra el elemento o sección abierto correspondiente.
Línea 47	<code></detail></code> → cierra el elemento o sección abierto correspondiente.
Línea 49	<code><columnFooter></code> → abre Column Footer.
Línea 50	<code><band height="40"></code> → define la banda y su altura; si aparece <code>splitType</code> , establece la política de división.
Línea 51	<code><staticText></code> → abre un texto estático.
Línea 52	<code><reportElement x="0" y="5" width="555" height="15" uuid="b8c9d0e1-f2a3-4b5c-6d7e-8f9a0b1c2d3e"/></code> → fija coordenadas, tamaño, UUID y, cuando existe, el estilo del elemento.
Línea 53	<code><textElement textAlignment="Center" verticalAlignment="Middle"></textElement></code> → configura alineación y/o marcado del texto.
Línea 54	<code><text><![CDATA[--- Fin de la tabla de datos ---]]></text></code> → define el contenido literal del elemento estático.
Línea 55	<code></staticText></code> → cierra el elemento o sección abierto correspondiente.
Línea 56	<code><staticText></code> → abre un texto estático.
Línea 57	<code><reportElement x="0" y="20" width="150" height="15" uuid="c9d0e1f2-a3b4-5c6d-7e8f-9a0b1c2d3e4f"/></code> → fija coordenadas, tamaño, UUID y, cuando existe, el estilo del elemento.
Línea 58	<code><textElement verticalAlignment="Middle"></textElement></code> → configura alineación y/o marcado del texto.
Línea 59	<code><text><![CDATA[Registros procesados:]]></text></code> → define el contenido literal del elemento estático.
Línea 60	<code></staticText></code> → cierra el elemento o sección abierto correspondiente.
Línea 61	<code><textField></code> → abre un campo de texto dinámico; sus atributos controlan evaluación, formato o nulos.
Línea 62	<code><reportElement x="150" y="20" width="100" height="15" uuid="d0e1f2a3-b4c5-6d7e-8f9a-0b1c2d3e4f5a"/></code> → fija coordenadas, tamaño, UUID y, cuando existe, el estilo del elemento.
Línea 63	<code><textElement verticalAlignment="Middle"></textElement></code> → configura alineación y/o marcado del texto.
Línea 64	<code><textFieldExpression><![CDATA[\${REPORT_COUNT}]]></textFieldExpression></code> → abre un campo de texto dinámico; sus atributos controlan evaluación, formato o nulos.
Línea 65	<code></textField></code> → cierra el elemento o sección abierto correspondiente.
Línea 66	<code></band></code> → cierra el elemento o sección abierto correspondiente.
Línea 67	<code></columnFooter></code> → cierra el elemento o sección abierto correspondiente.

Parte C — Código Java explicado línea por línea COMPLETADO

2.2 modifica el JRXML, no la lógica Java. Se reproduce la misma versión Java validada que hereda del checkpoint 2.1.

Los tres archivos siguientes se reproducen **literalmente desde el checkpoint ejecutable M2/2.2**. De este modo, la Parte C coincide con el código que compila y se ejecuta en la validación end-to-end.

Clase Libro.java

JAVA

```
import java.util.ArrayList;
import java.util.List;

public class Libro {
    private final String titulo;
    private final Double precio;

    public Libro(String titulo, Double precio) {
        this.titulo = titulo;
        this.precio = precio;
    }

    public String getTitulo() {
        return titulo;
    }

    public Double getPrecio() {
        return precio;
    }

    public static List<Libro> listaEjemplo() {
        List<Libro> libros = new ArrayList<Libro>();
        libros.add(new Libro("Cien años de soledad", 19.95));
        libros.add(new Libro("Rayuela", 22.50));
        libros.add(new Libro("La ciudad y los perros", 18.75));
        libros.add(new Libro("Pedro Páramo", 15.90));
        libros.add(new Libro("Ficciones", 21.00));
        libros.add(new Libro("La casa de los espíritus", 23.40));
        libros.add(new Libro("El amor en los tiempos del cólera", 20.80));
        libros.add(new Libro("La muerte de Artemio Cruz", 17.60));
        libros.add(new Libro("Doña Bárbara", 16.95));
        libros.add(new Libro("Martín Fierro", 14.50));
        libros.add(new Libro("Comala", 19.20));
        libros.add(new Libro("Paradiso", 25.00));
        return libros;
    }
}
```

Explicación línea por línea

Línea 1	<code>import java.util.ArrayList;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 2	<code>import java.util.List;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 4	<code>public class Libro {</code> → declara la clase Java y, si aparece <code>implements</code> , establece el contrato que debe implementar.
Línea 5	<code>private final String titulo;</code> → declara un campo de instancia inmutable después de la construcción del objeto.
Línea 6	<code>private final Double precio;</code> → declara un campo de instancia inmutable después de la construcción del objeto.
Línea 8	<code>public Libro(String titulo, Double precio) {</code> → declara el constructor completo del modelo <code>Libro</code> con los valores que necesita el informe.
Línea 9	<code>this.titulo = titulo;</code> → asigna al campo del objeto el valor recibido o calculado.
Línea 10	<code>this.precio = precio;</code> → asigna al campo del objeto el valor recibido o calculado.
Línea 11	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.
Línea 13	<code>public String getTitulo() {</code> → devuelve el título del libro.
Línea 14	<code>return titulo;</code> → forma parte del bloque Java reproducido literalmente desde el checkpoint ejecutable.
Línea 15	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.

Línea 17	<code>public Double getPrecio() {</code> → devuelve el precio del libro.
Línea 18	<code>return precio;</code> → forma parte del bloque Java reproducido literalmente desde el checkpoint ejecutable.
Línea 19	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.
Línea 21	<code>public static List<Libro> listaEjemplo() {</code> → abre el método que construye los datos de ejemplo del curso.
Línea 22	<code>List<Libro> libros = new ArrayList<Libro>();</code> → crea una lista tipada compatible con Java 8 y con la baseline del proyecto.
Línea 23	<code>libros.add(new Libro("Cien años de soledad", 19.95));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.
Línea 24	<code>libros.add(new Libro("Rayuela", 22.50));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.
Línea 25	<code>libros.add(new Libro("La ciudad y los perros", 18.75));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.
Línea 26	<code>libros.add(new Libro("Pedro Páramo", 15.90));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.
Línea 27	<code>libros.add(new Libro("Ficciones", 21.00));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.
Línea 28	<code>libros.add(new Libro("La casa de los espíritus", 23.40));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.
Línea 29	<code>libros.add(new Libro("El amor en los tiempos del cólera", 20.80));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.
Línea 30	<code>libros.add(new Libro("La muerte de Artemio Cruz", 17.60));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.
Línea 31	<code>libros.add(new Libro("Doña Bárbara", 16.95));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.
Línea 32	<code>libros.add(new Libro("Martín Fierro", 14.50));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.
Línea 33	<code>libros.add(new Libro("Comala", 19.20));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.
Línea 34	<code>libros.add(new Libro("Paradiso", 25.00));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.
Línea 35	<code>return libros;</code> → devuelve la lista completa que alimentará la fuente de datos.
Línea 36	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.
Línea 37	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.

Clase CatalogoDataSource.java

JAVA

```
import java.util.List;
import net.sf.jasperreports.engine.JRDataSource;
import net.sf.jasperreports.engine.JRException;
import net.sf.jasperreports.engine.JRField;

public class CatalogoDataSource implements JRDataSource {
    private final List<Libro> libros;
    private int indice = -1;

    public CatalogoDataSource(List<Libro> libros) {
        this.libros = libros;
    }

    @Override
    public boolean next() throws JRException {
```

```

        indice++;
        return indice < libros.size();
    }

    @Override
    public Object getFieldValue(JRField campo) throws JRException {
        Libro actual = libros.get(indice);
        if ("titulo".equals(campo.getName())) {
            return actual.getTitulo();
        }
        if ("precio".equals(campo.getName())) {
            return actual.getPrecio();
        }
        throw new JRException("Campo no soportado por CatalogoDataSource: " + campo.getName());
    }
}

```

Explicación línea por línea

Línea 1	<code>import java.util.List;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 2	<code>import net.sf.jasperreports.engine.JRDataSource;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 3	<code>import net.sf.jasperreports.engine.JRException;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 4	<code>import net.sf.jasperreports.engine.JRField;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 6	<code>public class CatalogoDataSource implements JRDataSource {</code> → declara la clase Java y, si aparece <code>implements</code> , establece el contrato que debe implementar.
Línea 7	<code>private final List<Libro> libros;</code> → declara un campo de instancia inmutable después de la construcción del objeto.
Línea 8	<code>private int indice = -1;</code> → declara el índice interno de la fuente de datos; empieza en -1 porque <code>next()</code> se invoca antes de leer el primer registro.
Línea 10	<code>public CatalogoDataSource(List<Libro> libros) {</code> → declara el constructor de la fuente de datos y recibe la lista de libros.
Línea 11	<code>this.libros = libros;</code> → asigna al campo del objeto el valor recibido o calculado.
Línea 12	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.
Línea 14	<code>@Override</code> → indica que el método implementa un método del contrato <code>JRDataSource</code> .
Línea 15	<code>public boolean next() throws JRException {</code> → implementa <code>JRDataSource.next()</code> y declara <code>JRException</code> según el contrato de JasperReports.
Línea 16	<code>indice++;</code> → avanza al siguiente registro.
Línea 17	<code>return indice < libros.size();</code> → indica al motor si todavía existe un registro válido.
Línea 18	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.
Línea 20	<code>@Override</code> → indica que el método implementa un método del contrato <code>JRDataSource</code> .
Línea 21	<code>public Object getFieldValue(JRField campo) throws JRException {</code> → implementa la resolución de un campo JRXML para el registro actual.
Línea 22	<code>Libro actual = libros.get(indice);</code> → obtiene el libro correspondiente al índice actual.
Línea 23	<code>if ("titulo".equals(campo.getName())) {</code> → resuelve el campo <code>titulo</code> .
Línea 24	<code>return actual.getTitulo();</code> → devuelve el valor del campo solicitado para el libro actual.
Línea 25	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.
Línea 26	<code>if ("precio".equals(campo.getName())) {</code> → resuelve el campo <code>precio</code> .

Línea 27	<code>return actual.getPrecio();</code> → devuelve el valor del campo solicitado para el libro actual.
Línea 28	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.
Línea 29	<code>throw new JRException("Campo no soportado por CatalogoDataSource: " + campo.getName());</code> → falla explícitamente si el JRXML solicita un campo que la fuente no soporta, evitando devolver silenciosamente un valor incorrecto.
Línea 30	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.
Línea 31	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.

Clase GeneradorInformeConcepto.java

JAVA

```
import java.io.File;
import java.util.HashMap;
import java.util.Map;

import net.sf.jasperreports.engine.JasperCompileManager;
import net.sf.jasperreports.engine.JasperExportManager;
import net.sf.jasperreports.engine.JasperFillManager;
import net.sf.jasperreports.engine.JasperPrint;

public class GeneradorInformeConcepto {
    public static void main(String[] args) {
        try {
            String rutaJrxml = "reports/informe_concepto.jrxml";
            String rutaJasper = "reports/informe_concepto.jasper";
            String rutaPdf = "output/informe_concepto.pdf";

            JasperCompileManager.compileReportToFile(rutaJrxml, rutaJasper);

            Map<String, Object> parametros = new HashMap<String, Object>();

            JasperPrint documento = JasperFillManager.fillReport(
                rutaJasper,
                parametros,
                new CatalogoDataSource(Libro.listaEjemplo()));

            JasperExportManager.exportReportToPdfFile(documento, rutaPdf);

            System.out.println("Informe generado en: " + new File(rutaPdf).getAbsolutePath());
            System.out.println("Paginas del documento: " + documento.getPages().size());
            System.out.println("Registros de ejemplo: " + Libro.listaEjemplo().size());
        } catch (Exception e) {
            e.printStackTrace();
            System.exit(1);
        }
    }
}
```

Explicación línea por línea

Línea 1	<code>import java.io.File;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 2	<code>import java.util.HashMap;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 3	<code>import java.util.Map;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 5	<code>import net.sf.jasperreports.engine.JasperCompileManager;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 6	<code>import net.sf.jasperreports.engine.JasperExportManager;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 7	<code>import net.sf.jasperreports.engine.JasperFillManager;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 8	<code>import net.sf.jasperreports.engine.JasperPrint;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.

Línea 10	<code>public class GeneradorInformeConcepto {</code> → declara la clase Java y, si aparece <code>implements</code> , establece el contrato que debe implementar.
Línea 11	<code>public static void main(String[] args) {</code> → declara el punto de entrada de la aplicación.
Línea 12	<code>try {</code> → abre el bloque protegido de ejecución.
Línea 13	<code>String rutaJrxml = "reports/informe_concepto.jrxml";</code> → define la ruta relativa de la plantilla JRXML.
Línea 14	<code>String rutaJasper = "reports/informe_concepto.jasper";</code> → define la ruta del artefacto compilado <code>.jasper</code> .
Línea 15	<code>String rutaPdf = "output/informe_concepto.pdf";</code> → define la ruta del PDF de salida.
Línea 17	<code>JasperCompileManager.compileReportToFile(rutaJrxml, rutaJasper);</code> → compila el JRXML con JasperReports Library.
Línea 19	<code>Map<String, Object> parametros = new HashMap<String, Object>();</code> → crea el mapa tipado de parámetros.
Línea 21	<code>JasperPrint documento = JasperFillManager.fillReport(</code> → declara el <code>JasperPrint</code> resultante del llenado.
Línea 22	<code>rutaJasper,</code> → pasa al llenado el informe compilado.
Línea 23	<code>parametros,</code> → pasa el mapa de parámetros.
Línea 24	<code>new CatalogoDataSource(Libro.listaEjemplo());</code> → pasa la fuente de datos construida con los libros de ejemplo.
Línea 26	<code>JasperExportManager.exportReportToPdfFile(documento, rutaPdf);</code> → exporta el <code>JasperPrint</code> a un PDF real.
Línea 28	<code>System.out.println("Informe generado en: " + new File(rutaPdf).getAbsolutePath());</code> → escribe en consola la ruta absoluta del PDF generado.
Línea 29	<code>System.out.println("Paginas del documento: " + documento.getPages().size());</code> → escribe en consola el número real de páginas del <code>JasperPrint</code> .
Línea 30	<code>System.out.println("Registros de ejemplo: " + Libro.listaEjemplo().size());</code> → escribe en consola el número de registros de ejemplo; el workflow usa esta salida como evidencia de ejecución.
Línea 31	<code>} catch (Exception e) {</code> → captura cualquier fallo de compilación, llenado o exportación.
Línea 32	<code>e.printStackTrace();</code> → imprime la traza del error para diagnóstico.
Línea 33	<code>System.exit(1);</code> → termina con código distinto de cero para que GitHub Actions detecte el fallo.
Línea 34	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.
Línea 35	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.
Línea 36	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.

Traza de consola esperada tras la ejecución

TEXT

```
Informe generado en: C:\Users\<usuario>\Documents\JasperProjects\EditorialReports\output\informe_concepto.pdf
Paginas del documento: 1
Registros de ejemplo: 12
```

La ruta depende del equipo. Los valores anteriores están comprobados en el run E2E 36011315054: checkpoint 2.2 → 1 página y 12 registros.

Parte D — Validación del resultado y estructura del proyecto

D.1 — Vista de diseño en Jaspersoft Studio

TEXT

informe_concepto.jrxml	[Design]	[Source]
Ruler: 0 100 200 300 400 500 555		
Title h = 70 Catálogo Editorial - Informe Conceptual Fecha de emisión: [new java.util.Date()]		
Page Header h = 40 Catálogo Editorial (cursiva) ["Página "+\${PAGE_NUM...}] Precio en euros con IVA incluido (Styled Text, "euros" negrita)		
Column Header h = 25 Título Precio		
Detail 1 h = 20 [\${F{titulo}}] [\${F{precio}}] [#] [\${V{REPORT...}}] (se emite 12 veces con los 12 libros)		
Column Footer h = 40 --- Fin de la tabla de datos --- Registros procesados: [\${V{REPORT_COUNT}}]		
Page Footer h = 30 EditorialReports - Documento...		
Last Page Footer h = 30 Documento generado en la última página		
Summary h = 70 Total de páginas: [\${V{PAGE_NUMBER}} [evaluationTime="Report"]] Fin del informe. EditorialReports. Total de libros: [\${V{REPORT_COUNT}}]		
Background h = 0		
Palette Elements [T] Static [F] TextF [=] Image [=] Table	Properties Element: textField Pattern: #,##0.00 € Blank When Null: [X] Stretch With Overflow: [] Expression: \${F{precio}}	

Qué representa: la disposición de las bandas en el editor central tras completar los doce pasos de la Parte A. La banda Page Header ha crecido a 40 unidades de informe y la banda Column Footer también. La banda Detail contiene cuatro elementos distribuidos horizontalmente.

Cómo verificarlo: comparar la presencia y la geometría de las secciones con este esquema. Para el orden XML válido, usar la Parte B: background se declara antes de las secciones de contenido aunque visualmente el diseñador pueda presentarlo en otra posición.

D.2 — Jerarquía del Outline

TEXT

```
informe_concepto
├── Properties
│   └── com.jaspersoft.studio.data.defaultdataadapter = EmptyDataSource
├── Styles
│   └── Sans_Normal [isDefault=true]
├── Fields
│   └── titulo [java.lang.String]
```

```

└─ precio [java.lang.Double]

Variables
└─ TotalPrecios [java.lang.Double, calculation=Sum]

Title [band, height=70]
└─ staticText "Catálogo Editorial - Informe Conceptual"
└─ staticText "Fecha de emisión:"
└─ textField [pattern=dd/MM/yyyy] new java.util.Date()

Page Header [band, height=40, splitType=Prevent]
└─ staticText "Catálogo Editorial - Informe Conceptual" (italic)
└─ textField [right] "Página " + ${PAGE_NUMBER} + " de"
└─ staticText [markup=styled] "Precio en <b>euros</b> con IVA incluido"

Column Header [band, height=25]
└─ staticText "Título" (bold)
└─ staticText "Precio" (bold)

Detail 1 [band, height=20, splitType=Stretch]
└─ textField [textAdjust=StretchHeight] ${titulo}
└─ textField [pattern=#,##0.00 €, isBlankWhenNull=true] ${precio}
└─ staticText [right] "# "
└─ textField [right] ${REPORT_COUNT}

Column Footer [band, height=40]
└─ staticText "--- Fin de la tabla de datos ---"
└─ staticText "Registros procesados: "
└─ textField [bold] ${REPORT_COUNT}

Page Footer [band, height=30]
└─ staticText "EditorialReports - Documento..."

Last Page Footer [band, height=30]
└─ staticText "Documento generado en la última página"

Summary [band, height=70]
└─ staticText "Total de páginas:"
└─ textField ${PAGE_NUMBER} [evaluationTime="Report"]
└─ staticText "Fin del informe. EditorialReports."
└─ staticText "Total de libros:" (bold)
└─ textField ${REPORT_COUNT} (bold)

Background [band, height=0]

```

Qué representa: el árbol de nodos del informe tal como aparece en el panel Outline tras completar los doce pasos. La novedad respecto al punto 2.1 es la ampliación de las bandas Page Header y Column Footer, y los nuevos elementos en la banda Detail.

Cómo verificarlo: expandir el nodo informe_concepto en el panel Outline y comparar la estructura. Cada textField debe mostrar sus propiedades específicas entre corchetes.

D.3 — Documento PDF resultante, página por página

TEXT

INFORME: informe_concepto.pdf
 PÁGINAS TOTALES: 1
 TAMAÑO DE PÁGINA: 595 × 842 unidades de informe (A4) (A4 vertical)
 MÁRGENES: izquierdo 20, derecho 20, superior 20, inferior 20
 REGISTROS PROCESADOS: 12
 BANDAS EMITIDAS:
 - Title, Page Header, Column Header, Detail (12 veces),
 Column Footer, Last Page Footer, Summary, Background

— Página 1 de 1 —

Catálogo Editorial - Informe Conceptual		
Fecha de emisión: 22/09/2026		
Catálogo Editorial (cursiva)	Página 1 de 1	
Precio en euros con IVA incluido	(euros en negrita)	
Título	Precio	
Cien años de soledad	19,95 €	# 1
Rayuela	22,50 €	# 2
La ciudad y los perros	18,75 €	# 3
Pedro Páramo	15,90 €	# 4
Ficciones	21,00 €	# 5
La casa de los espíritus	23,40 €	# 6
El amor en los tiempos...	20,80 €	# 7
La muerte de Artemio Cruz	17,60 €	# 8
Doña Bárbara	16,95 €	# 9
Martín Fierro	14,50 €	# 10
Comala	19,20 €	# 11
Paradiso	25,00 €	# 12

```

--- Fin de la tabla de datos ---
Registros procesados: 12

Documento generado en la última página

Total de páginas: 1
Fin del informe. EditorialReports.
Total de libros: 12

```

Qué representa: la página única del PDF resultante. Los precios aparecen con el símbolo del euro, el título largo El amor en los tiempos del cólera se ajusta en varias líneas y la columna de la derecha muestra el número correlativo del registro.

Cómo verificarlo: abrir el archivo output/informe_concepto.pdf con un lector de PDF y comprobar que los precios muestran el símbolo del euro, que el rótulo Precio en euros con IVA incluido aparece en el Page Header con la palabra euros en negrita y que la columna del número de registro muestra valores del 1 al 12.

D.4 — Árbol de carpetas del proyecto tras completar el punto

TEXT

```

EditorialReports/
├── ECOSISTEMA.md                (documentación del ecosistema)
├── ENTORNO.md                  (documentación del entorno)
├── BANDAS.md                   (documentación de las bandas)
├── JRXML.md                    (documentación del formato JRXML)
├── TEXTO.md                    (documentación de elementos textuales)
├── reports/
│   ├── informe_concepto.jrxml  (plantilla ampliada)
│   └── informe_concepto.jasper (artefacto compilado)
├── resources/
│   └── (vacía en este punto)
├── output/
│   └── informe_concepto.pdf    (documento con formato aplicado)
└── EditorialReportsJava/
    ├── lib/
    │   └── README.md          (el runtime real se resuelve con Maven)
    └── src/
        ├── GeneradorInformeConcepto.java
        ├── Libro.java
        └── CatalogoDataSource.java

```

Qué representa: el estado de los dos proyectos tras completar los doce pasos. La novedad respecto al punto anterior es el archivo TEXTO.md en la raíz del proyecto EditorialReports.

Cómo verificarlo: expandir los nodos del panel Project Explorer y comparar con este esquema. Si el archivo TEXTO.md no aparece, repetir el paso 12.

Errores comunes del ejercicio completo

Error	Causa	Solución
El patrón <code>###0.00 €</code> muestra el símbolo del euro como ?	La fuente del campo no soporta el carácter €	Utilizar DejaVu Sans mediante la extensión de fuentes validada del curso
El campo con <code>isBlankWhenNull</code> no se muestra vacío	La expresión devuelve una cadena vacía en lugar de null	Verificar que la fuente de datos devuelve null para los registros sin precio
El texto con <code>markup="styled"</code> imprime las etiquetas <code></code> como texto literal	La propiedad <code>markup="styled"</code> no está activada en <code>textElement</code>	Seleccionar <code>Styled Text/Styled</code> en las propiedades de marcado del elemento de texto
El texto con etiquetas HTML produce un error de análisis XML	El contenido no está encerrado en un bloque CDATA	Encerrar el contenido en <code><![CDATA[...]]></code>
El campo con <code>textAdjust="StretchHeight"</code> no muestra todo el texto	El elemento o la banda no dispone de espacio suficiente para crecer	Revisar la altura inicial, la posición de los elementos vecinos y permitir espacio vertical suficiente
La banda Page Header se divide entre páginas	La propiedad <code>splitType</code> está en <code>Stretch</code>	Cambiar a <code>splitType="Prevent"</code> en el panel <code>Properties</code> de la banda

Error	Causa	Solución
El número de registro aparece siempre como 12	Se usó <code>{REPORT_COUNT}</code> en la banda Summary en lugar de en la banda Detail	Verificar que el campo está en la banda Detail 1 y no en la Summary
El rótulo # aparece sin espacio antes del número	El texto se escribió sin el espacio al final	Editar el texto del <code>staticText</code> y añadir el espacio: #
El campo del número de registro se solapa con el campo del precio	El ancho de los elementos supera el ancho de la columna	Reducir el ancho del campo del título a 300 y ajustar las posiciones
El texto sigue recortado aunque se configuró <code>StretchHeight</code>	El campo se evalúa tarde o el diseño no permite crecer sin colisionar	Verificar <code>evaluationTime</code> y la distribución vertical de la banda; probar de nuevo en Preview

Reto resuelto paso a paso

Enunciado: añadir un rótulo estático en la banda Detail que muestre la palabra Disponible cuando el precio sea superior a 20 euros. Utilizar la configuración `markup="styled"` para aplicar negrita al texto.

- Paso 1. Hacer doble clic sobre el archivo `informe_concepto.jrxml` en el panel Project Explorer.
- Paso 2. Hacer clic sobre la pestaña Design en la parte inferior del editor central.
- Paso 3. Hacer clic sobre la pestaña Elements en el panel Palette (derecha del editor central).
- Paso 4. Hacer clic sobre el icono Text Field (una letra F dentro de un cuadrado).
- Paso 5. Arrastrar el icono Text Field y soltarlo dentro de la banda Detail 1, en la coordenada aproximada `x=460, y=0`.
- Paso 6. Hacer clic sobre el campo Text Field Expression en el panel Properties, pestaña Properties.
- Paso 7. Escribir exactamente `$F{precio}.doubleValue() > 20.0 ? "<b>Disponible</b>" : ""` y pulsar Enter.
- Paso 8. Hacer clic sobre el campo Width, escribir 80 y pulsar Enter.
- Paso 9. Hacer clic sobre el campo Height, escribir 20 y pulsar Enter.
- Paso 10. Hacer clic sobre el campo X, escribir 460 y pulsar Enter.
- Paso 11. Hacer clic sobre el campo Y, escribir 0 y pulsar Enter.
- Paso 12. Hacer clic sobre la pestaña Properties y marcar la casilla Styled Text.
- Paso 13. Hacer clic sobre el desplegable Horizontal Text Alignment y seleccionar Center.
- Paso 14. Pulsar `Ctrl+S` para guardar el archivo.
- Paso 15. Pulsar `Ctrl+Mayús+B` para compilar el informe.
- Paso 16. Hacer clic con el botón derecho sobre `GeneradorInformeConcepto.java` y seleccionar Run As > Java Application.
- Paso 17. Abrir el archivo `output/informe_concepto.pdf` y verificar que los libros con precio superior a 20 euros muestran la palabra Disponible en negrita.

Simulación ASCII del PDF tras el reto

TEXT

Catálogo Editorial - Informe Conceptual			
Título	Precio		Av.
Cien años de soledad	19,95 €	# 1	
Rayuela	22,50 €	# 2	Dispon
La ciudad y los perros	18,75 €	# 3	
Pedro Páramo	15,90 €	# 4	
Ficciones	21,00 €	# 5	Dispon
La casa de los espíritus	23,40 €	# 6	Dispon
El amor en los tiempos...	20,80 €	# 7	Dispon
La muerte de Artemio Cruz	17,60 €	# 8	
Doña Bárbara	16,95 €	# 9	
Martín Fierro	14,50 €	# 10	
Comala	19,20 €	# 11	
Paradiso	25,00 €	# 12	Dispon

Resultado del reto: el campo de la derecha muestra la palabra Disponible en negrita cuando el precio supera 20 euros y queda vacío en caso contrario. La expresión utiliza el operador ternario de Java y la configuración `markup="styled"` interpreta las etiquetas `<b>`. Los libros con precio superior a 20 euros son Rayuela, Ficciones, La casa de los espíritus, El amor en los tiempos del cólera y Paradiso.

Analogía final con el contexto de la editorial

Los elementos textuales son los bloques con los que se compone el catálogo. El texto estático es el rótulo impreso que el diseñador coloca a mano: Precio:, Título, Precio en euros con IVA incluido. El campo de texto es el espacio reservado para el dato que se estampa en el momento de la tirada: 19,95 €, Cien años de soledad, el número de registro. Las propiedades de formato son las decisiones tipográficas: negrita, cursiva, alineación, color. La combinación de rótulos y campos construye la tabla del catálogo fila a fila. Cada decisión de formato afecta a la legibilidad del documento y a la impresión que el lector se lleva de la editorial.

Resultado esperado

- Al finalizar este punto, el alumno dispone de:
- El archivo reports/informe_concepto.jrxml con las bandas Page Header y Column Footer ampliadas y con los nuevos elementos de la banda Detail.
- Los campos con las propiedades `textAdjust="StretchHeight"`, `isBlankWhenNull` y `pattern` aplicadas.
- El rótulo del Page Header con la configuración `markup="styled"` activada.
- El archivo output/informe_concepto.pdf con los precios formateados con el símbolo del euro, el número de registro por fila y el aviso sobre el IVA en el encabezado.
- El archivo TEXTO.md en la raíz del proyecto con la documentación de las propiedades.
- Comprensión operativa de las propiedades comunes y específicas de los elementos textuales.

Conclusión y enlace al siguiente punto

El punto 2.2 ha diferenciado el texto estático de los campos de texto y ha incorporado formato dinámico, `textAdjust="StretchHeight"`, `isBlankWhenNull`, patrones numéricos y marcado `styled`. El informe ya no se limita a colocar datos: controla cómo se presentan y cómo se comportan cuando el contenido cambia.

El punto 2.3, «Campos», amplía el modelo del libro y conecta nuevos tipos Java con campos JRXML. A partir de la base textual de este punto se incorporarán páginas, fecha de publicación y disponibilidad.

Punto 2.3 — Campos

PUNTO DE PARTIDA. Si vienes haciendo el curso, continúa con tu propio proyecto del punto anterior. Si te incorporas directamente aquí, usa M2/2.2 como estado inicial. El checkpoint M2/2.3 contiene la solución completa de este punto y no debe consultarse antes del ejercicio si quieres evitar spoilers.

Parte A — Práctica visual

Paso 1: Ampliar la clase Libro con los nuevos campos y el constructor completo VALIDADO

Acciones:

1. Hacer doble clic sobre EditorialReportsJava/src/Libro.java en Project Explorer.
2. Añadir, junto a los imports existentes, `import java.util.Calendar;` e `import java.util.GregorianCalendar;`
3. Declarar, además de `titulo` y `precio`, los campos `private final Integer paginas;`, `private final java.util.Date fechaPublicacion;` y `private final Boolean disponible;`
4. Sustituir el constructor anterior por `public Libro(String titulo, Double precio, Integer paginas, int anioPublicacion, Boolean disponible).`
5. Dentro del constructor asignar `this.titulo = titulo;`, `this.precio = precio;`, `this.paginas = paginas;`, `this.fechaPublicacion = fecha(anioPublicacion);` y `this.disponible = disponible;`
6. Añadir el método auxiliar `private static java.util.Date fecha(int anio)` exactamente como aparece en la Parte C: crea un `GregorianCalendar`, fija el 1 de enero, normaliza hora/minuto/segundo/milisegundo y devuelve `c.getTime()`.
7. Añadir o conservar los getters `getTitulo()`, `getPrecio()`, `getPaginas()`, `getFechaPublicacion()` y `getDisponible()`.
8. Pulsar Ctrl+S y verificar que Problems no muestra errores.

Verificación visual: Libro.java contiene cinco campos `final`, un constructor de cinco argumentos, el método auxiliar `fecha(int)` y cinco getters.

Qué hace: convierte Libro en el modelo completo que realmente usa el checkpoint 2.3.

Por qué: páginas, año y disponibilidad deben contener valores reales por libro; no sirven valores por defecto idénticos para todos los registros.

Error común: mantener el constructor de dos argumentos y asignar 0, la fecha actual y `true` a todos los libros. **Solución:** usar el constructor de cinco argumentos y los datos concretos reproducidos en la Parte C.

Analogía: es como completar la ficha bibliográfica real de cada título en lugar de rellenar todas las fichas con valores provisionales.

Paso 2: Sustituir listaEjemplo por los catorce registros completos VALIDADO

Acciones:

1. En Libro.java, localizar el método `listaEjemplo()`.
2. Sustituir su contenido por el método completo reproducido en la Parte C de este punto.
3. Comprobar que cada llamada usa cinco valores: título, precio, páginas, año y `Boolean.TRUE/Boolean.FALSE`.
4. Verificar que la lista contiene exactamente catorce libros y que termina con `La invención de Morel` y `El túnel`.
5. Pulsar Ctrl+S y comprobar que Problems queda vacío.

Verificación visual: las catorce llamadas `new Libro(...)` contienen páginas, año y disponibilidad reales de ejemplo; no quedan llamadas al antiguo constructor de dos argumentos.

Qué hace: alimenta el nuevo modelo con datos heterogéneos que permiten comprobar números, fechas y booleanos.

Por qué: 2.3 pretende validar tipos de campos reales; una lista con valores por defecto no demostraría que el mapeo funciona.

Error común: añadir los dos libros nuevos pero dejar los doce anteriores con el constructor antiguo. **Solución:** reemplazar las catorce llamadas por las de la Parte C.

Analogía: es como rehacer el fichero maestro del catálogo con la ficha completa de cada libro.

Paso 3: Declarar los nuevos campos en el JRXML VALIDADO

Acciones:

1. Hacer doble clic sobre el archivo informe_concepto.jrxml en el panel Project Explorer (superior izquierdo).
2. Hacer clic sobre la pestaña Source en la parte inferior del editor central.
3. Hacer clic al final de la línea que contiene `<field name="precio" class="java.lang.Double"/>` y pulsar Enter.
4. Escribir exactamente `<field name="paginas" class="java.lang.Integer"/>` y pulsar Enter.
5. Escribir exactamente `<field name="fechaPublicacion" class="java.util.Date"/>` y pulsar Enter.
6. Escribir exactamente `<field name="disponible" class="java.lang.Boolean"/>` y pulsar Enter.
7. Pulsar Ctrl+S para guardar el archivo.
8. Hacer clic sobre la pestaña Design en la parte inferior del editor central.
9. Expandir el nodo Fields en el panel Outline (inferior izquierdo) y verificar que aparecen los cinco campos.

Verificación visual: el panel Outline muestra el nodo Fields con cinco entradas: titulo, precio, paginas, fechaPublicacion y disponible.

Qué hace: declara los tres nuevos campos en el JRXML con sus tipos correspondientes.

Por qué: las nuevas expresiones `$F{paginas}`, `$F{fechaPublicacion}` y `$F{disponible}` necesitan que los campos estén declarados.

Error común: escribir el nombre del campo con mayúscula inicial (Paginas). El motor busca el campo por el nombre exacto y lanza Field not found: Paginas.

Solución: usar el nombre exacto en minúsculas que coincide con el método getter de la clase Libro.

Analogía: es como declarar en el pliego los nuevos datos que se van a extraer del manuscrito.

Paso 4: Compactar Título/Precio y añadir la columna de páginas VALIDADO

Acciones:

1. Seleccionar `$F{titulo}` en Detail y escribir X=0, Y=0, Width=245, Height=20.
2. Seleccionar `$F{precio}` y escribir X=245, Y=0, Width=80, Height=20; mantener alineación Right.
3. Arrastrar un Text Field a Detail y escribir `$F{paginas}` como expresión.
4. En Properties escribir X=325, Y=0, Width=55, Height=20.
5. Activar Blank When Null y seleccionar alineación Right.
6. Pulsar Ctrl+S.

Verificación visual: título ocupa 0-245, precio 245-325 y páginas 325-380; las tres zonas son contiguas y permanecen dentro de 555 unidades.

Qué hace: compacta las columnas existentes y añade el nuevo dato de páginas.

Por qué: la geometría debe reservar espacio para año, disponibilidad y contador sin superar `columnWidth="555"`.

Error común: añadir páginas sin reducir antes título y precio, lo que produce solapamientos. **Solución:** aplicar primero 245/80 y después colocar páginas en 325/55.

Analogía: es como estrechar dos columnas de una tabla para abrir una tercera sin aumentar el ancho del papel.

Paso 5: Reubicar el contador de registro VALIDADO

Acciones:

1. Seleccionar el Text Field cuya expresión es `$V{REPORT_COUNT}`.
2. Fijar X=495, Y=0, Width=60, Height=20.

3. Seleccionar alineación horizontal Right.
4. Si todavía existe un Static Text independiente con # procedente del punto 2.2, eliminarlo: a partir de este punto el rótulo # se mantiene en Column Header y Detail contiene solo el valor del contador.

Verificación visual: el contador queda al extremo derecho y todo el Detail sigue dentro de 555 unidades.

Qué hace: reserva el extremo derecho del Detail para el contador de registros.

Por qué: el contador debe convivir con las nuevas columnas sin sobrepasar el ancho útil del informe.

Error común: situar el contador en X=500 con Width=60 y superar 555. **Solución:** usar X=495 y Width=60.

Analogía: es como desplazar el número de línea al margen derecho para hacer sitio a nuevas columnas.

Paso 6: Compactar Column Header y añadir Páginas VALIDADO

Acciones:

1. Seleccionar Column Header en Outline.
2. Seleccionar el encabezado Título y escribir X=0, Y=5, Width=245, Height=15.
3. Seleccionar Precio y escribir X=245, Y=5, Width=80, Height=15; alinearlos a la derecha.
4. Arrastrar un Static Text, escribir Páginas y colocarlo en X=325, Y=5, Width=55, Height=15.
5. Seleccionar alineación Right y negrita para Páginas.
6. Pulsar Ctrl+S.

Verificación visual: las cabeceras Título, Precio y Páginas coinciden horizontalmente con sus campos de Detail.

Qué hace: reproduce en Column Header la geometría compactada de Detail.

Por qué: una cabecera desalineada hace que el lector atribuya un dato a la columna incorrecta.

Error común: conservar Título/Precio en sus anchos de 2.2. **Solución:** usar 245 y 80 antes de añadir Páginas.

Analogía: es como alinear con regla los rótulos superiores y las celdas de una tabla editorial.

Paso 7: Añadir la columna de fecha de publicación VALIDADO

Acciones:

1. Arrastrar un Text Field a Detail 1.
2. Escribir `$F{fechaPublicacion}`.
3. Fijar X=380, Y=0, Width=50, Height=20.
4. Configurar Pattern=yyyy.
5. Seleccionar alineación horizontal Center.

Verificación visual: el informe muestra el año de publicación en una columna compacta.

Qué hace: muestra el año de publicación a partir del campo fechaPublicacion.

Por qué: el patrón yyyy transforma la fecha completa en un año legible sin cambiar el dato original.

Error común: declarar el campo como String y aplicar un patrón de fecha. **Solución:** mantener `java.util.Date` en JRXML y en la fuente de datos.

Analogía: es como tomar de la ficha bibliográfica la fecha completa y mostrar solo el año en el catálogo.

Paso 8: Añadir el encabezado de año y mantener el contador al final VALIDADO

Acciones:

1. En Column Header, añadir un Static Text con el texto Año.
2. Fijar X=380, Y=5, Width=50, Height=15.

3. Seleccionar alineación Center y negrita.
4. Verificar que el contador de Detail permanece en X=495, Width=60.

Verificación visual: la columna de año ocupa 380-430 y el contador sigue reservado al extremo derecho.

Qué hace: añade la cabecera del año y verifica el espacio reservado para el contador.

Por qué: la cabecera debe reproducir exactamente la geometría de las columnas de Detail.

Error común: mover el encabezado sin mover su campo correspondiente. **Solución:** comprobar X y Width en Column Header y Detail.

Analogía: es como alinear con regla el título de una columna con todos los valores impresos debajo.

Paso 9: Añadir el encabezado de disponibilidad VALIDADO

Acciones:

1. En Column Header, añadir un Static Text con el texto Disp..
2. Fijar X=430, Y=5, Width=65, Height=15.
3. Seleccionar alineación Center y negrita.
4. Añadir otro Static Text con el texto # en X=495, Y=5, Width=60, Height=15, alineado a la derecha.

Verificación visual: la cabecera completa ocupa exactamente el ancho útil del informe.

Qué hace: crea los rótulos de disponibilidad y número de registro.

Por qué: completa la cabecera antes de insertar el último campo del modelo.

Error común: hacer que Disp. invada el rótulo #. **Solución:** respetar X=430/Width=65 y X=495/Width=60.

Analogía: es como repartir los últimos encabezados de una tabla hasta el borde derecho del pliego.

Paso 10: Añadir el campo disponible VALIDADO

Acciones:

1. Arrastrar un Text Field a Detail 1.
2. Fijar X=430, Y=0, Width=65, Height=20.
3. Escribir la expresión `$F{disponible}.booleanValue() ? "Sí" : "No"`.
4. Seleccionar alineación Center.
5. Guardar el JRXML y comprobar que no hay errores en Problems.

Verificación visual: aparecen seis columnas dentro de 555 unidades: título, precio, páginas, año, disponibilidad y número de registro.

Qué hace: completa el modelo visual de los cinco campos de negocio más el contador de registro.

Por qué: prepara el layout que en 2.4 reservará una franja izquierda para las imágenes sin salir del ancho A4.

Error común: escribir directamente `$F{disponible}` y obtener true/false cuando se desea Sí/No. **Solución:** utilizar la expresión ternaria indicada.

Analogía: es como traducir un indicador interno de la base de datos a una etiqueta comprensible para el lector.

Paso 11: Compilar y ejecutar el programa Java VALIDADO

Acciones:

1. Hacer clic sobre la pestaña Design del editor central.
2. Pulsar Ctrl+S para guardar el archivo JRXML.
3. Pulsar Ctrl+Mayús+B para compilar el informe.

4. Hacer clic sobre el panel Problems y verificar que no hay errores.
5. Hacer clic con el botón derecho sobre el archivo GeneradorInformeConcepto.java en el panel Project Explorer.
6. Hacer clic sobre la opción Run As en el menú contextual.
7. Hacer clic sobre la opción Java Application en el submenú.
8. Hacer clic sobre la vista Console en el panel inferior y observar el resultado.

Verificación visual: la vista Console muestra la línea Informe generado en: ... con la ruta absoluta del archivo PDF. El panel Problems permanece vacío.

Qué hace: compila el informe y ejecuta el programa Java con los nuevos campos declarados.

Por qué: la ejecución confirma que los tres campos nuevos se resuelven correctamente desde la fuente de datos.

Error común: olvidar compilar el informe después de modificar el JRXML y obtener un PDF con la versión anterior.

Solución: pulsar Ctrl+Mayús+B antes de ejecutar el programa.

Analogía: es como imprimir la tirada del catálogo con los nuevos datos incluidos en la tabla.

Paso 12: Documentar los campos del proyecto VALIDADO

Acciones:

1. Hacer clic con el botón derecho sobre el nodo EditorialReports en el panel Project Explorer (superior izquierdo).
2. Hacer clic sobre la opción New en el menú contextual.
3. Hacer clic sobre la opción File en el submenú.
4. Escribir exactamente CAMPOS.md en el campo File name del diálogo.
5. Hacer clic sobre el botón Finish.
6. En el editor central, escribir exactamente # Campos del informe y pulsar Enter dos veces.
7. Escribir exactamente | Nombre | Tipo Java | Origen | y pulsar Enter.
8. Escribir exactamente |---|---|---| y pulsar Enter.
9. Escribir exactamente | titulo | java.lang.String | Libro.getTitulo() | y pulsar Enter.
10. Escribir exactamente | precio | java.lang.Double | Libro.getPrecio() | y pulsar Enter.
11. Escribir exactamente | paginas | java.lang.Integer | Libro.getPaginas() | y pulsar Enter.
12. Escribir exactamente | fechaPublicacion | java.util.Date | Libro.getFechaPublicacion() | y pulsar Enter.
13. Escribir exactamente | disponible | java.lang.Boolean | Libro.getDisponible() | y pulsar Enter.
14. Pulsar Ctrl+S para guardar el archivo.

Verificación visual: el panel Project Explorer muestra el archivo CAMPOS.md en la raíz del proyecto EditorialReports con la tabla de campos documentada.

Qué hace: incorpora al proyecto un documento que registra los campos del informe y su origen.

Por qué: la documentación de los campos facilita el mantenimiento y la incorporación de nuevos desarrolladores.

Error común: olvidar la barra vertical al final de cada línea de la tabla Markdown.

Solución: revisar cada línea y asegurarse de que comienza y termina con |.

Analogía: es como dejar en la editorial una tabla con los datos que se extraen de cada manuscrito para el catálogo.

Parte B — JRXML explicado y contrastado COMPLETADO

Se reproducen las declaraciones de campos y las bandas Column Header y Detail del checkpoint 2.3. Las coordenadas suman como máximo 555 unidades y el contador ocupa X=495, Width=60; no queda ningún elemento de Detail fuera del ancho útil.

XML

```

<field name="titulo" class="java.lang.String"/>
<field name="precio" class="java.lang.Double"/>
<field name="paginas" class="java.lang.Integer"/>
<field name="fechaPublicacion" class="java.util.Date"/>
<field name="disponible" class="java.lang.Boolean"/>

<columnHeader>
  <band height="25">
    <staticText><reportElement x="0" y="5" width="245" height="15" uuid="11111111-1111-1111-1111-111111111111"/><textElement>
<font size="10" isBold="true"/></textElement><text><![CDATA[Título]]></text></staticText>
    <staticText><reportElement x="245" y="5" width="80" height="15" uuid="22222222-2222-2222-2222-222222222222"/>
<textElement textAlignment="Right"><font size="10" isBold="true"/></textElement><text><![CDATA[Precio]]></text></staticText>
    <staticText><reportElement x="325" y="5" width="55" height="15" uuid="33333333-3333-3333-3333-333333333333"/>
<textElement textAlignment="Right"><font size="10" isBold="true"/></textElement><text><![CDATA[Páginas]]></text></staticText>
    <staticText><reportElement x="380" y="5" width="50" height="15" uuid="44444444-4444-4444-4444-444444444444"/>
<textElement textAlignment="Center"><font size="10" isBold="true"/></textElement><text><![CDATA[Año]]></text></staticText>
    <staticText><reportElement x="430" y="5" width="65" height="15" uuid="55555555-5555-5555-5555-555555555555"/>
<textElement textAlignment="Center"><font size="10" isBold="true"/></textElement><text><![CDATA[Disp.]]></text></staticText>
    <staticText><reportElement x="495" y="5" width="60" height="15" uuid="66666666-6666-6666-6666-666666666666"/>
<textElement textAlignment="Right"><font size="10" isBold="true"/></textElement><text><![CDATA[#]]></text></staticText>
  </band>
</columnHeader>

<detail>
  <band height="20" splitType="Stretch">
    <textField textAdjust="StretchHeight"><reportElement x="0" y="0" width="245" height="20" uuid="77777777-7777-7777-7777-777777777777"/><textFieldExpression><![CDATA[{F{titulo}}]></textFieldExpression></textField>
    <textField pattern="#,##0.00 €" isBlankWhenNull="true"><reportElement x="245" y="0" width="80" height="20" uuid="88888888-8888-8888-8888-888888888888"/><textElement textAlignment="Right"><textFieldExpression><![CDATA[{F{precio}}]></textFieldExpression></textField>
    <textField isBlankWhenNull="true"><reportElement x="325" y="0" width="55" height="20" uuid="99999999-9999-9999-9999-999999999999"/><textElement textAlignment="Right"><textFieldExpression><![CDATA[{F{paginas}}]></textFieldExpression></textField>
    <textField pattern="yyyy" isBlankWhenNull="true"><reportElement x="380" y="0" width="50" height="20" uuid="aaaaaaa-aaaa-aaaa-aaaa-aaaaaa"/><textElement textAlignment="Center"><textFieldExpression><![CDATA[{F{fechaPublicacion}}]></textFieldExpression></textField>
    <textField isBlankWhenNull="true"><reportElement x="430" y="0" width="65" height="20" uuid="bbbbbbb-bbbb-bbbb-bbbb-bbbbbbbbbbb"/><textElement textAlignment="Center"><textFieldExpression><![CDATA[{F{disponible}.booleanValue() ? "Sí" : "No"}]></textFieldExpression></textField>
    <textField><reportElement x="495" y="0" width="60" height="20" uuid="ccccccc-cccc-cccc-cccc-ccccccccccc"/><textElement textAlignment="Right"><textFieldExpression><![CDATA[{V{REPORT_COUNT}}]></textFieldExpression></textField>
  </band>
</detail>

```

Explicación línea por línea

Línea 1	<code><field name="titulo" class="java.lang.String"/></code> → declara un campo JRXML y su tipo Java para que pueda resolverse desde la fuente de datos.
Línea 2	<code><field name="precio" class="java.lang.Double"/></code> → declara un campo JRXML y su tipo Java para que pueda resolverse desde la fuente de datos.
Línea 3	<code><field name="paginas" class="java.lang.Integer"/></code> → declara un campo JRXML y su tipo Java para que pueda resolverse desde la fuente de datos.
Línea 4	<code><field name="fechaPublicacion" class="java.util.Date"/></code> → declara un campo JRXML y su tipo Java para que pueda resolverse desde la fuente de datos.
Línea 5	<code><field name="disponible" class="java.lang.Boolean"/></code> → declara un campo JRXML y su tipo Java para que pueda resolverse desde la fuente de datos.
Línea 7	<code><columnHeader></code> → abre Column Header, generado al comienzo de cada columna.
Línea 8	<code><band height="25"></code> → define la banda y su altura; si aparece <code>splitType</code> , establece la política de división.
Línea 9	<code><staticText><reportElement x="0" y="5" width="245" height="15" uuid="11111111-1111-1111-1111-111111111111"/><textElement></textElement><text><![CDATA[Título]]></text></staticText></code> → abre un texto estático.
Línea 10	<code><staticText><reportElement x="245" y="5" width="80" height="15" uuid="22222222-2222-2222-2222-222222222222"/><textElement textAlignment="Right"></textElement><text><![CDATA[Precio]]></text></staticText></code> → abre un texto estático.
Línea 11	<code><staticText><reportElement x="325" y="5" width="55" height="15" uuid="33333333-3333-3333-3333-333333333333"/><textElement textAlignment="Right"></textElement><text><![CDATA[Páginas]]></text></staticText></code> → abre un texto estático.

Línea 12	<code><staticText><reportElement x="380" y="5" width="50" height="15" uuid="44444444-4444-4444-4444-444444444444"/><textElement textAlignment="Center"></textElement><text><![CDATA[Año]]></text></staticText></code> → abre un texto estático.
Línea 13	<code><staticText><reportElement x="430" y="5" width="65" height="15" uuid="55555555-5555-5555-5555-555555555555"/><textElement textAlignment="Center"></textElement><text><![CDATA[Disp.]]></text></staticText></code> → abre un texto estático.
Línea 14	<code><staticText><reportElement x="495" y="5" width="60" height="15" uuid="66666666-6666-6666-6666-666666666666"/><textElement textAlignment="Right"></textElement><text><![CDATA[#]]></text></staticText></code> → abre un texto estático.
Línea 15	<code></band></code> → cierra el elemento o sección abierto correspondiente.
Línea 16	<code></columnHeader></code> → cierra el elemento o sección abierto correspondiente.
Línea 18	<code><detail></code> → abre Detail, que el motor intenta generar por cada registro.
Línea 19	<code><band height="20" splitType="Stretch"></code> → define la banda y su altura; si aparece splitType, establece la política de división.
Línea 20	<code><textField textAdjust="StretchHeight"><reportElement x="0" y="0" width="245" height="20" uuid="77777777-7777-7777-7777-777777777777"/><textFieldExpression><![CDATA[{ \$ { titulo } }]]></textFieldExpression></textField></code> → abre un campo de texto dinámico; sus atributos controlan evaluación, formato o nulos.
Línea 21	<code><textField pattern="# , ##0.00 €" isBlankWhenNull="true"><reportElement x="245" y="0" width="80" height="20" uuid="88888888-8888-8888-8888-888888888888"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA[{ \$ { precio } }]]></textFieldExpression></textField></code> → abre un campo de texto dinámico; sus atributos controlan evaluación, formato o nulos.
Línea 22	<code><textField isBlankWhenNull="true"><reportElement x="325" y="0" width="55" height="20" uuid="99999999-9999-9999-9999-999999999999"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA[{ \$ { paginas } }]]></textFieldExpression></textField></code> → abre un campo de texto dinámico; sus atributos controlan evaluación, formato o nulos.
Línea 23	<code><textField pattern="yyyy" isBlankWhenNull="true"><reportElement x="380" y="0" width="50" height="20" uuid="aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaaaaaaa"/><textElement textAlignment="Center"/><textFieldExpression><![CDATA[{ \$ { fechaPublicacion } }]]></textFieldExpression></textField></code> → abre un campo de texto dinámico; sus atributos controlan evaluación, formato o nulos.
Línea 24	<code><textField isBlankWhenNull="true"><reportElement x="430" y="0" width="65" height="20" uuid="bbbbbbbbb-bbbb-bbbb-bbbb-bbbbbbbbbbbbbb"/><textElement textAlignment="Center"/><textFieldExpression><![CDATA[{ \$ { disponible } .booleanValue() ? "Sí" : "No"}]]></textFieldExpression></textField></code> → abre un campo de texto dinámico; sus atributos controlan evaluación, formato o nulos.
Línea 25	<code><textField><reportElement x="495" y="0" width="60" height="20" uuid="cccccccc-cccc-cccc-cccc-cccccccccccc"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA[{ \$ { REPORT_COUNT } }]]></textFieldExpression></textField></code> → abre un campo de texto dinámico; sus atributos controlan evaluación, formato o nulos.
Línea 26	<code></band></code> → cierra el elemento o sección abierto correspondiente.
Línea 27	<code></detail></code> → cierra el elemento o sección abierto correspondiente.

Parte C — Código Java explicado línea por línea COMPLETADO

2.3 amplía realmente el modelo Java con páginas, fecha de publicación y disponibilidad, y hace que la fuente resuelva esos cinco campos.

Los tres archivos siguientes se reproducen **literalmente desde el checkpoint ejecutable M2/2.3**. De este modo, la Parte C coincide con el código que compila y se ejecuta en la validación end-to-end.

Clase Libro.java

JAVA

```
import java.util.ArrayList;
import java.util.Calendar;
import java.util.GregorianCalendar;
import java.util.List;

public class Libro {
    private final String titulo;
    private final Double precio;
    private final Integer paginas;
    private final java.util.Date fechaPublicacion;
    private final Boolean disponible;

    public Libro(String titulo, Double precio, Integer paginas, int anioPublicacion, Boolean disponible) {
        this.titulo = titulo;
        this.precio = precio;
        this.paginas = paginas;
        this.fechaPublicacion = fecha(anioPublicacion);
        this.disponible = disponible;
    }

    private static java.util.Date fecha(int anio) {
        Calendar c = new GregorianCalendar(anio, Calendar.JANUARY, 1);
        c.set(Calendar.HOUR_OF_DAY, 0);
        c.set(Calendar.MINUTE, 0);
        c.set(Calendar.SECOND, 0);
        c.set(Calendar.MILLISECOND, 0);
        return c.getTime();
    }

    public String getTitulo() { return titulo; }
    public Double getPrecio() { return precio; }
    public Integer getPaginas() { return paginas; }
    public java.util.Date getFechaPublicacion() { return fechaPublicacion; }
    public Boolean getDisponible() { return disponible; }

    public static List<Libro> listaEjemplo() {
        List<Libro> libros = new ArrayList<Libro>();
        libros.add(new Libro("Cien años de soledad", 19.95, 471, 1967, Boolean.TRUE));
        libros.add(new Libro("Rayuela", 22.50, 736, 1963, Boolean.TRUE));
        libros.add(new Libro("La ciudad y los perros", 18.75, 432, 1963, Boolean.TRUE));
        libros.add(new Libro("Pedro Páramo", 15.90, 132, 1955, Boolean.TRUE));
        libros.add(new Libro("Ficciones", 21.00, 224, 1944, Boolean.TRUE));
        libros.add(new Libro("La casa de los espíritus", 23.40, 448, 1982, Boolean.TRUE));
        libros.add(new Libro("El amor en los tiempos del cólera", 20.80, 496, 1985, Boolean.TRUE));
        libros.add(new Libro("La muerte de Artemio Cruz", 17.60, 320, 1962, Boolean.TRUE));
        libros.add(new Libro("Doña Bárbara", 16.95, 400, 1929, Boolean.FALSE));
        libros.add(new Libro("Martín Fierro", 14.50, 288, 1872, Boolean.FALSE));
        libros.add(new Libro("Comala", 19.20, 148, 1955, Boolean.TRUE));
        libros.add(new Libro("Paradiso", 25.00, 576, 1966, Boolean.TRUE));
        libros.add(new Libro("La invención de Morel", 18.30, 128, 1940, Boolean.TRUE));
        libros.add(new Libro("El túnel", 16.20, 160, 1948, Boolean.FALSE));
        return libros;
    }
}
```

Explicación línea por línea

Línea 1	<code>import java.util.ArrayList;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 2	<code>import java.util.Calendar;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 3	<code>import java.util.GregorianCalendar;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 4	<code>import java.util.List;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 6	<code>public class Libro {</code> → declara la clase Java y, si aparece <code>implements</code> , establece el contrato que debe implementar.
Línea 7	<code>private final String titulo;</code> → declara un campo de instancia inmutable después de la construcción del objeto.

Línea 8	<code>private final Double precio;</code> → declara un campo de instancia inmutable después de la construcción del objeto.
Línea 9	<code>private final Integer paginas;</code> → declara un campo de instancia inmutable después de la construcción del objeto.
Línea 10	<code>private final java.util.Date fechaPublicacion;</code> → declara un campo de instancia inmutable después de la construcción del objeto.
Línea 11	<code>private final Boolean disponible;</code> → declara un campo de instancia inmutable después de la construcción del objeto.
Línea 13	<code>public Libro(String titulo, Double precio, Integer paginas, int anioPublicacion, Boolean disponible) {</code> → declara el constructor completo del modelo Libro con los valores que necesita el informe.
Línea 14	<code>this.titulo = titulo;</code> → asigna al campo del objeto el valor recibido o calculado.
Línea 15	<code>this.precio = precio;</code> → asigna al campo del objeto el valor recibido o calculado.
Línea 16	<code>this.paginas = paginas;</code> → asigna al campo del objeto el valor recibido o calculado.
Línea 17	<code>this.fechaPublicacion = fecha(anioPublicacion);</code> → asigna al campo del objeto el valor recibido o calculado.
Línea 18	<code>this.disponible = disponible;</code> → asigna al campo del objeto el valor recibido o calculado.
Línea 19	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.
Línea 21	<code>private static java.util.Date fecha(int anio) {</code> → abre el método auxiliar que convierte un año en una fecha Java reproducible.
Línea 22	<code>Calendar c = new GregorianCalendar(anio, Calendar.JANUARY, 1);</code> → crea un calendario situado el 1 de enero del año indicado.
Línea 23	<code>c.set(Calendar.HOUR_OF_DAY, 0);</code> → normaliza un componente horario del calendario para obtener una fecha estable.
Línea 24	<code>c.set(Calendar.MINUTE, 0);</code> → normaliza un componente horario del calendario para obtener una fecha estable.
Línea 25	<code>c.set(Calendar.SECOND, 0);</code> → normaliza un componente horario del calendario para obtener una fecha estable.
Línea 26	<code>c.set(Calendar.MILLISECOND, 0);</code> → normaliza un componente horario del calendario para obtener una fecha estable.
Línea 27	<code>return c.getTime();</code> → devuelve la fecha construida por el calendario.
Línea 28	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.
Línea 30	<code>public String getTitulo() { return titulo; }</code> → devuelve el título del libro.
Línea 31	<code>public Double getPrecio() { return precio; }</code> → devuelve el precio del libro.
Línea 32	<code>public Integer getPaginas() { return paginas; }</code> → devuelve el número de páginas.
Línea 33	<code>public java.util.Date getFechaPublicacion() { return fechaPublicacion; }</code> → devuelve la fecha de publicación.
Línea 34	<code>public Boolean getDisponible() { return disponible; }</code> → devuelve el estado de disponibilidad.
Línea 36	<code>public static List<Libro> listaEjemplo() {</code> → abre el método que construye los datos de ejemplo del curso.
Línea 37	<code>List<Libro> libros = new ArrayList<Libro>();</code> → crea una lista tipada compatible con Java 8 y con la baseline del proyecto.
Línea 38	<code>libros.add(new Libro("Cien años de soledad", 19.95, 471, 1967, Boolean.TRUE));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.

Línea 39	<code>libros.add(new Libro("Rayuela", 22.50, 736, 1963, Boolean.TRUE));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.
Línea 40	<code>libros.add(new Libro("La ciudad y los perros", 18.75, 432, 1963, Boolean.TRUE));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.
Línea 41	<code>libros.add(new Libro("Pedro Páramo", 15.90, 132, 1955, Boolean.TRUE));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.
Línea 42	<code>libros.add(new Libro("Ficciones", 21.00, 224, 1944, Boolean.TRUE));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.
Línea 43	<code>libros.add(new Libro("La casa de los espíritus", 23.40, 448, 1982, Boolean.TRUE));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.
Línea 44	<code>libros.add(new Libro("El amor en los tiempos del cólera", 20.80, 496, 1985, Boolean.TRUE));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.
Línea 45	<code>libros.add(new Libro("La muerte de Artemio Cruz", 17.60, 320, 1962, Boolean.TRUE));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.
Línea 46	<code>libros.add(new Libro("Doña Bárbara", 16.95, 400, 1929, Boolean.FALSE));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.
Línea 47	<code>libros.add(new Libro("Martín Fierro", 14.50, 288, 1872, Boolean.FALSE));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.
Línea 48	<code>libros.add(new Libro("Comala", 19.20, 148, 1955, Boolean.TRUE));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.
Línea 49	<code>libros.add(new Libro("Paradiso", 25.00, 576, 1966, Boolean.TRUE));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.
Línea 50	<code>libros.add(new Libro("La invención de Morel", 18.30, 128, 1940, Boolean.TRUE));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.
Línea 51	<code>libros.add(new Libro("El túnel", 16.20, 160, 1948, Boolean.FALSE));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.
Línea 52	<code>return libros;</code> → devuelve la lista completa que alimentará la fuente de datos.
Línea 53	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.
Línea 54	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.

Clase CatalogoDataSource.java

JAVA

```
import java.util.List;
import net.sf.jasperreports.engine.JRDataSource;
import net.sf.jasperreports.engine.JRException;
import net.sf.jasperreports.engine.JRField;

public class CatalogoDataSource implements JRDataSource {
    private final List<Libro> libros;
    private int indice = -1;

    public CatalogoDataSource(List<Libro> libros) {
        this.libros = libros;
    }

    @Override
    public boolean next() throws JRException {
        indice++;
        return indice < libros.size();
    }

    @Override
    public Object getFieldValue(JRField campo) throws JRException {
        Libro actual = libros.get(indice);
        if ("titulo".equals(campo.getName())) return actual.getTitulo();
        if ("precio".equals(campo.getName())) return actual.getPrecio();
        if ("paginas".equals(campo.getName())) return actual.getPaginas();
        if ("fechaPublicacion".equals(campo.getName())) return actual.getFechaPublicacion();
    }
}
```

```

        if ("disponible".equals(campo.getName())) return actual.getDisponible();
        throw new JRException("Campo no soportado por CatalogoDataSource: " + campo.getName());
    }
}

```

Explicación línea por línea

Línea 1	<code>import java.util.List;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 2	<code>import net.sf.jasperreports.engine.JRDataSource;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 3	<code>import net.sf.jasperreports.engine.JRException;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 4	<code>import net.sf.jasperreports.engine.JRField;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 6	<code>public class CatalogoDataSource implements JRDataSource {</code> → declara la clase Java y, si aparece <code>implements</code> , establece el contrato que debe implementar.
Línea 7	<code>private final List<Libro> libros;</code> → declara un campo de instancia inmutable después de la construcción del objeto.
Línea 8	<code>private int indice = -1;</code> → declara el índice interno de la fuente de datos; empieza en -1 porque <code>next()</code> se invoca antes de leer el primer registro.
Línea 10	<code>public CatalogoDataSource(List<Libro> libros) {</code> → declara el constructor de la fuente de datos y recibe la lista de libros.
Línea 11	<code>this.libros = libros;</code> → asigna al campo del objeto el valor recibido o calculado.
Línea 12	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.
Línea 14	<code>@Override</code> → indica que el método implementa un método del contrato <code>JRDataSource</code> .
Línea 15	<code>public boolean next() throws JRException {</code> → implementa <code>JRDataSource.next()</code> y declara <code>JRException</code> según el contrato de JasperReports.
Línea 16	<code>indice++;</code> → avanza al siguiente registro.
Línea 17	<code>return indice < libros.size();</code> → indica al motor si todavía existe un registro válido.
Línea 18	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.
Línea 20	<code>@Override</code> → indica que el método implementa un método del contrato <code>JRDataSource</code> .
Línea 21	<code>public Object getFieldValue(JRField campo) throws JRException {</code> → implementa la resolución de un campo JRXML para el registro actual.
Línea 22	<code>Libro actual = libros.get(indice);</code> → obtiene el libro correspondiente al índice actual.
Línea 23	<code>if ("titulo".equals(campo.getName())) return actual.getTitulo();</code> → resuelve el campo <code>titulo</code> .
Línea 24	<code>if ("precio".equals(campo.getName())) return actual.getPrecio();</code> → resuelve el campo <code>precio</code> .
Línea 25	<code>if ("paginas".equals(campo.getName())) return actual.getPaginas();</code> → resuelve el campo <code>paginas</code> .
Línea 26	<code>if ("fechaPublicacion".equals(campo.getName())) return actual.getFechaPublicacion();</code> → resuelve el campo <code>fechaPublicacion</code> .
Línea 27	<code>if ("disponible".equals(campo.getName())) return actual.getDisponible();</code> → resuelve el campo <code>disponible</code> .
Línea 28	<code>throw new JRException("Campo no soportado por CatalogoDataSource: " + campo.getName());</code> → falla explícitamente si el JRXML solicita un campo que la fuente no soporta, evitando devolver silenciosamente un valor incorrecto.
Línea 29	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.

Línea 30 } → abre o cierra el bloque sintáctico correspondiente.

Clase GeneradorInformeConcepto.java

JAVA

```
import java.io.File;
import java.util.HashMap;
import java.util.Map;

import net.sf.jasperreports.engine.JasperCompileManager;
import net.sf.jasperreports.engine.JasperExportManager;
import net.sf.jasperreports.engine.JasperFillManager;
import net.sf.jasperreports.engine.JasperPrint;

public class GeneradorInformeConcepto {
    public static void main(String[] args) {
        try {
            String rutaJrxml = "reports/informe_concepto.jrxml";
            String rutaJasper = "reports/informe_concepto.jasper";
            String rutaPdf = "output/informe_concepto.pdf";

            JasperCompileManager.compileReportToFile(rutaJrxml, rutaJasper);

            Map<String, Object> parametros = new HashMap<String, Object>();

            JasperPrint documento = JasperFillManager.fillReport(
                rutaJasper,
                parametros,
                new CatalogoDataSource(Libro.listaEjemplo()));

            JasperExportManager.exportReportToPdfFile(documento, rutaPdf);

            System.out.println("Informe generado en: " + new File(rutaPdf).getAbsolutePath());
            System.out.println("Paginas del documento: " + documento.getPages().size());
            System.out.println("Registros de ejemplo: " + Libro.listaEjemplo().size());
        } catch (Exception e) {
            e.printStackTrace();
            System.exit(1);
        }
    }
}
```

Explicación línea por línea

Línea 1	import java.io.File; → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 2	import java.util.HashMap; → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 3	import java.util.Map; → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 5	import net.sf.jasperreports.engine.JasperCompileManager; → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 6	import net.sf.jasperreports.engine.JasperExportManager; → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 7	import net.sf.jasperreports.engine.JasperFillManager; → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 8	import net.sf.jasperreports.engine.JasperPrint; → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 10	public class GeneradorInformeConcepto { → declara la clase Java y, si aparece implements, establece el contrato que debe implementar.
Línea 11	public static void main(String[] args) { → declara el punto de entrada de la aplicación.
Línea 12	try { → abre el bloque protegido de ejecución.
Línea 13	String rutaJrxml = "reports/informe_concepto.jrxml"; → define la ruta relativa de la plantilla JRXML.

Línea 14	<code>String rutaJasper = "reports/informe_concepto.jasper";</code> → define la ruta del artefacto compilado <code>.jasper</code> .
Línea 15	<code>String rutaPdf = "output/informe_concepto.pdf";</code> → define la ruta del PDF de salida.
Línea 17	<code>JasperCompileManager.compileReportToFile(rutaJrxml, rutaJasper);</code> → compila el JRXML con JasperReports Library.
Línea 19	<code>Map<String, Object> parametros = new HashMap<String, Object>();</code> → crea el mapa tipado de parámetros.
Línea 21	<code>JasperPrint documento = JasperFillManager.fillReport(</code> → declara el <code>JasperPrint</code> resultante del llenado.
Línea 22	<code>rutaJasper,</code> → pasa al llenado el informe compilado.
Línea 23	<code>parametros,</code> → pasa el mapa de parámetros.
Línea 24	<code>new CatalogoDataSource(Libro.listaEjemplo());</code> → pasa la fuente de datos construida con los libros de ejemplo.
Línea 26	<code>JasperExportManager.exportReportToPdfFile(documento, rutaPdf);</code> → exporta el <code>JasperPrint</code> a un PDF real.
Línea 28	<code>System.out.println("Informe generado en: " + new File(rutaPdf).getAbsolutePath());</code> → escribe en consola la ruta absoluta del PDF generado.
Línea 29	<code>System.out.println("Paginas del documento: " + documento.getPages().size());</code> → escribe en consola el número real de páginas del <code>JasperPrint</code> .
Línea 30	<code>System.out.println("Registros de ejemplo: " + Libro.listaEjemplo().size());</code> → escribe en consola el número de registros de ejemplo; el workflow usa esta salida como evidencia de ejecución.
Línea 31	<code>} catch (Exception e) {</code> → captura cualquier fallo de compilación, llenado o exportación.
Línea 32	<code>e.printStackTrace();</code> → imprime la traza del error para diagnóstico.
Línea 33	<code>System.exit(1);</code> → termina con código distinto de cero para que GitHub Actions detecte el fallo.
Línea 34	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.
Línea 35	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.
Línea 36	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.

Traza de consola esperada tras la ejecución

TEXT

```
Informe generado en: C:\Users\<usuario>\Documents\JasperProjects\EditorialReports\output\informe_concepto.pdf
Paginas del documento: 1
Registros de ejemplo: 14
```

La ruta depende del equipo. Los valores anteriores están comprobados en el run E2E 36011315054: checkpoint 2.3 → 1 página y 14 registros.

Parte D — Validación del resultado y estructura del proyecto

D.1 — Vista de diseño en Jaspersoft Studio

TEXT

```
+-----+
| informe_concepto.jrxml                                [Design] [Source] |
+-----+
| Ruler: 0  100  200  300  400  500  555  600  650 |
+-----+
|
|  Column Header  h = 25
|  Título         | Precio | Páginas | Año   | Disp. |
|-----|-----|-----|-----|-----|
|
```

```

    Detail 1 ----- h = 20
    [ $F{titulo} ] [ $F{precio} ] [ $F{pag} ] [ $F{fech} ] [ $F{disp} ]
    [ $V{REPORT_COUNT} ]

```

Panel Outline muestra:

Fields

- titulo [java.lang.String]
- precio [java.lang.Double]
- paginas [java.lang.Integer]
- fechaPublicacion [java.util.Date]
- disponible [java.lang.Boolean]

Qué representa: la disposición de la banda Column Header y de la banda Detail con los nuevos campos. La tabla de datos tiene ahora seis columnas: Título, Precio, Páginas, Año, Disponible y Número de registro.

Cómo verificarlo: comparar la vista del editor con este esquema. Los encabezados deben aparecer en el orden indicado y los campos de la banda Detail deben estar alineados con sus encabezados.

D.2 — Jerarquía del Outline

TEXT

```

informe_concepto
├── Fields
│   ├── titulo [java.lang.String]
│   ├── precio [java.lang.Double]
│   ├── paginas [java.lang.Integer]
│   ├── fechaPublicacion [java.util.Date]
│   └── disponible [java.lang.Boolean]
├── Column Header [band, height=25]
│   ├── staticText "Título" [x=0, width=245, bold]
│   ├── staticText "Precio" [x=245, width=80, bold, right]
│   ├── staticText "Páginas" [x=325, width=55, bold, right]
│   ├── staticText "Año" [x=380, width=50, bold, center]
│   ├── staticText "Disp." [x=430, width=65, bold, center]
│   └── staticText "#" [x=495, width=60, bold, right]
└── Detail 1 [band, height=20, splitType=Stretch]
    ├── textField [x=0, width=245, textAdjust=StretchHeight] $F{titulo}
    ├── textField [x=245, width=80, pattern=##,##0.00 €] $F{precio}
    ├── textField [x=325, width=55, right] $F{paginas}
    ├── textField [x=380, width=50, pattern=yyyy, center] $F{fechaPublicacion}
    ├── textField [x=430, width=65, center] $F{disponible} ? "Sí" : "No"
    └── textField [x=495, width=60, right] $V{REPORT_COUNT}

```

Qué representa: el árbol de nodos del checkpoint 2.3. El rótulo # está en Column Header; en Detail sólo se imprime el valor de \$V{REPORT_COUNT}. Las seis columnas terminan exactamente en $x + width = 555$.

Cómo verificarlo: expandir Fields, Column Header y Detail 1 en Outline y comprobar después X y Width en Properties para cada elemento.

D.3 — Documento PDF resultante, página por página

TEXT

INFORME: informe_concepto.pdf
 PÁGINAS TOTALES: 1
 TAMAÑO DE PÁGINA: 595 × 842 unidades de informe (A4 vertical)
 REGISTROS PROCESADOS: 14
 CAMPOS RESUELTOS POR REGISTRO: 5

————— Página 1 de 1 —————

Título	Precio	Páginas	Año	Disp.	#
Cien años de sol.	19,95 €	471	1967	Sí	1
Rayuela	22,50 €	736	1963	Sí	2
La ciudad y los...	18,75 €	432	1963	Sí	3
Pedro Páramo	15,90 €	132	1955	Sí	4
Ficciones	21,00 €	224	1944	Sí	5
La casa de los...	23,40 €	448	1982	Sí	6
El amor en los...	20,80 €	496	1985	Sí	7
La muerte de Ar...	17,60 €	320	1962	Sí	8
Doña Bárbara	16,95 €	400	1929	No	9
Martín Fierro	14,50 €	288	1872	No	10
Comala	19,20 €	148	1955	Sí	11
Paradiso	25,00 €	576	1966	Sí	12
La invención de...	18,30 €	128	1940	Sí	13

```

El túnel      | 16,20 € | 160 | 1948 | No | 14 |
--- Fin de la tabla de datos ---
Registros procesados: 14
Documento generado en la última página
Total de páginas: 1
Fin del informe. EditorialReports.
Total de libros: 14

```

Qué representa: el resultado del checkpoint 2.3 con los valores realmente definidos en `Libro.listaEjemplo()`. Páginas, año y disponibilidad cambian por libro; Doña Bárbara, Martín Fierro y El túnel usan `Boolean.FALSE` y se muestran como No.

Cómo verificarlo: abrir `output/informe_concepto.pdf` o el artefacto `M2-2.3-runtime`. El run E2E final confirma Páginas del documento: 1 y Registros de ejemplo: 14.

D.4 — Árbol de carpetas del proyecto tras completar el punto

TEXT

```

EditorialReports/
├── ECOSISTEMA.md                (documentación del ecosistema)
├── ENTORNO.md                  (documentación del entorno)
├── BANDAS.md                   (documentación de las bandas)
├── JRXML.md                    (documentación del formato JRXML)
├── TEXTO.md                    (documentación de elementos textuales)
├── CAMPOS.md                   (documentación de campos)
├── reports/
│   ├── informe_concepto.jrxml   (plantilla con 5 campos)
│   └── informe_concepto.jasper  (artefacto compilado)
├── resources/
│   └── (vacía en este punto)
├── output/
│   └── informe_concepto.pdf      (documento con 14 registros)
└── EditorialReportsJava/
    ├── lib/
    │   └── README.md            (el runtime real se resuelve con Maven)
    └── src/
        ├── GeneradorInformeConcepto.java
        ├── Libro.java           (ampliada con 3 campos)
        └── CatalogoDataSource.java (ampliada con 3 casos)

```

Qué representa: el estado de los dos proyectos tras completar los doce pasos. La novedad respecto al punto anterior es el archivo `CAMPOS.md` en la raíz del proyecto `EditorialReports` y la ampliación de las clases `Libro` y `CatalogoDataSource`.

Cómo verificarlo: expandir los nodos del panel Project Explorer y comparar con este esquema. Si el archivo `CAMPOS.md` no aparece, repetir el paso 12.

Errores comunes del ejercicio completo

Error	Causa	Solución
Field not found: paginas al compilar	El campo no está declarado en el JRXML	Añadir <code><field name="paginas" class="java.lang.Integer"/></code> antes de las bandas
ClassCastException: java.lang.Double cannot be cast to java.lang.Integer	El tipo declarado no coincide con el valor devuelto	Cambiar el tipo del campo al tipo del valor devuelto por la fuente
El campo disponible se muestra siempre como Sí	El constructor asigna <code>Boolean.TRUE</code> a todos los registros	Modificar el constructor o el método <code>listaEjemplo</code> para asignar valores distintos según el libro
La expresión del operador ternario no compila	Se usó <code>#{disponible}</code> sin invocar <code>booleanValue()</code>	Escribir <code>#{disponible}.booleanValue() ? "Sí" : "No"</code>
La columna de páginas aparece vacía	El método getter devuelve null o el campo no está resuelto	Verificar que <code>Libro.getPaginas()</code> devuelve un valor no nulo
La columna de fecha muestra 2026 en todos los registros	El constructor asigna <code>new java.util.Date()</code> a todos los libros	Modificar el constructor para recibir la fecha como parámetro o asignar fechas distintas
Los encabezados de la banda Column Header no se alinean con los datos	Las coordenadas X de los encabezados no coinciden con las de los campos	Igualar las coordenadas X de cada par encabezado-campo

Error	Causa	Solución
El campo paginas muestra 0 en todos los registros	El constructor asigna 0 por defecto	Modificar el constructor o el método listaEjemplo para asignar valores distintos
El informe produce StackOverflowError al llenar	Un campo se resuelve a sí mismo en el getFieldValue	Revisar el método getFieldValue y asegurarse de que devuelve valores del objeto actual
El campo disponible no muestra No en ningún registro	El constructor asigna Boolean.TRUE a todos	Modificar el constructor para asignar Boolean.FALSE a algunos libros

Reto resuelto paso a paso

Enunciado: demostrar que un cambio en el modelo Java llega realmente al PDF y, después, restaurar el checkpoint. Cambiaremos temporalmente las páginas de Pedro Páramo de 132 a 133 y la disponibilidad de El túnel de FALSE a TRUE, verificaremos el resultado y dejaremos de nuevo los valores canónicos.

- Paso 1. Hacer doble clic sobre EditorialReportsJava/src/Libro.java en Project Explorer.
- Paso 2. Localizar la llamada `new Libro("Pedro Páramo", 15.90, 132, 1955, Boolean.TRUE)`.
- Paso 3. Sustituir temporalmente 132 por 133 y pulsar Ctrl+S.
- Paso 4. Localizar `new Libro("El túnel", 16.20, 160, 1948, Boolean.FALSE)`.
- Paso 5. Sustituir temporalmente `Boolean.FALSE` por `Boolean.TRUE` y pulsar Ctrl+S.
- Paso 6. Ejecutar `GeneradorInformeConcepto.java` como **Java Application** con `EditorialReports` como Working Directory.
- Paso 7. Abrir `output/informe_concepto.pdf` y comprobar que Pedro Páramo muestra 133 páginas y El túnel muestra Sí.
- Paso 8. Volver a `Libro.java` y restaurar Pedro Páramo a 132 páginas.
- Paso 9. Restaurar El túnel a `Boolean.FALSE` y pulsar Ctrl+S.
- Paso 10. Ejecutar de nuevo `GeneradorInformeConcepto.java`.
- Paso 11. Abrir el PDF regenerado y comprobar que Pedro Páramo vuelve a 132 y El túnel vuelve a No.
- Paso 12. Comprobar en Console Páginas del documento: 1 y Registros de ejemplo: 14.

Simulación ASCII del PDF durante la prueba temporal

TEXT

```
|| Pedro Páramo | 15,90 € | 133 | 1955 | Sí | 4 ||
|| El túnel | 16,20 € | 160 | 1948 | Sí | 14 ||
```

Simulación ASCII tras restaurar el checkpoint

TEXT

```
|| Pedro Páramo | 15,90 € | 132 | 1955 | Sí | 4 ||
|| El túnel | 16,20 € | 160 | 1948 | No | 14 ||
```

Resultado del reto: se comprueba de forma controlada que `CatalogoDataSource` entrega al JRXML los valores del objeto `Libro` actual y que el PDF refleja esos cambios. Al terminar el reto, el código queda exactamente con los valores del checkpoint 2.3.

Analogía final con el contexto de la editorial

Los campos son los datos que se extraen de la ficha de cada libro del catálogo. El título es un campo de texto. El precio es un campo numérico. El número de páginas es un campo entero. La fecha de publicación es un campo de fecha. La disponibilidad es un campo booleano. Cada campo tiene un tipo distinto y el motor lo resuelve de forma diferente según el tipo. La declaración del campo en el JRXML es como la etiqueta que el editor coloca en la ficha del manuscrito para indicar qué dato debe extraerse. La resolución del campo en la fuente de datos es como el trabajo del asistente que busca el dato en el manuscrito y lo transcribe en la ficha. La combinación de campos y expresiones construye la tabla del catálogo, fila a fila, con toda la información descriptiva de cada libro.

Resultado esperado

- Al finalizar este punto, el alumno dispone de:
- La clase Libro ampliada con tres campos nuevos (paginas, fechaPublicacion, disponible) y sus métodos getter.
- El método listaEjemplo() ampliado con catorce libros y datos realistas para cada uno.
- El archivo reports/informe_concepto.jrxml con cinco campos declarados y cinco expresiones \$F{} en la banda Detail.
- El archivo output/informe_concepto.pdf con las seis columnas de la tabla y los catorce registros.
- El archivo CAMPOS.md en la raíz del proyecto con la tabla de campos documentada.
- Comprensión operativa de la declaración de campos, de la resolución desde la fuente de datos y de las diferencias entre campo, parámetro y variable.

Conclusión y enlace al siguiente punto

El punto 2.3 ha conectado cinco campos JRXML con una fuente de datos Java real y ha mostrado cómo el tipo declarado en el informe debe corresponder con el objeto devuelto por `JRDataSource`. El catálogo dispone ya de título, precio, páginas, fecha de publicación y disponibilidad.

El punto 2.4, «Imágenes», utiliza esos datos para enriquecer visualmente el documento con logotipo, portadas e iconos condicionales sin romper el flujo de llenado.

Punto 2.4 — Imágenes

PUNTO DE PARTIDA. Si vienes haciendo el curso, continúa con tu propio proyecto del punto anterior. Si te incorporas directamente aquí, usa M2/2.3 como estado inicial. El checkpoint M2/2.4 contiene la solución completa de este punto y no debe consultarse antes del ejercicio si quieres evitar spoilers.

Parte A — Práctica visual

Paso 1: Crear la carpeta resources y añadir el logotipo VALIDADO

Acciones:

1. Abrir el explorador de archivos del sistema operativo.
2. Navegar hasta la carpeta Documents\JasperProjects\EditorialReports.
3. Hacer clic con el botón derecho sobre una zona vacía de la carpeta y seleccionar Nuevo > Carpeta.
4. Escribir exactamente resources en el campo de nombre.
5. Pulsar Enter para confirmar.
6. Copiar el archivo logo.png desde la carpeta de recursos del curso a la nueva carpeta resources.
7. Volver a Jaspersoft Studio.
8. Hacer clic con el botón derecho sobre el nodo EditorialReports en el panel Project Explorer (superior izquierdo).
9. Hacer clic sobre la opción Refresh en el menú contextual.
10. Expandir el nodo EditorialReports y verificar que aparece la carpeta resources con el archivo logo.png.

Verificación visual: el panel Project Explorer muestra la carpeta resources con el archivo logo.png en su interior.

Qué hace: crea la carpeta de recursos y coloca el logotipo que se va a utilizar en el informe.

Por qué: la carpeta resources es la ubicación convencional para los recursos auxiliares del proyecto.

Error común: copiar el archivo con un nombre distinto a logo.png. La expresión del JRXML busca el archivo por el nombre exacto.

Solución: renombrar el archivo a logo.png.

Analogía: es como colocar el logotipo de la editorial en la carpeta de materiales gráficos del catálogo.

Paso 2: Añadir el logotipo en la banda Title VALIDADO

Acciones:

1. Hacer doble clic sobre el archivo informe_concepto.jrxml en el panel Project Explorer (superior izquierdo).
2. Hacer clic sobre la pestaña Design en la parte inferior del editor central.
3. Hacer clic sobre el nodo Title en el panel Outline (inferior izquierdo).
4. Hacer clic sobre el campo Band height en el panel Properties (inferior derecho), pestaña Properties, escribir 100 y pulsar Enter.
5. Hacer clic sobre la pestaña Elements en el panel Palette (derecha del editor central).
6. Hacer clic sobre el icono Image (un cuadrado con un paisaje).
7. Arrastrar el icono Image y soltarlo dentro de la banda Title, en la coordenada aproximada x=0, y=10.
8. Hacer clic sobre el campo X en el panel Properties, escribir 0 y pulsar Enter.
9. Hacer clic sobre el campo Y, escribir 10 y pulsar Enter.
10. Hacer clic sobre el campo Width, escribir 80 y pulsar Enter.
11. Hacer clic sobre el campo Height, escribir 80 y pulsar Enter.
12. Hacer clic sobre el desplegable Scale Image y seleccionar RetainShape.

- Hacer clic sobre el campo Image Expression y escribir exactamente "resources/logo.png" (con comillas dobles) y pulsar Enter.

Verificación visual: la banda Title muestra el logotipo en la esquina superior izquierda. Si aparece un icono de imagen rota, la ruta es incorrecta.

Qué hace: inserta el logotipo de la editorial en la banda de título.

Por qué: el logotipo identifica la editorial en la portada del catálogo.

Error común: escribir la ruta sin comillas dobles y provocar que JasperReports la interprete como una expresión en lugar de una cadena.

Solución: escribir la ruta entre comillas dobles: "resources/logo.png".

Analogía: es como poner el sello de la editorial en la portada del catálogo.

Paso 3: Mover el título a la derecha del logotipo VALIDADO

Acciones:

- Hacer clic sobre el primer Static Text de la banda Title en el editor central (el que contiene el texto Catálogo Editorial - Informe Conceptual).
- Hacer clic sobre el campo X en el panel Properties, pestaña Properties, escribir 90 y pulsar Enter.
- Hacer clic sobre el campo Y, escribir 25 y pulsar Enter.
- Hacer clic sobre el campo Width, escribir 465 y pulsar Enter.
- Hacer clic sobre el campo Height, escribir 30 y pulsar Enter.
- Hacer clic sobre el desplegable Horizontal Text Alignment y seleccionar Left.
- Hacer clic sobre el campo Font size y escribir 18. Pulsar Enter.

Verificación visual: el título aparece a la derecha del logotipo, alineado a la izquierda.

Qué hace: desplaza el título principal para dejar espacio al logotipo.

Por qué: el logotipo ocupa la esquina izquierda de la banda y el título debe colocarse a su derecha.

Error común: dejar el título en su posición original y provocar el solapamiento con el logotipo.

Solución: ajustar la coordenada X del título a 90.

Analogía: es como reorganizar la portada del catálogo para que el sello y el título convivan sin solaparse.

Paso 4: Ajustar el rótulo de fecha y el campo de fecha VALIDADO

Acciones:

- Hacer clic sobre el Static Text que contiene el texto Fecha de emisión: en la banda Title.
- Hacer clic sobre el campo X en el panel Properties, pestaña Properties, escribir 90 y pulsar Enter.
- Hacer clic sobre el campo Y, escribir 60 y pulsar Enter.
- Hacer clic sobre el campo Width, escribir 120 y pulsar Enter.
- Hacer clic sobre el campo Height, escribir 20 y pulsar Enter.
- Hacer clic sobre el Text Field que contiene la expresión new java.util.Date() en la banda Title.
- Hacer clic sobre el campo X, escribir 215 y pulsar Enter.
- Hacer clic sobre el campo Y, escribir 60 y pulsar Enter.
- Hacer clic sobre el campo Width, escribir 150 y pulsar Enter.
- Hacer clic sobre el campo Height, escribir 20 y pulsar Enter.

Verificación visual: el rótulo de fecha y el campo con la fecha aparecen debajo del título, alineados a la derecha del logotipo.

Qué hace: desplaza el rótulo de fecha y el campo de fecha para alinearlos con el título.

Por qué: el logotipo ocupa la franja izquierda de la banda y el resto de elementos deben desplazarse a la derecha.

Error común: dejar el rótulo y el campo en sus posiciones originales y provocar el solapamiento con el logotipo.

Solución: ajustar las coordenadas X de ambos elementos a 90 y 215 respectivamente.

Analogía: es como reorganizar la portada del catálogo para que todos los datos queden alineados tras el sello.

Paso 5: Añadir una carpeta para las portadas de los libros VALIDADO

Acciones:

1. Abrir el explorador de archivos del sistema operativo.
2. Navegar hasta la carpeta Documents\JasperProjects\EditorialReports\resources.
3. Hacer clic con el botón derecho sobre una zona vacía de la carpeta y seleccionar Nuevo > Carpeta.
4. Escribir exactamente portadas en el campo de nombre.
5. Pulsar Enter para confirmar.
6. Copiar tres archivos PNG de ejemplo a la carpeta portadas con los nombres Cien años de soledad.png, Rayuela.png y Pedro Páramo.png.
7. Volver a Jaspersoft Studio.
8. Hacer clic con el botón derecho sobre el nodo EditorialReports en el panel Project Explorer y seleccionar Refresh.
9. Expandir la carpeta resources y verificar que aparece la subcarpeta portadas con los tres archivos.

Verificación visual: el panel Project Explorer muestra la carpeta resources/portadas con los tres archivos PNG.

Qué hace: crea la carpeta para las portadas de los libros y coloca tres imágenes de ejemplo.

Por qué: el informe mostrará la portada de cada libro en la banda Detail si el archivo existe.

Error común: copiar los archivos con nombres que no coincidan con los títulos de los libros.

Solución: nombrar cada archivo con el título del libro seguido de .png.

Analogía: es como preparar las portadas de los libros en la carpeta de materiales gráficos del catálogo.

Paso 6: Añadir la imagen dinámica de portada en la banda Detail VALIDADO

Acciones:

1. Hacer clic sobre el nodo Detail 1 en el panel Outline (inferior izquierdo).
2. Hacer clic sobre el campo Band height en el panel Properties, pestaña Properties, escribir 60 y pulsar Enter.
3. Hacer clic sobre el desplegable Split Type y seleccionar Prevent.
4. Hacer clic sobre la pestaña Elements en el panel Palette (derecha del editor central).
5. Hacer clic sobre el icono Image (un cuadrado con un paisaje).
6. Arrastrar el icono Image y soltarlo dentro de la banda Detail 1, en la coordenada aproximada x=0, y=5.
7. Hacer clic sobre el campo X en el panel Properties, escribir 0 y pulsar Enter.
8. Hacer clic sobre el campo Y, escribir 5 y pulsar Enter.
9. Hacer clic sobre el campo Width, escribir 50 y pulsar Enter.
10. Hacer clic sobre el campo Height, escribir 50 y pulsar Enter.
11. Hacer clic sobre el desplegable Scale Image y seleccionar RetainShape.
12. Hacer clic sobre el desplegable On Error Type y seleccionar Blank.
13. Hacer clic sobre el campo Image Expression y escribir exactamente "resources/portadas/" + \$F{titulo} + ".png" y pulsar Enter.

Verificación visual: la banda Detail 1 tiene Height=60, Split Type=Prevent y muestra un elemento de imagen con la expresión dinámica. Si el archivo no existe, el espacio queda vacío.

Qué hace: amplía Detail, configura Prevent para intentar mantener cada ficha unida en el primer intento de paginación e inserta una imagen dinámica que muestra la portada de cada libro.

Por qué: la portada identifica visualmente cada libro del catálogo.

Error común: olvidar configurar onErrorType="Blank" y provocar que el informe falle cuando algún libro no tiene portada.

Solución: seleccionar Blank en el desplegable On Error Type del panel Properties.

Analogía: es como mostrar la cubierta de cada libro junto a sus datos en el catálogo.

Paso 7: Compactar los campos de Detail para dejar espacio a la portada VALIDADO

Acciones:

1. Seleccionar el Text Field `$F{titulo}` y fijar X=55, Y=20, Width=180, Height=20.
2. Seleccionar `$F{precio}` y fijar X=235, Y=20, Width=80, Height=20.
3. Seleccionar `$F{paginas}` y fijar X=315, Y=20, Width=50, Height=20.
4. Seleccionar `$F{fechaPublicacion}` y fijar X=365, Y=20, Width=50, Height=20.
5. Seleccionar el campo de disponibilidad y fijar X=415, Y=20, Width=55, Height=20.
6. Seleccionar `$V{REPORT_COUNT}` y fijar X=500, Y=20, Width=55, Height=20.
7. Mantener la imagen de portada en X=0, Y=5, Width=50, Height=50.

Verificación visual: ninguna caja supera $x + width = 555$.

Qué hace: recompone la tabla para incorporar la portada sin cambiar el tamaño A4 del informe.

Por qué: la imagen ocupa una franja nueva de 50 unidades y obliga a redistribuir el resto de campos.

Error común: conservar las coordenadas 560/590/620/680 del borrador. Esas coordenadas quedan fuera del columnWidth y no representan el checkpoint ejecutado.

Solución: usar las coordenadas validadas del checkpoint: disponibilidad X=415/Width=55, icono X=475/Width=20 y contador X=500/Width=55.

Analogía: es como reservar en cada ficha del catálogo una columna fija para la miniatura de la portada.

Paso 8: Añadir el encabezado de portada y compactar Column Header VALIDADO

Acciones:

1. En Column Header, crear Port. en X=0, Y=5, Width=50, Height=15, centrado y en negrita.
2. Mover Título a X=55, Width=180.
3. Mover Precio a X=235, Width=80, alineado a la derecha.
4. Mover Págs. a X=315, Width=50, alineado a la derecha.
5. Mover Año a X=365, Width=50, centrado.
6. Mover Disp. a X=415, Width=80, centrado.
7. Mover # a X=500, Width=55, alineado a la derecha.

Verificación visual: la cabecera replica exactamente las columnas del Detail.

Qué hace: adapta la cabecera a la misma geometría usada por Detail después de incorporar la portada.

Por qué: cabecera y datos deben compartir exactamente los mismos límites de columna.

Error común: mantener las coordenadas del punto 2.3 y desalinear títulos y datos. **Solución:** aplicar las coordenadas indicadas a todos los encabezados.

Analogía: es como volver a trazar con regla la cabecera de la tabla después de añadir una columna de imágenes.

Paso 9: Verificar la alineación título-portada VALIDADO

Acciones:

1. Seleccionar el encabezado Título y confirmar X=55, Width=180.
2. Seleccionar el campo `$F{titulo}` y confirmar X=55, Width=180.
3. Confirmar que la portada ocupa X=0 a 50 y queda un margen de 5 unidades antes del título.

Verificación visual: cabecera y datos del título quedan alineados verticalmente.

Qué hace: realiza una comprobación geométrica explícita de la columna de título.

Por qué: una diferencia de pocos píxeles entre cabecera y Detail se percibe como un defecto de maquetación.

Error común: verificar solo visualmente y dejar valores X/Width distintos. **Solución:** comprobar ambos elementos en Properties.

Analogía: es como superponer dos reglas para comprobar que la columna superior y la inferior coinciden.

Paso 10: Añadir el icono de disponibilidad VALIDADO

Acciones:

1. Arrastrar un elemento Image a Detail 1.
2. Fijar X=475, Y=20, Width=20, Height=20.
3. Seleccionar RetainShape en Scale Image.
4. Seleccionar Blank en On Error Type.
5. Escribir `$F{disponible}.booleanValue() ? "resources/icono_disponible.png" : "resources/icono_no_disponible.png"` como Image Expression.
6. Confirmar que el contador comienza en X=500.

Verificación visual: el icono queda entre el texto de disponibilidad y el contador, sin salir del ancho útil.

Qué hace: incorpora una representación visual del estado sin desplazar el resto de columnas fuera de página.

Por qué: el icono comunica de un vistazo la disponibilidad y refuerza el texto Sí/No.

Error común: usar `onErrorType="Error"` para un recurso opcional y detener el informe si falta el archivo. **Solución:** usar Blank para estos iconos.

Analogía: es como colocar un pequeño semáforo visual junto al estado textual de cada libro.

Paso 11: Compilar y ejecutar el programa Java VALIDADO

Acciones:

1. Pulsar Ctrl+S para guardar el archivo JRXML.
2. Pulsar Ctrl+Mayús+B para compilar el informe.
3. Hacer clic sobre el panel Problems (inferior) y verificar que no hay errores.
4. Hacer clic con el botón derecho sobre el archivo GeneradorInformeConcepto.java en el panel Project Explorer.
5. Hacer clic sobre la opción Run As en el menú contextual.
6. Hacer clic sobre la opción Java Application en el submenú.
7. Hacer clic sobre la vista Console en el panel inferior y observar el resultado.

Verificación visual: la vista Console muestra la línea Informe generado en: ... con la ruta absoluta del archivo PDF. El panel Problems permanece vacío.

Qué hace: compila el informe y ejecuta el programa Java con las imágenes configuradas.


```

2121212"/><textFieldExpression><![CDATA[new java.util.Date()]]></textFieldExpression></textField>
</band>
</title>

<columnHeader>
  <band height="25">
    <staticText><reportElement x="0" y="5" width="50" height="15" uuid="13131313-1313-1313-1313-131313131313"/><textElement
textAlignment="Center"><font size="9" isBold="true"/></textElement><text><![CDATA[Port.]]></text></staticText>
    <staticText><reportElement x="55" y="5" width="180" height="15" uuid="14141414-1414-1414-1414-141414141414"/
><textElement><font size="9" isBold="true"/></textElement><text><![CDATA[Título]]></text></staticText>
    <staticText><reportElement x="235" y="5" width="80" height="15" uuid="15151515-1515-1515-1515-151515151515"/
><textElement textAlignment="Right"><font size="9" isBold="true"/></textElement><text><![CDATA[Precio]]></text></staticText>
    <staticText><reportElement x="315" y="5" width="50" height="15" uuid="16161616-1616-1616-1616-161616161616"/
><textElement textAlignment="Right"><font size="9" isBold="true"/></textElement><text><![CDATA[Págs.]]></text></staticText>
    <staticText><reportElement x="365" y="5" width="50" height="15" uuid="17171717-1717-1717-1717-171717171717"/
><textElement textAlignment="Center"><font size="9" isBold="true"/></textElement><text><![CDATA[Año]]></text></staticText>
    <staticText><reportElement x="415" y="5" width="80" height="15" uuid="18181818-1818-1818-1818-181818181818"/
><textElement textAlignment="Center"><font size="9" isBold="true"/></textElement><text><![CDATA[Disp.]]></text></staticText>
    <staticText><reportElement x="500" y="5" width="55" height="15" uuid="19191919-1919-1919-1919-191919191919"/
><textElement textAlignment="Right"><font size="9" isBold="true"/></textElement><text><![CDATA[#]]></text></staticText>
  </band>
</columnHeader>

<detail>
  <band height="60" splitType="Prevent">
    <image onErrorType="Blank" scaleImage="RetainShape"><reportElement x="0" y="5" width="50" height="50" uuid="20202020-2020
-2020-2020-202020202020"/><imageExpression><![CDATA["resources/portadas/" + $F{titulo} + ".png"]]></imageExpression></image>
    <textField textAdjust="StretchHeight"><reportElement x="55" y="20" width="180" height="20" uuid="21212121-2121-2121-2121-
212121212121"/><textFieldExpression><![CDATA[$F{titulo}]]></textFieldExpression></textField>
    <textField pattern="#,##0.00 €" isBlankWhenNull="true"><reportElement x="235" y="20" width="80" height="20" uuid="2222222
2-3333-4444-5555-666666666666"/><textFieldExpression><![CDATA[$F{precio}]]></textFieldExpression>
</textField>
    <textField isBlankWhenNull="true"><reportElement x="315" y="20" width="50" height="20" uuid="23232323-2323-2323-2323-2323
23232323"/><textFieldExpression><![CDATA[$F{paginas}]]></textFieldExpression></textField>
    <textField pattern="yyyy" isBlankWhenNull="true"><reportElement x="365" y="20" width="50" height="20" uuid="24242424-2424
-2424-242424242424"/><textFieldExpression><![CDATA[$F{fechaPublicacion}]]></textFieldExpres
sion></textField>
    <textField><reportElement x="415" y="20" width="55" height="20" uuid="25252525-2525-2525-2525-252525252525"/
><textElement textAlignment="Center"><textFieldExpression><![CDATA[$F{disponible}.booleanValue() ? "Sí" : "No"]]></textFieldExpresi
on></textField>
    <image onErrorType="Blank" scaleImage="RetainShape"><reportElement x="475" y="20" width="20" height="20" uuid="26262626-2
626-2626-2626-262626262626"/><imageExpression><![CDATA[$F{disponible}.booleanValue() ? "resources/icono_disponible.png" : "resources/
icono_no_disponible.png"]]></imageExpression></image>
    <textField><reportElement x="500" y="20" width="55" height="20" uuid="27272727-2727-2727-2727-272727272727"/
><textElement textAlignment="Right"><textFieldExpression><![CDATA[$V{REPORT_COUNT}]]></textFieldExpression></textField>
  </band>
</detail>

```

Explicación línea por línea

Línea 1	<title> → abre la sección Title, generada una vez al comienzo del informe.
Línea 2	<band height="100"> → define la banda y su altura; si aparece splitType, establece la política de división.
Línea 3	<image scaleImage="RetainShape" onErrorType="Error"> → abre un elemento de imagen y define su comportamiento ante escala o errores.
Línea 4	<reportElement x="0" y="10" width="80" height="80" uuid="dddddddd-dddd-dddd-dddd-ddddddddddd"/> → fija coordenadas, tamaño, UUID y, cuando existe, el estilo del elemento.
Línea 5	<imageExpression><![CDATA["resources/logo.png"]]></imageExpression> → define la expresión que resuelve el recurso de imagen.
Línea 6	</image> → cierra el elemento o sección abierto correspondiente.
Línea 7	<staticText> → abre un texto estático.
Línea 8	<reportElement x="90" y="25" width="465" height="30" uuid="eeeeeeee-eeee-eeee-eeee-eeeeeeeeeeee"/> → fija coordenadas, tamaño, UUID y, cuando existe, el estilo del elemento.
Línea 9	<textElement verticalAlignment="Middle"></textElement> → configura alineación y/o marcado del texto.
Línea 10	<text><![CDATA[Catálogo Editorial - Informe Conceptual]]></text> → define el contenido literal del elemento estático.
Línea 11	</staticText> → cierra el elemento o sección abierto correspondiente.

Línea 12	<code><staticText><reportElement x="90" y="60" width="120" height="20" uuid="ffffffff-ffff-ffff-ffff-ffffffffffffff"/><text><![CDATA[Fecha de emisión:]]></text></staticText></code> → abre un texto estático.
Línea 13	<code><textField pattern="dd/MM/yyyy"><reportElement x="215" y="60" width="150" height="20" uuid="12121212-1212-1212-1212-121212121212"/><textFieldExpression><![CDATA[new java.util.Date()]]></textFieldExpression></textField></code> → abre un campo de texto dinámico; sus atributos controlan evaluación, formato o nulos.
Línea 14	<code></band></code> → cierra el elemento o sección abierto correspondiente.
Línea 15	<code></title></code> → cierra el elemento o sección abierto correspondiente.
Línea 17	<code><columnHeader></code> → abre Column Header, generado al comienzo de cada columna.
Línea 18	<code><band height="25"></code> → define la banda y su altura; si aparece <code>splitType</code> , establece la política de división.
Línea 19	<code><staticText><reportElement x="0" y="5" width="50" height="15" uuid="13131313-1313-1313-1313-131313131313"/><textElement textAlignment="Center"></textElement><text><![CDATA[Port.]]></text></staticText></code> → abre un texto estático.
Línea 20	<code><staticText><reportElement x="55" y="5" width="180" height="15" uuid="14141414-1414-1414-1414-141414141414"/><textElement></textElement><text><![CDATA[Título]]></text></staticText></code> → abre un texto estático.
Línea 21	<code><staticText><reportElement x="235" y="5" width="80" height="15" uuid="15151515-1515-1515-1515-151515151515"/><textElement textAlignment="Right"></textElement><text><![CDATA[Precio]]></text></staticText></code> → abre un texto estático.
Línea 22	<code><staticText><reportElement x="315" y="5" width="50" height="15" uuid="16161616-1616-1616-1616-161616161616"/><textElement textAlignment="Right"></textElement><text><![CDATA[Págs.]]></text></staticText></code> → abre un texto estático.
Línea 23	<code><staticText><reportElement x="365" y="5" width="50" height="15" uuid="17171717-1717-1717-1717-171717171717"/><textElement textAlignment="Center"></textElement><text><![CDATA[Año]]></text></staticText></code> → abre un texto estático.
Línea 24	<code><staticText><reportElement x="415" y="5" width="80" height="15" uuid="18181818-1818-1818-1818-181818181818"/><textElement textAlignment="Center"></textElement><text><![CDATA[Disp.]]></text></staticText></code> → abre un texto estático.
Línea 25	<code><staticText><reportElement x="500" y="5" width="55" height="15" uuid="19191919-1919-1919-1919-191919191919"/><textElement textAlignment="Right"></textElement><text><![CDATA[#]]></text></staticText></code> → abre un texto estático.
Línea 26	<code></band></code> → cierra el elemento o sección abierto correspondiente.
Línea 27	<code></columnHeader></code> → cierra el elemento o sección abierto correspondiente.
Línea 29	<code><detail></code> → abre Detail, que el motor intenta generar por cada registro.
Línea 30	<code><band height="60" splitType="Prevent"></code> → define la banda y su altura; si aparece <code>splitType</code> , establece la política de división.
Línea 31	<code><image onErrorType="Blank" scaleImage="RetainShape"><reportElement x="0" y="5" width="50" height="50" uuid="20202020-2020-2020-2020-202020202020"/><imageExpression><![CDATA["resources/portadas/" + \${titulo} + ".png"]]></imageExpression></image></code> → abre un elemento de imagen y define su comportamiento ante escala o errores.
Línea 32	<code><textField textAdjust="StretchHeight"><reportElement x="55" y="20" width="180" height="20" uuid="21212121-2121-2121-2121-212121212121"/><textFieldExpression><![CDATA[\${titulo}]]></textFieldExpression></textField></code> → abre un campo de texto dinámico; sus atributos controlan evaluación, formato o nulos.

Línea 33	<code><textField pattern="#,##0.00 €" isBlankWhenNull="true"><reportElement x="235" y="20" width="80" height="20" uuid="2222222-3333-4444-5555-666666666666"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA[{\$F{precio}}]></textFieldExpression></textField></code> → abre un campo de texto dinámico; sus atributos controlan evaluación, formato o nulos.
Línea 34	<code><textField isBlankWhenNull="true"><reportElement x="315" y="20" width="50" height="20" uuid="23232323-2323-2323-2323-232323232323"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA[{\$F{paginas}}]></textFieldExpression></textField></code> → abre un campo de texto dinámico; sus atributos controlan evaluación, formato o nulos.
Línea 35	<code><textField pattern="yyyy" isBlankWhenNull="true"><reportElement x="365" y="20" width="50" height="20" uuid="24242424-2424-2424-2424-242424242424"/><textElement textAlignment="Center"/><textFieldExpression><![CDATA[{\$F{fechaPublicacion}}]></textFieldExpression></textField></code> → abre un campo de texto dinámico; sus atributos controlan evaluación, formato o nulos.
Línea 36	<code><textField><reportElement x="415" y="20" width="55" height="20" uuid="25252525-2525-2525-2525-252525252525"/><textElement textAlignment="Center"/><textFieldExpression><![CDATA[{\$F{disponible}.booleanValue() ? "Sí" : "No"}]></textFieldExpression></textField></code> → abre un campo de texto dinámico; sus atributos controlan evaluación, formato o nulos.
Línea 37	<code><image onErrorType="Blank" scaleImage="RetainShape"><reportElement x="475" y="20" width="20" height="20" uuid="26262626-2626-2626-2626-262626262626"/><imageExpression><![CDATA[{\$F{disponible}.booleanValue() ? "resources/icono_disponible.png" : "resources/icono_no_disponible.png"}]></imageExpression></image></code> → abre un elemento de imagen y define su comportamiento ante escala o errores.
Línea 38	<code><textField><reportElement x="500" y="20" width="55" height="20" uuid="27272727-2727-2727-2727-272727272727"/><textElement textAlignment="Right"/><textFieldExpression><![CDATA[{\$V{REPORT_COUNT}}]></textFieldExpression></textField></code> → abre un campo de texto dinámico; sus atributos controlan evaluación, formato o nulos.
Línea 39	<code></band></code> → cierra el elemento o sección abierto correspondiente.
Línea 40	<code></detail></code> → cierra el elemento o sección abierto correspondiente.

Parte C — Código Java explicado línea por línea COMPLETADO

2.4 incorpora recursos gráficos en el JRXML; el modelo y la fuente Java permanecen iguales a 2.3. Se reproduce la versión exacta del checkpoint para mantener trazabilidad.

Los tres archivos siguientes se reproducen **literalmente desde el checkpoint ejecutable M2/2.4**. De este modo, la Parte C coincide con el código que compila y se ejecuta en la validación end-to-end.

Clase Libro.java

JAVA

```
import java.util.ArrayList;
import java.util.Calendar;
import java.util.GregorianCalendar;
import java.util.List;

public class Libro {
    private final String titulo;
    private final Double precio;
    private final Integer paginas;
    private final java.util.Date fechaPublicacion;
    private final Boolean disponible;

    public Libro(String titulo, Double precio, Integer paginas, int anioPublicacion, Boolean disponible) {
        this.titulo = titulo;
        this.precio = precio;
        this.paginas = paginas;
        this.fechaPublicacion = fecha(anioPublicacion);
        this.disponible = disponible;
    }

    private static java.util.Date fecha(int anio) {
        Calendar c = new GregorianCalendar(anio, Calendar.JANUARY, 1);
        c.set(Calendar.HOUR_OF_DAY, 0);
        c.set(Calendar.MINUTE, 0);
        c.set(Calendar.SECOND, 0);
        c.set(Calendar.MILLISECOND, 0);
        return c.getTime();
    }
}
```

```

    }

    public String getTitulo() { return titulo; }
    public Double getPrecio() { return precio; }
    public Integer getPaginas() { return paginas; }
    public java.util.Date getFechaPublicacion() { return fechaPublicacion; }
    public Boolean getDisponible() { return disponible; }

    public static List<Libro> listaEjemplo() {
        List<Libro> libros = new ArrayList<Libro>();
        libros.add(new Libro("Cien años de soledad", 19.95, 471, 1967, Boolean.TRUE));
        libros.add(new Libro("Rayuela", 22.50, 736, 1963, Boolean.TRUE));
        libros.add(new Libro("La ciudad y los perros", 18.75, 432, 1963, Boolean.TRUE));
        libros.add(new Libro("Pedro Páramo", 15.90, 132, 1955, Boolean.TRUE));
        libros.add(new Libro("Ficciones", 21.00, 224, 1944, Boolean.TRUE));
        libros.add(new Libro("La casa de los espíritus", 23.40, 448, 1982, Boolean.TRUE));
        libros.add(new Libro("El amor en los tiempos del cólera", 20.80, 496, 1985, Boolean.TRUE));
        libros.add(new Libro("La muerte de Artemio Cruz", 17.60, 320, 1962, Boolean.TRUE));
        libros.add(new Libro("Doña Bárbara", 16.95, 400, 1929, Boolean.FALSE));
        libros.add(new Libro("Martín Fierro", 14.50, 288, 1872, Boolean.FALSE));
        libros.add(new Libro("Comala", 19.20, 148, 1955, Boolean.TRUE));
        libros.add(new Libro("Paradiso", 25.00, 576, 1966, Boolean.TRUE));
        libros.add(new Libro("La invención de Morel", 18.30, 128, 1940, Boolean.TRUE));
        libros.add(new Libro("El túnel", 16.20, 160, 1948, Boolean.FALSE));
        return libros;
    }
}

```

Explicación línea por línea

Línea 1	<code>import java.util.ArrayList;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 2	<code>import java.util.Calendar;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 3	<code>import java.util.GregorianCalendar;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 4	<code>import java.util.List;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 6	<code>public class Libro {</code> → declara la clase Java y, si aparece <code>implements</code> , establece el contrato que debe implementar.
Línea 7	<code>private final String titulo;</code> → declara un campo de instancia inmutable después de la construcción del objeto.
Línea 8	<code>private final Double precio;</code> → declara un campo de instancia inmutable después de la construcción del objeto.
Línea 9	<code>private final Integer paginas;</code> → declara un campo de instancia inmutable después de la construcción del objeto.
Línea 10	<code>private final java.util.Date fechaPublicacion;</code> → declara un campo de instancia inmutable después de la construcción del objeto.
Línea 11	<code>private final Boolean disponible;</code> → declara un campo de instancia inmutable después de la construcción del objeto.
Línea 13	<code>public Libro(String titulo, Double precio, Integer paginas, int anioPublicacion, Boolean disponible) {</code> → declara el constructor completo del modelo Libro con los valores que necesita el informe.
Línea 14	<code>this.titulo = titulo;</code> → asigna al campo del objeto el valor recibido o calculado.
Línea 15	<code>this.precio = precio;</code> → asigna al campo del objeto el valor recibido o calculado.
Línea 16	<code>this.paginas = paginas;</code> → asigna al campo del objeto el valor recibido o calculado.
Línea 17	<code>this.fechaPublicacion = fecha(anioPublicacion);</code> → asigna al campo del objeto el valor recibido o calculado.
Línea 18	<code>this.disponible = disponible;</code> → asigna al campo del objeto el valor recibido o calculado.
Línea 19	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.

Línea 21	<code>private static java.util.Date fecha(int anio) {</code> → abre el método auxiliar que convierte un año en una fecha Java reproducible.
Línea 22	<code>Calendar c = new GregorianCalendar(anio, Calendar.JANUARY, 1);</code> → crea un calendario situado el 1 de enero del año indicado.
Línea 23	<code>c.set(Calendar.HOUR_OF_DAY, 0);</code> → normaliza un componente horario del calendario para obtener una fecha estable.
Línea 24	<code>c.set(Calendar.MINUTE, 0);</code> → normaliza un componente horario del calendario para obtener una fecha estable.
Línea 25	<code>c.set(Calendar.SECOND, 0);</code> → normaliza un componente horario del calendario para obtener una fecha estable.
Línea 26	<code>c.set(Calendar.MILLISECOND, 0);</code> → normaliza un componente horario del calendario para obtener una fecha estable.
Línea 27	<code>return c.getTime();</code> → devuelve la fecha construida por el calendario.
Línea 28	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.
Línea 30	<code>public String getTitulo() { return titulo; }</code> → devuelve el título del libro.
Línea 31	<code>public Double getPrecio() { return precio; }</code> → devuelve el precio del libro.
Línea 32	<code>public Integer getPaginas() { return paginas; }</code> → devuelve el número de páginas.
Línea 33	<code>public java.util.Date getFechaPublicacion() { return fechaPublicacion; }</code> → devuelve la fecha de publicación.
Línea 34	<code>public Boolean getDisponible() { return disponible; }</code> → devuelve el estado de disponibilidad.
Línea 36	<code>public static List<Libro> listaEjemplo() {</code> → abre el método que construye los datos de ejemplo del curso.
Línea 37	<code>List<Libro> libros = new ArrayList<Libro>();</code> → crea una lista tipada compatible con Java 8 y con la baseline del proyecto.
Línea 38	<code>libros.add(new Libro("Cien años de soledad", 19.95, 471, 1967, Boolean.TRUE));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.
Línea 39	<code>libros.add(new Libro("Rayuela", 22.50, 736, 1963, Boolean.TRUE));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.
Línea 40	<code>libros.add(new Libro("La ciudad y los perros", 18.75, 432, 1963, Boolean.TRUE));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.
Línea 41	<code>libros.add(new Libro("Pedro Páramo", 15.90, 132, 1955, Boolean.TRUE));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.
Línea 42	<code>libros.add(new Libro("Ficciones", 21.00, 224, 1944, Boolean.TRUE));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.
Línea 43	<code>libros.add(new Libro("La casa de los espíritus", 23.40, 448, 1982, Boolean.TRUE));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.
Línea 44	<code>libros.add(new Libro("El amor en los tiempos del cólera", 20.80, 496, 1985, Boolean.TRUE));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.
Línea 45	<code>libros.add(new Libro("La muerte de Artemio Cruz", 17.60, 320, 1962, Boolean.TRUE));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.
Línea 46	<code>libros.add(new Libro("Doña Bárbara", 16.95, 400, 1929, Boolean.FALSE));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.
Línea 47	<code>libros.add(new Libro("Martín Fierro", 14.50, 288, 1872, Boolean.FALSE));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.

Línea 48	<code>libros.add(new Libro("Comala", 19.20, 148, 1955, Boolean.TRUE));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.
Línea 49	<code>libros.add(new Libro("Paradiso", 25.00, 576, 1966, Boolean.TRUE));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.
Línea 50	<code>libros.add(new Libro("La invención de Morel", 18.30, 128, 1940, Boolean.TRUE));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.
Línea 51	<code>libros.add(new Libro("El túnel", 16.20, 160, 1948, Boolean.FALSE));</code> → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.
Línea 52	<code>return libros;</code> → devuelve la lista completa que alimentará la fuente de datos.
Línea 53	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.
Línea 54	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.

Clase CatalogoDataSource.java

JAVA

```
import java.util.List;
import net.sf.jasperreports.engine.JRDataSource;
import net.sf.jasperreports.engine.JRException;
import net.sf.jasperreports.engine.JRField;

public class CatalogoDataSource implements JRDataSource {
    private final List<Libro> libros;
    private int indice = -1;

    public CatalogoDataSource(List<Libro> libros) {
        this.libros = libros;
    }

    @Override
    public boolean next() throws JRException {
        indice++;
        return indice < libros.size();
    }

    @Override
    public Object getFieldValue(JRField campo) throws JRException {
        Libro actual = libros.get(indice);
        if ("titulo".equals(campo.getName())) return actual.getTitulo();
        if ("precio".equals(campo.getName())) return actual.getPrecio();
        if ("paginas".equals(campo.getName())) return actual.getPaginas();
        if ("fechaPublicacion".equals(campo.getName())) return actual.getFechaPublicacion();
        if ("disponible".equals(campo.getName())) return actual.getDisponible();
        throw new JRException("Campo no soportado por CatalogoDataSource: " + campo.getName());
    }
}
```

Explicación línea por línea

Línea 1	<code>import java.util.List;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 2	<code>import net.sf.jasperreports.engine.JRDataSource;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 3	<code>import net.sf.jasperreports.engine.JRException;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 4	<code>import net.sf.jasperreports.engine.JRField;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 6	<code>public class CatalogoDataSource implements JRDataSource {</code> → declara la clase Java y, si aparece <code>implements</code> , establece el contrato que debe implementar.
Línea 7	<code>private final List<Libro> libros;</code> → declara un campo de instancia inmutable después de la construcción del objeto.

Línea 8	<code>private int indice = -1;</code> → declara el índice interno de la fuente de datos; empieza en -1 porque <code>next()</code> se invoca antes de leer el primer registro.
Línea 10	<code>public CatalogoDataSource(List<Libro> libros) {</code> → declara el constructor de la fuente de datos y recibe la lista de libros.
Línea 11	<code>this.libros = libros;</code> → asigna al campo del objeto el valor recibido o calculado.
Línea 12	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.
Línea 14	<code>@Override</code> → indica que el método implementa un método del contrato <code>JRDataSource</code> .
Línea 15	<code>public boolean next() throws JRException {</code> → implementa <code>JRDataSource.next()</code> y declara <code>JRException</code> según el contrato de JasperReports.
Línea 16	<code>indice++;</code> → avanza al siguiente registro.
Línea 17	<code>return indice < libros.size();</code> → indica al motor si todavía existe un registro válido.
Línea 18	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.
Línea 20	<code>@Override</code> → indica que el método implementa un método del contrato <code>JRDataSource</code> .
Línea 21	<code>public Object getFieldValue(JRField campo) throws JRException {</code> → implementa la resolución de un campo JRXML para el registro actual.
Línea 22	<code>Libro actual = libros.get(indice);</code> → obtiene el libro correspondiente al índice actual.
Línea 23	<code>if ("titulo".equals(campo.getName())) return actual.getTitulo();</code> → resuelve el campo titulo.
Línea 24	<code>if ("precio".equals(campo.getName())) return actual.getPrecio();</code> → resuelve el campo precio.
Línea 25	<code>if ("paginas".equals(campo.getName())) return actual.getPaginas();</code> → resuelve el campo paginas.
Línea 26	<code>if ("fechaPublicacion".equals(campo.getName())) return actual.getFechaPublicacion();</code> → resuelve el campo fechaPublicacion.
Línea 27	<code>if ("disponible".equals(campo.getName())) return actual.getDisponible();</code> → resuelve el campo disponible.
Línea 28	<code>throw new JRException("Campo no soportado por CatalogoDataSource: " + campo.getName());</code> → falla explícitamente si el JRXML solicita un campo que la fuente no soporta, evitando devolver silenciosamente un valor incorrecto.
Línea 29	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.
Línea 30	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.

Clase `GeneradorInformeConcepto.java`

JAVA

```
import java.io.File;
import java.util.HashMap;
import java.util.Map;

import net.sf.jasperreports.engine.JasperCompileManager;
import net.sf.jasperreports.engine.JasperExportManager;
import net.sf.jasperreports.engine.JasperFillManager;
import net.sf.jasperreports.engine.JasperPrint;

public class GeneradorInformeConcepto {
    public static void main(String[] args) {
        try {
            String rutaJrxml = "reports/informe_concepto.jrxml";
            String rutaJasper = "reports/informe_concepto.jasper";
            String rutaPdf = "output/informe_concepto.pdf";

            JasperCompileManager.compileReportToFile(rutaJrxml, rutaJasper);

            Map<String, Object> parametros = new HashMap<String, Object>();

            JasperPrint documento = JasperFillManager.fillReport(
                rutaJasper,
                parametros,
```

```

        new CatalogoDataSource(Libro.listaEjemplo());

JasperExportManager.exportReportToPdfFile(documento, rutaPdf);

System.out.println("Informe generado en: " + new File(rutaPdf).getAbsolutePath());
System.out.println("Paginas del documento: " + documento.getPages().size());
System.out.println("Registros de ejemplo: " + Libro.listaEjemplo().size());
} catch (Exception e) {
    e.printStackTrace();
    System.exit(1);
}
}
}

```

Explicación línea por línea

Línea 1	<code>import java.io.File;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 2	<code>import java.util.HashMap;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 3	<code>import java.util.Map;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 5	<code>import net.sf.jasperreports.engine.JasperCompileManager;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 6	<code>import net.sf.jasperreports.engine.JasperExportManager;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 7	<code>import net.sf.jasperreports.engine.JasperFillManager;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 8	<code>import net.sf.jasperreports.engine.JasperPrint;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 10	<code>public class GeneradorInformeConcepto {</code> → declara la clase Java y, si aparece <code>implements</code> , establece el contrato que debe implementar.
Línea 11	<code>public static void main(String[] args) {</code> → declara el punto de entrada de la aplicación.
Línea 12	<code>try {</code> → abre el bloque protegido de ejecución.
Línea 13	<code>String rutaJrxml = "reports/informe_concepto.jrxml";</code> → define la ruta relativa de la plantilla JRXML.
Línea 14	<code>String rutaJasper = "reports/informe_concepto.jasper";</code> → define la ruta del artefacto compilado <code>.jasper</code> .
Línea 15	<code>String rutaPdf = "output/informe_concepto.pdf";</code> → define la ruta del PDF de salida.
Línea 17	<code>JasperCompileManager.compileReportToFile(rutaJrxml, rutaJasper);</code> → compila el JRXML con JasperReports Library.
Línea 19	<code>Map<String, Object> parametros = new HashMap<String, Object>();</code> → crea el mapa tipado de parámetros.
Línea 21	<code>JasperPrint documento = JasperFillManager.fillReport(</code> → declara el <code>JasperPrint</code> resultante del llenado.
Línea 22	<code>rutaJasper,</code> → pasa al llenado el informe compilado.
Línea 23	<code>parametros,</code> → pasa el mapa de parámetros.
Línea 24	<code>new CatalogoDataSource(Libro.listaEjemplo());</code> → pasa la fuente de datos construida con los libros de ejemplo.
Línea 26	<code>JasperExportManager.exportReportToPdfFile(documento, rutaPdf);</code> → exporta el <code>JasperPrint</code> a un PDF real.
Línea 28	<code>System.out.println("Informe generado en: " + new File(rutaPdf).getAbsolutePath());</code> → escribe en consola la ruta absoluta del PDF generado.

Línea 29	<code>System.out.println("Paginas del documento: " + documento.getPages().size());</code> → escribe en consola el número real de páginas del JasperPrint.
Línea 30	<code>System.out.println("Registros de ejemplo: " + Libro.listaEjemplo().size());</code> → escribe en consola el número de registros de ejemplo; el workflow usa esta salida como evidencia de ejecución.
Línea 31	<code>} catch (Exception e) {</code> → captura cualquier fallo de compilación, llenado o exportación.
Línea 32	<code>e.printStackTrace();</code> → imprime la traza del error para diagnóstico.
Línea 33	<code>System.exit(1);</code> → termina con código distinto de cero para que GitHub Actions detecte el fallo.
Línea 34	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.
Línea 35	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.
Línea 36	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.

Traza de consola esperada tras la ejecución

TEXT

Informe generado en: C:\Users\<usuario>\Documents\JasperProjects\EditorialReports\output\informe_concepto.pdf
 Paginas del documento: 2
 Registros de ejemplo: 14

La ruta depende del equipo. Los valores anteriores están comprobados en el run E2E 36011315054: checkpoint 2.4 → 2 páginas y 14 registros.

Parte D — Validación del resultado y estructura del proyecto

D.1 — Vista de diseño en Jaspersoft Studio

TEXT

```
+-----+
| informe_concepto.jrxml                                [Design] [Source] |
+-----+
| Title [height=100]                                     |
| [logo 0,10,80,80] Título [90,25,465,30]              |
| Fecha [90,60,120,20] [215,60,150,20]                 |
|-----|
| Column Header [height=25]                             |
| Port. 0/50 | Título 55/180 | Precio 235/80 | Págs. 315/50 |
| Año 365/50 | Disp. 415/80 | # 500/55                  |
|-----|
| Detail 1 [height=60, splitType=Prevent]               |
| portada 0/50 | título 55/180 | precio 235/80 | páginas 315/50 |
| año 365/50 | disp. 415/55 | icono 475/20 | contador 500/55 |
|-----|
| ancho útil: 0 ----- 555 |
+-----+
```

Qué representa: la geometría real del checkpoint 2.4. Ningún elemento de Column Header o Detail supera `columnWidth="555"`; el icono condicional ocupa 475–495 y el contador 500–555.

Cómo verificarlo: seleccionar los elementos en Design y comprobar X/Width en Properties. Detail 1 debe tener Height=60 y Split Type=Prevent.

D.2 — Jerarquía del Outline

TEXT

```
informe_concepto
├── Fields
│   ├── titulo, precio, paginas, fechaPublicacion, disponible
├── Title [height=100]
│   ├── image [0,10,80,80] "resources/logo.png"
│   ├── staticText [90,25,465,30] título
│   ├── staticText [90,60,120,20] "Fecha de emisión:"
│   └── textField [215,60,150,20] new java.util.Date()
└── Column Header [height=25]
```

```

├── Port. [0,5,50,15]
├── Título [55,5,180,15]
├── Precio [235,5,80,15]
├── Págs. [315,5,50,15]
├── Año [365,5,50,15]
├── Disp. [415,5,80,15]
├── # [500,5,55,15]
├── Detail 1 [height=60, splitType=Prevent]
│   ├── image [0,5,50,50] portada dinámica
│   ├── textField [55,20,180,20] ${título}
│   ├── textField [235,20,80,20] ${precio}
│   ├── textField [315,20,50,20] ${paginas}
│   ├── textField [365,20,50,20] ${fechaPublicacion}
│   ├── textField [415,20,55,20] disponibilidad
│   ├── image [475,20,20,20] icono condicional
│   └── textField [500,20,55,20] ${REPORT_COUNT}
├── Column Footer, Page Footer, Last Page Footer, Summary
└── Background [height=0]

```

Qué representa: el árbol del checkpoint 2.4 con el logotipo de Title, la portada dinámica y el icono condicional. No existe un Static Text # dentro de Detail; el rótulo # pertenece a Column Header.

Cómo verificarlo: expandir Title, Column Header y Detail 1 en Outline y contrastar cada elemento con Properties.

D.3 — Documento PDF resultante, página por página

TEXT

INFORME: informe_concepto.pdf
 PÁGINAS TOTALES: 2
 REGISTROS PROCESADOS: 14

Página 1 de 2

- Title con logotipo, título y fecha.
- Page Header y Column Header.
- Registros 1 a 9 del catálogo con portada opcional, datos e icono de disponibilidad.
- Column Footer: Registros procesados: 9.
- Page Footer estándar.

Página 2 de 2

- Page Header y Column Header repetidos.
- Registros 10 a 14.
- Summary: Total de páginas: 2; Total de libros: 14; Subtotal precios: 270.05 €.
- Column Footer: Registros procesados: 14.
- Last Page Footer: Documento generado en la última página.

Qué representa: el resultado real del checkpoint 2.4 ejecutado en CI con 14 registros. La incorporación de imágenes aumenta la altura de Detail y el informe pasa a dos páginas.

Cómo verificarlo: abrir el PDF de M2-2.4-runtime y comprobar la distribución 9 + 5 registros, el Summary en la segunda página y el Last Page Footer.

D.4 — Árbol de carpetas del proyecto tras completar el punto

TEXT

```

EditorialReports/
├── ECOSISTEMA.md (documentación del ecosistema)
├── ENTORNO.md (documentación del entorno)
├── BANDAS.md (documentación de las bandas)
├── JRXML.md (documentación del formato JRXML)
├── TEXTO.md (documentación de elementos textuales)
├── CAMPOS.md (documentación de campos)
├── IMAGENES.md (documentación de imágenes)
├── reports/
│   ├── informe_concepto.jrxml (plantilla con imágenes)
│   └── informe_concepto.jasper (artefacto compilado)
├── resources/
│   ├── logo.png (logotipo de la editorial)
│   ├── icono_disponible.png (icono de disponible)
│   ├── icono_no_disponible.png (icono de no disponible)
│   └── portadas/
│       ├── Cien años de soledad.png
│       ├── Rayuela.png
│       └── Pedro Páramo.png
└── output/
    └── informe_concepto.pdf (documento con imágenes)

```

```

EditorialReportsJava/
├── lib/
│   └── README.md    (el runtime real se resuelve con Maven)
└── src/
    ├── GeneradorInformeConcepto.java
    ├── Libro.java
    └── CatalogoDataSource.java

```

Qué representa: el estado de los dos proyectos tras completar los doce pasos. La novedad respecto al punto anterior es la carpeta `resources` con el logotipo, los iconos y la subcarpeta portadas, además del archivo `IMAGENES.md`.

Cómo verificarlo: expandir los nodos del panel Project Explorer y comparar con este esquema. Si los archivos de imagen no aparecen, hacer clic con el botón derecho sobre el nodo `EditorialReports` y seleccionar `Refresh`.

Errores comunes del ejercicio completo

Error	Causa	Solución
La imagen no aparece y se muestra un icono roto	La ruta de la imagen es incorrecta o el archivo no existe	Verificar que el archivo existe en la ruta indicada y que la ruta es correcta
Could not load image al previsualizar	La imagen está en una ruta absoluta que no existe en el equipo actual	Usar rutas relativas al proyecto o cargar la imagen desde el classpath
La imagen aparece deformada	El modo de escala es <code>FillFrame</code> o las dimensiones no coinciden con la proporción	Cambiar el modo de escala a <code>RetainShape</code>
La imagen se recorta por los bordes	El modo de escala es <code>Clip</code> y las dimensiones del rectángulo son menores que la imagen	Ampliar el rectángulo o cambiar el modo a <code>RetainShape</code>
El informe falla cuando falta una portada	La propiedad <code>onErrorType</code> está en <code>Error</code>	Cambiar la propiedad a <code>Blank</code> en el panel <code>Properties</code>
La imagen del logotipo aparece estirada	El modo de escala no está configurado	Seleccionar <code>RetainShape</code> en el desplegable <code>Scale Image</code>
La imagen se carga pero aparece en blanco	El archivo PNG tiene fondo transparente y se ha perdido la transparencia	Exportar la imagen con fondo blanco o cambiar el color de fondo de la banda
La carpeta <code>resources</code> no aparece en el panel Project Explorer	El panel no se ha refrescado después de crear la carpeta	Hacer clic con el botón derecho sobre el nodo <code>EditorialReports</code> y seleccionar <code>Refresh</code>
Las imágenes consumen demasiado espacio en el PDF	Las imágenes tienen resolución muy alta	Reducir la resolución de las imágenes originales antes de incluirlas en el proyecto
La expresión de la imagen dinámica no compila	Se usan comillas simples en lugar de comillas dobles	Usar comillas dobles para las cadenas y el operador <code>+</code> para concatenar

Reto resuelto paso a paso

Enunciado: añadir un logotipo secundario en el extremo derecho de `Page Footer`, sin superar el ancho útil de 555. El logotipo será `resources/logo.png`, con tamaño `20 × 20`; antes reduciremos el ancho del texto existente para evitar solapamientos.

Paso 1. Hacer doble clic sobre el archivo `informe_concepto.jrxml` en el panel Project Explorer.

Paso 2. Hacer clic sobre la pestaña `Design` en la parte inferior del editor central.

Paso 3. Hacer clic sobre el nodo `Page Footer` en el panel Outline (inferior izquierdo).

Paso 4. Hacer clic sobre el campo `Band height` en el panel Properties (inferior derecho), pestaña `Properties`, escribir 30 y pulsar `Enter`.

Paso 5. Hacer clic sobre la pestaña `Elements` en el panel Palette (derecha del editor central).

Paso 6. Hacer clic sobre el icono `Image` (un cuadrado con un paisaje).

Paso 7. Seleccionar el `Static Text` existente de `Page Footer` y cambiar `Width` a 525, manteniendo `X=0`, `Y=5` y `Height=20`.

Paso 8. Arrastrar el icono `Image` y soltarlo dentro de `Page Footer` cerca del extremo derecho.

Paso 9. Fijar `X=535` y `Y=5` en `Properties`.

Paso 10. Fijar `Width=20` y `Height=20`; comprobar que `535 + 20 = 555`.

Paso 11. Seleccionar `RetainShape` en `Scale Image`.

Paso 12. Seleccionar `Blank` en `On Error Type`.

Paso 13. Escribir exactamente "resources/logo.png" en Image Expression.

Paso 14. Pulsar Ctrl+S para guardar.

Paso 15. Pulsar Ctrl+Mayús+B para compilar el informe.

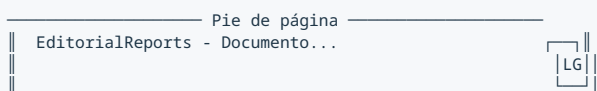
Paso 16. Ejecutar GeneradorInformeConcepto.java como **Java Application** con EditorialReports como Working Directory.

Paso 17. Abrir output/informe_concepto.pdf y verificar que el logotipo aparece en el extremo derecho de Page Footer, sin solaparse con el texto.

Paso 18. Volver a Design y comprobar en Properties que ningún elemento del pie termina más allá de X=555.

Simulación ASCII del PDF tras el reto

TEXT



 Resultado del reto: el logotipo aparece en el extremo derecho de Page Footer, dentro del ancho útil. El texto se reduce a 525 unidades y el icono ocupa X=535..555, por lo que no existe desbordamiento.

Analogía final con el contexto de la editorial

Las imágenes son los elementos visuales del catálogo. El logotipo de la editorial es el sello que aparece en la portada de cada edición. Las portadas de los libros son las cubiertas que acompañan a cada ficha en la tabla del catálogo. Los iconos de disponibilidad son los símbolos que indican de un vistazo si un libro está en stock. Los modos de escala son las decisiones de maquetación que determinan cómo se ajusta cada imagen a su espacio. La propiedad `onErrorType` es la previsión del editor ante la falta de material gráfico: si falta una portada, el espacio queda en blanco y el catálogo se imprime igual. Las imágenes convierten el catálogo en un documento visualmente rico que el lector puede recorrer con la vista además de con la lectura.

Resultado esperado

- Al finalizar este punto, el alumno dispone de:
- La carpeta resources con el logotipo, los iconos y la subcarpeta portadas.
- El archivo reports/informe_concepto.jrxml con tres elementos de imagen: el logotipo en la banda Title, la portada dinámica en la banda Detail y el icono de disponibilidad en la banda Detail.
- El archivo output/informe_concepto.pdf con el logotipo, las portadas que existan y los iconos de disponibilidad.
- El archivo IMAGENES.md en la raíz del proyecto con la documentación de las imágenes.
- Comprensión operativa de los elementos de imagen, de los modos de escala y de la propiedad `onErrorType`.

Conclusión y enlace al siguiente punto

El punto 2.4 ha incorporado recursos gráficos estáticos y dinámicos al mismo informe acumulativo. El alumno ha comprobado que una imagen también se resuelve mediante una expresión y que `onErrorType="Blank"` permite tolerar recursos opcionales sin detener la generación.

El punto 2.5, «Formato y estilos», reorganiza la presentación del catálogo con estilos reutilizables y condicionales para evitar duplicación y mantener una identidad visual coherente.

Punto 2.5 — Formato y estilos

PUNTO DE PARTIDA. Si vienes haciendo el curso, continúa con tu proyecto resultante del punto 2.4. Si te incorporas directamente aquí, usa M2/2.4 como estado inicial. M2/2.5 contiene la solución completa del punto.

Parte A — Práctica visual

Paso 1: Abrir el informe y localizar la sección de estilos VALIDADO

Acciones:

1. Hacer doble clic sobre el archivo `informe_concepto.jrxml` en el panel Project Explorer (superior izquierdo).
2. Hacer clic sobre la pestaña Source en la parte inferior del editor central.
3. Localizar la línea que contiene `<style name="Sans_Normal" isDefault="true" .../>`.
4. Hacer clic al final de esa línea y pulsar Enter.

Verificación visual: el editor central muestra la línea del estilo `Sans_Normal` seguida de una línea vacía.

Qué hace: abre el archivo JRXML en la vista de código fuente y posiciona el cursor después del estilo por defecto.

Por qué: los nuevos estilos deben declararse después del estilo por defecto y antes de las bandas.

Error común: declarar los nuevos estilos después de las bandas. El esquema XSD rechaza la estructura y el editor muestra un subrayado amarillo.

Solución: mover las declaraciones de estilo a la posición correcta.

Analogía: es como preparar la hoja de estilo tipográfico del catálogo antes de empezar a componer las páginas.

Paso 2: Declarar el estilo `TituloPrincipal` VALIDADO

Acciones:

1. En la línea vacía después de `Sans_Normal`, escribir exactamente `<style name="TituloPrincipal" style="Sans_Normal" fontSize="18" isBold="true" forecolor="#1A3D6B"/>` y pulsar Enter.

Verificación visual: el editor central muestra la nueva línea con el estilo `TituloPrincipal` declarado.

Qué hace: declara un estilo para los títulos principales del informe.

Por qué: el estilo agrupa las propiedades tipográficas del título y permite reutilizarlas en varios elementos.

Error común: olvidar el atributo `style`. El estilo no hereda la tipografía del estilo por defecto.

Solución: añadir `style="Sans_Normal"`.

Analogía: es como definir el estilo tipográfico de los títulos principales del catálogo.

Paso 3: Declarar el estilo `TituloSecundario` VALIDADO

Acciones:

1. En la línea después de `TituloPrincipal`, escribir exactamente `<style name="TituloSecundario" style="Sans_Normal" fontSize="14" isBold="true" forecolor="#4A6B8A"/>` y pulsar Enter.

Verificación visual: el editor central muestra la nueva línea con el estilo `TituloSecundario` declarado.

Qué hace: declara un estilo para los títulos secundarios del informe.

Por qué: el estilo permite diferenciar visualmente los títulos secundarios de los principales.

Error común: usar el mismo color que el título principal. Los títulos secundarios deben tener un contraste menor.

Solución: usar un color más claro.

Analogía: es como definir el estilo de los subtítulos del catálogo.

Paso 4: Declarar el estilo `TextoTablaCabecera` VALIDADO

Acciones:

1. En la línea después de `TituloSecundario`, escribir exactamente `<style name="TextoTablaCabecera" style="Sans_Normal" fontSize="9" isBold="true" forecolor="#FFFFFF" backcolor="#4A6B8A" mode="Opaque"/>` y pulsar Enter.

Verificación visual: el editor central muestra la nueva línea con el estilo `TextoTablaCabecera` declarado.

Qué hace: declara un estilo para las cabeceras de la tabla de datos.

Por qué: las cabeceras se destacan visualmente con un fondo de color y texto en blanco.

Error común: olvidar el atributo `mode="Opaque"`. El fondo no se rellena con el color indicado y el texto blanco queda invisible.

Solución: añadir `mode="Opaque"` al estilo.

Analogía: es como definir el estilo de los títulos de las columnas del catálogo con fondo destacado.

Paso 5: Declarar el estilo `TextoTabla` VALIDADO

Acciones:

1. En la línea después de `TextoTablaCabecera`, escribir exactamente `<style name="TextoTabla" style="Sans_Normal" fontSize="9"/>` y pulsar Enter.

Verificación visual: el editor central muestra la nueva línea con el estilo `TextoTabla` declarado.

Qué hace: declara un estilo para las celdas de la tabla de datos.

Por qué: el estilo garantiza la coherencia tipográfica de todas las celdas.

Error común: olvidar el atributo `style`. El estilo no hereda la tipografía.

Solución: añadir `style="Sans_Normal"`.

Analogía: es como definir el estilo del cuerpo de texto de las filas del catálogo.

Paso 6: Declarar el estilo `TextoPrecio` con estilo condicional VALIDADO

Acciones:

1. En la línea después de `TextoTabla`, escribir exactamente `<style name="TextoPrecio" style="TextoTabla">` y pulsar Enter.
2. Escribir exactamente `<conditionalStyle>` y pulsar Enter.

3. Escribir exactamente `<conditionExpression><![CDATA[${F{precio}.doubleValue() > 20.0}]]></conditionExpression>` y pulsar Enter.
4. Escribir exactamente `<style forecolor="#CC0000" isBold="true"/>` y pulsar Enter.
5. Escribir exactamente `</conditionalStyle>` y pulsar Enter.
6. Escribir exactamente `</style>` y pulsar Enter.

Verificación visual: el editor central muestra el bloque completo del estilo TextoPrecio con su estilo condicional.

Qué hace: declara un estilo para los precios que resalta en rojo los precios superiores a 20 euros.

Por qué: el estilo condicional permite cambiar el formato de un elemento según el valor del campo.

Error común: escribir la condición con comillas simples en lugar de dobles o con el operador incorrecto.

Solución: revisar la expresión y asegurarse de que usa `${F{precio}.doubleValue() > 20.0}` con comillas dobles en las cadenas si las hubiera.

Analogía: es como destacar en rojo los precios altos del catálogo para llamar la atención del lector.

Paso 7: Declarar el estilo TextoPequeno VALIDADO

Acciones:

1. En la línea después de `</style>` de TextoPrecio, escribir exactamente `<style name="TextoPequeno" style="Sans_Normal" fontSize="9" isItalic="true" forecolor="#666666"/>` y pulsar Enter.

Verificación visual: el editor central muestra la nueva línea con el estilo TextoPequeno declarado.

Qué hace: declara un estilo para los textos pequeños y secundarios del informe.

Por qué: el estilo permite diferenciar las notas y aclaraciones del cuerpo principal.

Error común: usar un tamaño muy pequeño que resulte ilegible.

Solución: mantener el tamaño en 9 unidades de informe como mínimo.

Analogía: es como definir el estilo de las notas al pie del catálogo.

Paso 8: Guardar y verificar la compilación de los estilos VALIDADO

Acciones:

1. Pulsar Ctrl+S para guardar el archivo.
2. Pulsar Ctrl+Mayús+B para compilar el informe.
3. Hacer clic sobre el panel Problems (inferior) y verificar que no hay errores.
4. Hacer clic sobre la pestaña Design en la parte inferior del editor central.
5. Expandir el nodo Styles en el panel Outline (inferior izquierdo).

Verificación visual: el panel Outline muestra el nodo Styles con los siete estilos declarados: Sans_Normal, TituloPrincipal, TituloSecundario, TextoTablaCabecera, TextoTabla, TextoPrecio y TextoPequeno.

Qué hace: compila el informe y verifica que los estilos se han declarado correctamente.

Por qué: la compilación detecta errores de sintaxis en las declaraciones de estilo antes de aplicar los estilos a los elementos.

Error común: olvidar el cierre `</style>` en uno de los estilos con bloques hijos. El compilador informa `The element type "style" must be terminated.`

Solución: revisar cada bloque y asegurarse de que tiene su cierre.

Analogía: es como revisar la hoja de estilo del catálogo antes de aplicarla a los elementos.

Paso 9: Aplicar el estilo `TituloPrincipal` al título de la banda `Title` VALIDADO

Acciones:

1. Hacer clic sobre el nodo `Title` en el panel Outline (inferior izquierdo).
2. Hacer clic sobre el Static Text que contiene el texto `Catálogo Editorial - Informe Conceptual` en el editor central.
3. Hacer clic sobre la pestaña `Properties` en el panel `Properties` (inferior derecho).
4. Hacer clic sobre el desplegable `Style` y seleccionar `TituloPrincipal`.
5. Hacer clic sobre la pestaña `Source` en la parte inferior del editor central.
6. Localizar el `<reportElement>` del Static Text del título y verificar que contiene el atributo `style="TituloPrincipal"`.

Verificación visual: el título aparece con el tamaño 18, en negrita y con el color azul oscuro del estilo.

Qué hace: aplica el estilo `TituloPrincipal` al título del informe.

Por qué: el título hereda las propiedades del estilo y mantiene la coherencia con el resto del documento.

Error común: olvidar seleccionar el estilo en el desplegable. El título conserva las propiedades anteriores.

Solución: seleccionar `TituloPrincipal` en el desplegable `Style` del panel `Properties`.

Analogía: es como aplicar el estilo tipográfico de los títulos principales al rótulo de la portada.

Paso 10: Aplicar el estilo `TextoTablaCabecera` a los encabezados de la banda `Column Header` VALIDADO

Acciones:

1. Hacer clic sobre el nodo `Column Header` en el panel Outline (inferior izquierdo).
2. Hacer clic sobre el Static Text que contiene el texto `Port .` en el editor central.
3. Hacer clic sobre el desplegable `Style` en el panel `Properties` y seleccionar `TextoTablaCabecera`.
4. Hacer clic sobre el Static Text que contiene el texto `Título`.
5. Hacer clic sobre el desplegable `Style` y seleccionar `TextoTablaCabecera`.
6. Hacer clic sobre el Static Text que contiene el texto `Precio`.
7. Hacer clic sobre el desplegable `Style` y seleccionar `TextoTablaCabecera`.
8. Hacer clic sobre el Static Text que contiene el texto `Págs .`
9. Hacer clic sobre el desplegable `Style` y seleccionar `TextoTablaCabecera`.
10. Hacer clic sobre el Static Text que contiene el texto `Año`.
11. Hacer clic sobre el desplegable `Style` y seleccionar `TextoTablaCabecera`.

Verificación visual: todos los encabezados de la banda `Column Header` aparecen con fondo azul y texto blanco en negrita.

Qué hace: aplica el estilo de cabecera de tabla a todos los encabezados de la banda `Column Header`.

Por qué: la coherencia visual de la cabecera refuerza la legibilidad de la tabla de datos.

Error común: olvidar aplicar el estilo a alguno de los encabezados y provocar que uno de ellos aparezca con el estilo por defecto.

Solución: revisar todos los encabezados y aplicar el estilo a cada uno.

Analogía: es como aplicar el mismo estilo a todos los títulos de columna de la tabla del catálogo.

Paso 11: Aplicar los estilos de tabla a los campos de Detail

VALIDADO

Acciones:

1. Seleccionar `$F{titulo}` y elegir `TextoTabla` en `Style`.
2. Seleccionar `$F{paginas}` y elegir `TextoTabla`.
3. Seleccionar `$F{fechaPublicacion}` y elegir `TextoTabla`.
4. Seleccionar el campo de disponibilidad y elegir `TextoTabla`.
5. Seleccionar `$V{REPORT_COUNT}` y elegir `TextoPequeno`.
6. Comprobar en `Source` que cada `reportElement` contiene el atributo `style` esperado.

Verificación visual: los campos de datos comparten la misma tipografía y el contador utiliza el estilo de texto pequeño.

Qué hace: aplica estilos reutilizables a los elementos reales presentes en el checkpoint 2.5.

Por qué: separar la presentación en estilos evita repetir propiedades tipográficas en cada elemento y facilita cambios globales. El rótulo `#` permanece únicamente en `Column Header`; `Detail` contiene el valor de `$V{REPORT_COUNT}`.

Error común: intentar aplicar un estilo a un elemento distinto del seleccionado o dejar un campo con propiedades locales que contradicen al estilo. **Solución:** comprobar el atributo `style` del `reportElement` en `Source` y eliminar sobrescrituras innecesarias.

Analogía: es como aplicar a todas las celdas de una tabla la misma plantilla tipográfica en lugar de formatearlas una por una.

Paso 12: Aplicar TextoPrecio, ejecutar y documentar los estilos

VALIDADO

Acciones:

1. Hacer clic sobre el nodo `Detail 1` en el panel `Outline`.
2. Seleccionar el `Text Field` cuya expresión es `$F{precio}`.
3. En `Properties > Style`, seleccionar `TextoPrecio`.
4. Abrir `Source` y verificar que su `reportElement` contiene `style="TextoPrecio"`.
5. Pulsar `Ctrl+S` y después `Ctrl+Mayús+B`; comprobar que `Problems` no muestra errores.
6. Ejecutar `GeneradorInformeConcepto.java` mediante `Run As > Java Application`.
7. Abrir `output/informe_concepto.pdf` y comprobar que los precios superiores a 20 euros aparecen en rojo y negrita.
8. Hacer clic con el botón derecho sobre `EditorialReports > New > File`.
9. Escribir `ESTILOS.md` y pulsar `Finish`.
10. Documentar en una tabla `Markdown` los estilos `Sans_Normal`, `TituloPrincipal`, `TituloSecundario`, `TextoTablaCabecera`, `TextoTabla`, `TextoPrecio` y `TextoPequeno`, indicando su estilo padre y uso.
11. Verificar que `TextoPrecio` hereda de `TextoTabla` y que los demás estilos reutilizables heredan de `Sans_Normal`.
12. Guardar `ESTILOS.md` con `Ctrl+S` y refrescar `Project Explorer` si el archivo no aparece inmediatamente.

Verificación visual: el PDF presenta los precios superiores a 20 euros en rojo y negrita, y `Project Explorer` muestra `ESTILOS.md` en la raíz de `EditorialReports`.

Qué hace: aplica el estilo condicional al precio, valida el resultado real y deja documentada la jerarquía de estilos del informe.

Por qué: cierra el punto demostrando que los estilos no son solo declaraciones `JRXML`, sino reglas reutilizables que afectan al PDF generado y quedan mantenibles para el equipo.

Error común: documentar un estilo padre distinto del usado realmente en el `JRXML`. **Solución:** contrastar `ESTILOS.md` con los atributos `style="..."` de las declaraciones antes de guardar.

Analogía: es como aplicar la hoja de estilo definitiva al catálogo, imprimir una prueba y archivar la guía tipográfica para futuras ediciones.

Parte B — JRXML explicado y contrastado

COMPLETADO

Se reproduce únicamente la sección de estilos del `JRXML` y las bandas modificadas. Las secciones modificadas son la declaración de estilos, la banda `title`, la banda `columnHeader` y la banda `detail`.

XML

```

<property name="com.jaspersoft.studio.data.defaultdataadapter" value="EmptyDataSource"/>
<style name="Sans_Normal" isDefault="true" fontName="DejaVu Sans" fontSize="10"/>
<style name="TituloPrincipal" style="Sans_Normal" fontSize="18" isBold="true" forecolor="#1A3D6B"/>
<style name="TituloSecundario" style="Sans_Normal" fontSize="14" isBold="true" forecolor="#4A6B8A"/>
<style name="TextoTablaCabecera" style="Sans_Normal" fontSize="9" isBold="true" forecolor="FFFFFF" backcolor="#4A6B8A"
mode="Opaque"/>
<style name="TextoTabla" style="Sans_Normal" fontSize="9"/>
<style name="TextoPrecio" style="TextoTabla">
  <conditionalStyle>
    <conditionExpression><![CDATA[{$F{precio}.doubleValue() > 20.0}]]></conditionExpression>
    <style forecolor="#CC0000" isBold="true"/>
  </conditionalStyle>
</style>
<style name="TextoPequeno" style="Sans_Normal" fontSize="9" isItalic="true" forecolor="#666666"/>
<title>
  <band height="100">
    <image scaleImage="RetainShape" onErrorType="Error">
      <reportElement x="0" y="10" width="80" height="80" uuid="ddddddd-dddd-dddd-ddddddd"/>
      <imageExpression><![CDATA["resources/logo.png"]]></imageExpression>
    </image>
    <staticText>
      <reportElement x="90" y="25" width="465" height="30" uuid="eeeeeee-eeee-eeee-eeee-eeeeeeeeee" style="TituloPrincipal"/>
    </staticText>
    <staticText>
      <reportElement x="90" y="60" width="120" height="20" uuid="ffffff-ffff-ffff-ffff-ffffff"/>
      <textElement verticalAlignment="Middle">
        <font fontName="DejaVu Sans" size="10"/>
      </textElement>
      <text><![CDATA[Fecha de emisión:]]></text>
    </staticText>
    <textField pattern="dd/MM/yyyy">
      <reportElement x="215" y="60" width="150" height="20" uuid="12121212-1212-1212-1212-121212121212" style="TextoTabla"/>
      <textElement verticalAlignment="Middle">
        <font fontName="DejaVu Sans" size="10"/>
      </textElement>
      <textFieldExpression><![CDATA[new java.util.Date()]]></textFieldExpression>
    </textField>
  </band>
</title>
<columnHeader>
  <band height="25">
    <staticText><reportElement x="0" y="5" width="50" height="15" uuid="13131313-1313-1313-1313-131313131313" style="TextoTablaCa
becera"/><textElement textAlignment="Center"/><![CDATA[Port.]]></staticText>
    <staticText><reportElement x="55" y="5" width="180" height="15" uuid="14141414-1414-1414-1414-141414141414" style="TextoTabla
Cabecera"/><text><![CDATA[Título]]></text></staticText>
    <staticText><reportElement x="235" y="5" width="80" height="15" uuid="15151515-1515-1515-1515-151515151515" style="TextoTabla
Cabecera"/><textElement textAlignment="Right"/><![CDATA[Precio]]></staticText>
    <staticText><reportElement x="315" y="5" width="50" height="15" uuid="16161616-1616-1616-1616-161616161616" style="TextoTabla
Cabecera"/><textElement textAlignment="Right"/><![CDATA[Págs.]]></staticText>
    <staticText><reportElement x="365" y="5" width="50" height="15" uuid="17171717-1717-1717-1717-171717171717" style="TextoTabla
Cabecera"/><textElement textAlignment="Center"/><![CDATA[Año]]></staticText>
    <staticText><reportElement x="415" y="5" width="80" height="15" uuid="18181818-1818-1818-1818-181818181818" style="TextoTabla
Cabecera"/><textElement textAlignment="Center"/><![CDATA[Disp.]]></staticText>
    <staticText><reportElement x="500" y="5" width="55" height="15" uuid="19191919-1919-1919-1919-191919191919" style="TextoTabla
Cabecera"/><textElement textAlignment="Right"/><![CDATA[#]]></staticText>
  </band>
</columnHeader>
<detail>
  <band height="60" splitType="Prevent">
    <image onErrorType="Blank" scaleImage="RetainShape">
      <reportElement x="0" y="5" width="50" height="50" uuid="20202020-2020-2020-2020-202020202020"/>
      <imageExpression><![CDATA["resources/portadas/" + $F{titulo} + ".png"]]></imageExpression>
    </image>
    <textField textAdjust="StretchHeight">
      <reportElement x="55" y="20" width="180" height="20" uuid="21212121-2121-2121-2121-212121212121" style="TextoTabla"/>
      <textElement verticalAlignment="Middle">
        <textFieldExpression><![CDATA[{$F{titulo}]]></textFieldExpression>
      </textField>
    <textField pattern="#,##0.00 €" isBlankWhenNull="true">
      <reportElement x="235" y="20" width="80" height="20" uuid="22222222-3333-4444-5555-666666666666" style="TextoPrecio"/>
      <textElement verticalAlignment="Middle">
        <textFieldExpression><![CDATA[{$F{precio}]]></textFieldExpression>
      </textField>
    <textField isBlankWhenNull="true">
      <reportElement x="315" y="20" width="50" height="20" uuid="23232323-2323-2323-2323-232323232323" style="TextoTabla"/>
      <textElement textAlignment="Right" verticalAlignment="Middle">
        <textFieldExpression><![CDATA[{$F{paginas}]]></textFieldExpression>
      </textField>
    <textField pattern="yyyy" isBlankWhenNull="true">
      <reportElement x="365" y="20" width="50" height="20" uuid="24242424-2424-2424-2424-242424242424" style="TextoTabla"/>
      <textElement textAlignment="Center" verticalAlignment="Middle">
        <textFieldExpression><![CDATA[{$F{fechaPublicacion}]]></textFieldExpression>
      </textField>
    <textField>
      <reportElement x="415" y="20" width="55" height="20" uuid="25252525-2525-2525-2525-252525252525" style="TextoTabla"/>

```

```

        <textFieldExpression><![CDATA[{$F{disponible}.booleanValue() ? "Sí" : "No"}]></textFieldExpression>
    </textField>
    <image onErrorType="Blank" scaleImage="RetainShape">
        <reportElement x="475" y="20" width="20" height="20" uuid="26262626-2626-2626-2626-262626262626"/>
        <imageExpression><![CDATA[{$F{disponible}.booleanValue() ? "resources/icono_disponible.png" : "resources/
icono_no_disponible.png"}]></imageExpression>
    </image>
    <textField>
        <reportElement x="500" y="20" width="55" height="20" uuid="27272727-2727-2727-2727-272727272727" style="TextoPequeno"/>
        <textFieldExpression><![CDATA[{$V{REPORT_COUNT}}]></textFieldExpression>
    </textField>
</band>
</detail>

```

Línea 1	<code><property name="com.jaspersoft.studio.data.defaultdataadapter" value="EmptyDataSource"/></code> → propiedad de Jaspersoft Studio que asocia el adaptador <code>EmptyDataSource</code> al informe.
Línea 2	<code><style name="Sans_Normal" isDefault="true" fontName="DejaVu Sans" fontSize="10" .../></code> → estilo por defecto del informe. Hereda la tipografía <code>DejaVu Sans</code> y el tamaño 10.
Línea 3	<code><style name="TituloPrincipal" style="Sans_Normal" fontSize="18" isBold="true" forecolor="#1A3D6B"/></code> → estilo para los títulos principales. Hereda la tipografía de <code>Sans_Normal</code> y sobrescribe el tamaño, la negrita y el color.
Línea 4	<code><style name="TituloSecundario" style="Sans_Normal" fontSize="14" isBold="true" forecolor="#4A6B8A"/></code> → estilo para los títulos secundarios. Hereda la tipografía de <code>Sans_Normal</code> y sobrescribe el tamaño, la negrita y el color.
Línea 5	<code><style name="TextoTablaCabecera" style="Sans_Normal" fontSize="9" isBold="true" forecolor="#FFFFFF" backcolor="#4A6B8A" mode="Opaque"/></code> → estilo para las cabeceras de tabla. Hereda la tipografía de <code>Sans_Normal</code> y sobrescribe el color del texto (blanco), el color de fondo (azul) y el modo opaco.
Línea 6	<code><style name="TextoTabla" style="Sans_Normal" fontSize="9"/></code> → estilo para las celdas de tabla. Hereda la tipografía de <code>Sans_Normal</code> y sobrescribe el tamaño con 9.
Línea 7	<code><style name="TextoPrecio" style="TextoTabla"></code> → estilo para los precios. Hereda las propiedades de <code>TextoTabla</code> y añade un estilo condicional.
Línea 8	<code><conditionalStyle></code> → abre el bloque de estilo condicional.
Línea 9	<code><conditionExpression><![CDATA[{\$F{precio}.doubleValue() > 20.0}]]></conditionExpression></code> → condición que se evalúa en cada emisión. Si el precio es superior a 20, se aplica el estilo condicional.
Línea 10	<code><style forecolor="#CC0000" isBold="true"/></code> → propiedades que se aplican cuando la condición es verdadera: color rojo y negrita.
Línea 11	<code></conditionalStyle></code> → cierra el bloque condicional.
Línea 12	<code></style></code> → cierra la declaración del estilo.
Línea 13	<code><style name="TextoPequeno" style="Sans_Normal" fontSize="9" isItalic="true" forecolor="#666666"/></code> → estilo para textos pequeños. Hereda la tipografía de <code>Sans_Normal</code> y sobrescribe el tamaño (9), la cursiva y el color gris.

Línea Línea 14-35: banda `title` con el logotipo, el título principal con el estilo `TituloPrincipal` y el rótulo de fecha con el campo de fecha.

Línea 23	<code><reportElement x="90" y="25" width="465" height="30" uuid="..." style="TituloPrincipal"/></code> → el título principal referencia el estilo <code>TituloPrincipal</code> . El elemento hereda la tipografía, el tamaño 18, la negrita y el color azul oscuro.
Línea 36-70	banda <code>columnHeader</code> con los siete encabezados <code>Port.</code> , <code>Título</code> , <code>Precio</code> , <code>Págs.</code> , <code>Año</code> , <code>Disp.</code> y <code>#</code> . Todos referencia el estilo <code>TextoTablaCabecera</code> .
Línea 38	<code><reportElement x="0" y="5" width="50" height="15" uuid="..." style="TextoTablaCabecera"/></code> → el encabezado <code>Port.</code> referencia el estilo de cabecera. El elemento hereda el fondo azul, el texto blanco y la negrita.
Línea 71-136	banda <code>detail</code> con la portada dinámica, los campos de texto y el icono de disponibilidad.

Línea 78	<code><reportElement x="55" y="20" width="180" height="20" uuid="..." style="TextoTabla"/></code> → el campo del título referencia el estilo <code>TextoTabla</code> . Hereda la tipografía y el tamaño del estilo.
Línea 83	<code><reportElement x="235" y="20" width="80" height="20" uuid="..." style="TextoPrecio"/></code> → el campo del precio referencia el estilo <code>TextoPrecio</code> . Hereda la tipografía de <code>TextoTabla</code> y el estilo condicional que resalta en rojo los precios superiores a 20.
Línea 108	<code><reportElement x="500" y="20" width="55" height="20" uuid="..." style="TextoPequeno"/></code> → el campo del número de registro referencia el mismo estilo.

Parte C — Código Java explicado línea por línea COMPLETADO

2.5 cambia estilos del JRXML; Java permanece igual que en 2.4. Se reproduce literalmente el checkpoint ejecutable para evitar divergencias documentales.

Los tres archivos siguientes se reproducen **literalmente desde el checkpoint ejecutable M2/2.5**. De este modo, la Parte C coincide con el código que compila y se ejecuta en la validación end-to-end.

Clase Libro.java

JAVA

```
import java.util.ArrayList;
import java.util.Calendar;
import java.util.GregorianCalendar;
import java.util.List;

public class Libro {
    private final String titulo;
    private final Double precio;
    private final Integer paginas;
    private final java.util.Date fechaPublicacion;
    private final Boolean disponible;

    public Libro(String titulo, Double precio, Integer paginas, int anioPublicacion, Boolean disponible) {
        this.titulo = titulo;
        this.precio = precio;
        this.paginas = paginas;
        this.fechaPublicacion = fecha(anioPublicacion);
        this.disponible = disponible;
    }

    private static java.util.Date fecha(int anio) {
        Calendar c = new GregorianCalendar(anio, Calendar.JANUARY, 1);
        c.set(Calendar.HOUR_OF_DAY, 0);
        c.set(Calendar.MINUTE, 0);
        c.set(Calendar.SECOND, 0);
        c.set(Calendar.MILLISECOND, 0);
        return c.getTime();
    }

    public String getTitulo() { return titulo; }
    public Double getPrecio() { return precio; }
    public Integer getPaginas() { return paginas; }
    public java.util.Date getFechaPublicacion() { return fechaPublicacion; }
    public Boolean getDisponible() { return disponible; }

    public static List<Libro> listaEjemplo() {
        List<Libro> libros = new ArrayList<Libro>();
        libros.add(new Libro("Cien años de soledad", 19.95, 471, 1967, Boolean.TRUE));
        libros.add(new Libro("Rayuela", 22.50, 736, 1963, Boolean.TRUE));
        libros.add(new Libro("La ciudad y los perros", 18.75, 432, 1963, Boolean.TRUE));
        libros.add(new Libro("Pedro Páramo", 15.90, 132, 1955, Boolean.TRUE));
        libros.add(new Libro("Ficciones", 21.00, 224, 1944, Boolean.TRUE));
        libros.add(new Libro("La casa de los espíritus", 23.40, 448, 1982, Boolean.TRUE));
        libros.add(new Libro("El amor en los tiempos del cólera", 20.80, 496, 1985, Boolean.TRUE));
        libros.add(new Libro("La muerte de Artemio Cruz", 17.60, 320, 1962, Boolean.TRUE));
        libros.add(new Libro("Doña Bárbara", 16.95, 400, 1929, Boolean.FALSE));
        libros.add(new Libro("Martín Fierro", 14.50, 288, 1872, Boolean.FALSE));
        libros.add(new Libro("Comala", 19.20, 148, 1955, Boolean.TRUE));
        libros.add(new Libro("Paradiso", 25.00, 576, 1966, Boolean.TRUE));
        libros.add(new Libro("La invención de Morel", 18.30, 128, 1940, Boolean.TRUE));
        libros.add(new Libro("El túnel", 16.20, 160, 1948, Boolean.FALSE));
        return libros;
    }
}
```

Explicación línea por línea

Línea 1	<code>import java.util.ArrayList;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 2	<code>import java.util.Calendar;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 3	<code>import java.util.GregorianCalendar;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 4	<code>import java.util.List;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 6	<code>public class Libro {</code> → declara la clase Java y, si aparece <code>implements</code> , establece el contrato que debe implementar.
Línea 7	<code>private final String titulo;</code> → declara un campo de instancia inmutable después de la construcción del objeto.
Línea 8	<code>private final Double precio;</code> → declara un campo de instancia inmutable después de la construcción del objeto.
Línea 9	<code>private final Integer paginas;</code> → declara un campo de instancia inmutable después de la construcción del objeto.
Línea 10	<code>private final java.util.Date fechaPublicacion;</code> → declara un campo de instancia inmutable después de la construcción del objeto.
Línea 11	<code>private final Boolean disponible;</code> → declara un campo de instancia inmutable después de la construcción del objeto.
Línea 13	<code>public Libro(String titulo, Double precio, Integer paginas, int anioPublicacion, Boolean disponible) {</code> → declara el constructor completo del modelo Libro con los valores que necesita el informe.
Línea 14	<code>this.titulo = titulo;</code> → asigna al campo del objeto el valor recibido o calculado.
Línea 15	<code>this.precio = precio;</code> → asigna al campo del objeto el valor recibido o calculado.
Línea 16	<code>this.paginas = paginas;</code> → asigna al campo del objeto el valor recibido o calculado.
Línea 17	<code>this.fechaPublicacion = fecha(anioPublicacion);</code> → asigna al campo del objeto el valor recibido o calculado.
Línea 18	<code>this.disponible = disponible;</code> → asigna al campo del objeto el valor recibido o calculado.
Línea 19	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.
Línea 21	<code>private static java.util.Date fecha(int anio) {</code> → abre el método auxiliar que convierte un año en una fecha Java reproducible.
Línea 22	<code>Calendar c = new GregorianCalendar(anio, Calendar.JANUARY, 1);</code> → crea un calendario situado el 1 de enero del año indicado.
Línea 23	<code>c.set(Calendar.HOUR_OF_DAY, 0);</code> → normaliza un componente horario del calendario para obtener una fecha estable.
Línea 24	<code>c.set(Calendar.MINUTE, 0);</code> → normaliza un componente horario del calendario para obtener una fecha estable.
Línea 25	<code>c.set(Calendar.SECOND, 0);</code> → normaliza un componente horario del calendario para obtener una fecha estable.
Línea 26	<code>c.set(Calendar.MILLISECOND, 0);</code> → normaliza un componente horario del calendario para obtener una fecha estable.
Línea 27	<code>return c.getTime();</code> → devuelve la fecha construida por el calendario.
Línea 28	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.

Línea 30	<code>public String getTitulo() { return titulo; } → devuelve el título del libro.</code>
Línea 31	<code>public Double getPrecio() { return precio; } → devuelve el precio del libro.</code>
Línea 32	<code>public Integer getPaginas() { return paginas; } → devuelve el número de páginas.</code>
Línea 33	<code>public java.util.Date getFechaPublicacion() { return fechaPublicacion; } → devuelve la fecha de publicación.</code>
Línea 34	<code>public Boolean getDisponible() { return disponible; } → devuelve el estado de disponibilidad.</code>
Línea 36	<code>public static List<Libro> listaEjemplo() { → abre el método que construye los datos de ejemplo del curso.</code>
Línea 37	<code>List<Libro> libros = new ArrayList<Libro>(); → crea una lista tipada compatible con Java 8 y con la baseline del proyecto.</code>
Línea 38	<code>libros.add(new Libro("Cien años de soledad", 19.95, 471, 1967, Boolean.TRUE)); → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.</code>
Línea 39	<code>libros.add(new Libro("Rayuela", 22.50, 736, 1963, Boolean.TRUE)); → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.</code>
Línea 40	<code>libros.add(new Libro("La ciudad y los perros", 18.75, 432, 1963, Boolean.TRUE)); → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.</code>
Línea 41	<code>libros.add(new Libro("Pedro Páramo", 15.90, 132, 1955, Boolean.TRUE)); → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.</code>
Línea 42	<code>libros.add(new Libro("Ficciones", 21.00, 224, 1944, Boolean.TRUE)); → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.</code>
Línea 43	<code>libros.add(new Libro("La casa de los espíritus", 23.40, 448, 1982, Boolean.TRUE)); → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.</code>
Línea 44	<code>libros.add(new Libro("El amor en los tiempos del cólera", 20.80, 496, 1985, Boolean.TRUE)); → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.</code>
Línea 45	<code>libros.add(new Libro("La muerte de Artemio Cruz", 17.60, 320, 1962, Boolean.TRUE)); → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.</code>
Línea 46	<code>libros.add(new Libro("Doña Bárbara", 16.95, 400, 1929, Boolean.FALSE)); → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.</code>
Línea 47	<code>libros.add(new Libro("Martín Fierro", 14.50, 288, 1872, Boolean.FALSE)); → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.</code>
Línea 48	<code>libros.add(new Libro("Comala", 19.20, 148, 1955, Boolean.TRUE)); → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.</code>
Línea 49	<code>libros.add(new Libro("Paradiso", 25.00, 576, 1966, Boolean.TRUE)); → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.</code>
Línea 50	<code>libros.add(new Libro("La invención de Morel", 18.30, 128, 1940, Boolean.TRUE)); → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.</code>
Línea 51	<code>libros.add(new Libro("El túnel", 16.20, 160, 1948, Boolean.FALSE)); → añade a la lista un libro con valores concretos de título, precio, páginas, año y disponibilidad.</code>
Línea 52	<code>return libros; → devuelve la lista completa que alimentará la fuente de datos.</code>
Línea 53	<code>} → abre o cierra el bloque sintáctico correspondiente.</code>
Línea 54	<code>} → abre o cierra el bloque sintáctico correspondiente.</code>

Clase CatalogoDataSource.java

JAVA

```
import java.util.List;
import net.sf.jasperreports.engine.JRDataSource;
import net.sf.jasperreports.engine.JRException;
import net.sf.jasperreports.engine.JRField;

public class CatalogoDataSource implements JRDataSource {
    private final List<Libro> libros;
    private int indice = -1;

    public CatalogoDataSource(List<Libro> libros) {
        this.libros = libros;
    }

    @Override
    public boolean next() throws JRException {
        indice++;
        return indice < libros.size();
    }

    @Override
    public Object getFieldValue(JRField campo) throws JRException {
        Libro actual = libros.get(indice);
        if ("titulo".equals(campo.getName())) return actual.getTitulo();
        if ("precio".equals(campo.getName())) return actual.getPrecio();
        if ("paginas".equals(campo.getName())) return actual.getPaginas();
        if ("fechaPublicacion".equals(campo.getName())) return actual.getFechaPublicacion();
        if ("disponible".equals(campo.getName())) return actual.getDisponible();
        throw new JRException("Campo no soportado por CatalogoDataSource: " + campo.getName());
    }
}
```

Explicación línea por línea

Línea 1	<code>import java.util.List;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 2	<code>import net.sf.jasperreports.engine.JRDataSource;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 3	<code>import net.sf.jasperreports.engine.JRException;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 4	<code>import net.sf.jasperreports.engine.JRField;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 6	<code>public class CatalogoDataSource implements JRDataSource {</code> → declara la clase Java y, si aparece <code>implements</code> , establece el contrato que debe implementar.
Línea 7	<code>private final List<Libro> libros;</code> → declara un campo de instancia inmutable después de la construcción del objeto.
Línea 8	<code>private int indice = -1;</code> → declara el índice interno de la fuente de datos; empieza en -1 porque <code>next()</code> se invoca antes de leer el primer registro.
Línea 10	<code>public CatalogoDataSource(List<Libro> libros) {</code> → declara el constructor de la fuente de datos y recibe la lista de libros.
Línea 11	<code>this.libros = libros;</code> → asigna al campo del objeto el valor recibido o calculado.
Línea 12	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.
Línea 14	<code>@Override</code> → indica que el método implementa un método del contrato <code>JRDataSource</code> .
Línea 15	<code>public boolean next() throws JRException {</code> → implementa <code>JRDataSource.next()</code> y declara <code>JRException</code> según el contrato de JasperReports.
Línea 16	<code>indice++;</code> → avanza al siguiente registro.
Línea 17	<code>return indice < libros.size();</code> → indica al motor si todavía existe un registro válido.
Línea 18	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.
Línea 20	<code>@Override</code> → indica que el método implementa un método del contrato <code>JRDataSource</code> .

Línea 21	<code>public Object getFieldValue(JRField campo) throws JRException {</code> → implementa la resolución de un campo JRXML para el registro actual.
Línea 22	<code>Libro actual = libros.get(indice);</code> → obtiene el libro correspondiente al índice actual.
Línea 23	<code>if ("titulo".equals(campo.getName())) return actual.getTitulo();</code> → resuelve el campo titulo.
Línea 24	<code>if ("precio".equals(campo.getName())) return actual.getPrecio();</code> → resuelve el campo precio.
Línea 25	<code>if ("paginas".equals(campo.getName())) return actual.getPaginas();</code> → resuelve el campo paginas.
Línea 26	<code>if ("fechaPublicacion".equals(campo.getName())) return actual.getFechaPublicacion();</code> → resuelve el campo fechaPublicacion.
Línea 27	<code>if ("disponible".equals(campo.getName())) return actual.getDisponible();</code> → resuelve el campo disponible.
Línea 28	<code>throw new JRException("Campo no soportado por CatalogoDataSource: " + campo.getName());</code> → falla explícitamente si el JRXML solicita un campo que la fuente no soporta, evitando devolver silenciosamente un valor incorrecto.
Línea 29	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.
Línea 30	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.

Clase GeneradorInformeConcepto.java

JAVA

```
import java.io.File;
import java.util.HashMap;
import java.util.Map;

import net.sf.jasperreports.engine.JasperCompileManager;
import net.sf.jasperreports.engine.JasperExportManager;
import net.sf.jasperreports.engine.JasperFillManager;
import net.sf.jasperreports.engine.JasperPrint;

public class GeneradorInformeConcepto {
    public static void main(String[] args) {
        try {
            String rutaJrxml = "reports/informe_concepto.jrxml";
            String rutaJasper = "reports/informe_concepto.jasper";
            String rutaPdf = "output/informe_concepto.pdf";

            JasperCompileManager.compileReportToFile(rutaJrxml, rutaJasper);

            Map<String, Object> parametros = new HashMap<String, Object>();

            JasperPrint documento = JasperFillManager.fillReport(
                rutaJasper,
                parametros,
                new CatalogoDataSource(Libro.listaEjemplo()));

            JasperExportManager.exportReportToPdfFile(documento, rutaPdf);

            System.out.println("Informe generado en: " + new File(rutaPdf).getAbsolutePath());
            System.out.println("Paginas del documento: " + documento.getPages().size());
            System.out.println("Registros de ejemplo: " + Libro.listaEjemplo().size());
        } catch (Exception e) {
            e.printStackTrace();
            System.exit(1);
        }
    }
}
```

Explicación línea por línea

Línea 1	<code>import java.io.File;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 2	<code>import java.util.HashMap;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.

Línea 3	<code>import java.util.Map;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 5	<code>import net.sf.jasperreports.engine.JasperCompileManager;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 6	<code>import net.sf.jasperreports.engine.JasperExportManager;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 7	<code>import net.sf.jasperreports.engine.JasperFillManager;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 8	<code>import net.sf.jasperreports.engine.JasperPrint;</code> → importa la clase o interfaz indicada para que el código pueda referenciarla por su nombre simple.
Línea 10	<code>public class GeneradorInformeConcepto {</code> → declara la clase Java y, si aparece <code>implements</code> , establece el contrato que debe implementar.
Línea 11	<code>public static void main(String[] args) {</code> → declara el punto de entrada de la aplicación.
Línea 12	<code>try {</code> → abre el bloque protegido de ejecución.
Línea 13	<code>String rutaJrxml = "reports/informe_concepto.jrxml";</code> → define la ruta relativa de la plantilla JRXML.
Línea 14	<code>String rutaJasper = "reports/informe_concepto.jasper";</code> → define la ruta del artefacto compilado <code>.jasper</code> .
Línea 15	<code>String rutaPdf = "output/informe_concepto.pdf";</code> → define la ruta del PDF de salida.
Línea 17	<code>JasperCompileManager.compileReportToFile(rutaJrxml, rutaJasper);</code> → compila el JRXML con JasperReports Library.
Línea 19	<code>Map<String, Object> parametros = new HashMap<String, Object>();</code> → crea el mapa tipado de parámetros.
Línea 21	<code>JasperPrint documento = JasperFillManager.fillReport(</code> → declara el <code>JasperPrint</code> resultante del llenado.
Línea 22	<code>rutaJasper,</code> → pasa al llenado el informe compilado.
Línea 23	<code>parametros,</code> → pasa el mapa de parámetros.
Línea 24	<code>new CatalogoDataSource(Libro.listaEjemplo());</code> → pasa la fuente de datos construida con los libros de ejemplo.
Línea 26	<code>JasperExportManager.exportReportToPdfFile(documento, rutaPdf);</code> → exporta el <code>JasperPrint</code> a un PDF real.
Línea 28	<code>System.out.println("Informe generado en: " + new File(rutaPdf).getAbsolutePath());</code> → escribe en consola la ruta absoluta del PDF generado.
Línea 29	<code>System.out.println("Paginas del documento: " + documento.getPages().size());</code> → escribe en consola el número real de páginas del <code>JasperPrint</code> .
Línea 30	<code>System.out.println("Registros de ejemplo: " + Libro.listaEjemplo().size());</code> → escribe en consola el número de registros de ejemplo; el workflow usa esta salida como evidencia de ejecución.
Línea 31	<code>} catch (Exception e) {</code> → captura cualquier fallo de compilación, llenado o exportación.
Línea 32	<code>e.printStackTrace();</code> → imprime la traza del error para diagnóstico.
Línea 33	<code>System.exit(1);</code> → termina con código distinto de cero para que GitHub Actions detecte el fallo.
Línea 34	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.
Línea 35	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.
Línea 36	<code>}</code> → abre o cierra el bloque sintáctico correspondiente.

Traza de consola esperada tras la ejecución

TEXT

```
Informe generado en: C:\Users\<usuario>\Documents\JasperProjects\EditorialReports\output\informe_concepto.pdf
Paginas del documento: 2
Registros de ejemplo: 14
```

La ruta depende del equipo. Los valores anteriores están comprobados en el run E2E 36011315054: checkpoint 2.5 → 2 páginas y 14 registros.

Parte D — Validación del resultado y estructura del proyecto

D.1 — Vista de diseño en Jaspersoft Studio

TEXT

```
+-----+
| informe_concepto.jrxml                                [Design] [Source] |
+-----+
| Title [height=100]                                     |
|   logo [0,10,80,80]                                    |
|   título [90,25,465,30] style=TituloPrincipal          |
|   fecha [90,60,120,20] + [215,60,150,20] style=TextoTabla |
|
| Column Header [height=25]                               |
|   Port. 0/50 · Título 55/180 · Precio 235/80 · Págs. 315/50 |
|   Año 365/50 · Disp. 415/80 · # 500/55                 |
|   todos con style=TextoTablaCabecera                   |
|
| Detail 1 [height=60, splitType=Prevent]                 |
|   portada 0/50 · título 55/180 · precio 235/80 · páginas 315/50 |
|   año 365/50 · disp. 415/55 · icono 475/20 · contador 500/55 |
|   TextoPrecio aplica la condición precio > 20          |
|
| Styles: Sans_Normal, TituloPrincipal, TituloSecundario, |
|         TextoTablaCabecera, TextoTabla, TextoPrecio, TextoPequeno |
+-----+
```

Qué representa: la disposición real del checkpoint 2.5 tras completar los doce pasos. La geometría es la de 2.4 y se añaden estilos reutilizables y el estilo condicional `TextoPrecio`.

Cómo verificarlo: expandir `Styles` en Outline y seleccionar título, cabeceras y campos de Detail para comprobar el atributo `Style` en Properties.

D.2 — Jerarquía del Outline

TEXT

```
informe_concepto
├── Styles
│   ├── Sans_Normal [isDefault=true, fontSize=10]
│   ├── TituloPrincipal [style=Sans_Normal, fontSize=18, bold]
│   ├── TituloSecundario [style=Sans_Normal, fontSize=14, bold]
│   ├── TextoTablaCabecera [style=Sans_Normal, fontSize=9, opaque]
│   ├── TextoTabla [style=Sans_Normal, fontSize=9]
│   ├── TextoPrecio [style=TextoTabla, conditional: precio > 20]
│   └── TextoPequeno [style=Sans_Normal, fontSize=9, italic]
├── Title [height=100]
│   ├── image [0,10,80,80]
│   ├── staticText [90,25,465,30] style=TituloPrincipal
│   ├── staticText [90,60,120,20] style=TextoTabla
│   └── textField [215,60,150,20] style=TextoTabla
├── Column Header [height=25]
│   ├── Port. [0,5,50,15] style=TextoTablaCabecera
│   ├── Título [55,5,180,15] style=TextoTablaCabecera
│   ├── Precio [235,5,80,15] style=TextoTablaCabecera
│   ├── Págs. [315,5,50,15] style=TextoTablaCabecera
│   ├── Año [365,5,50,15] style=TextoTablaCabecera
│   ├── Disp. [415,5,80,15] style=TextoTablaCabecera
│   └── # [500,5,55,15] style=TextoTablaCabecera
├── Detail 1 [height=60, splitType=Prevent]
│   ├── image [0,5,50,50] portada
│   ├── textField [55,20,180,20] style=TextoTabla
│   ├── textField [235,20,80,20] style=TextoPrecio
│   ├── textField [315,20,50,20] style=TextoTabla
│   └── textField [365,20,50,20] style=TextoTabla
```

```
└─ textField [415,20,55,20] style=TextoTabla
└─ image [475,20,20,20] icono condicional
└─ textField [500,20,55,20] style=TextoPequeno
└─ Column Footer, Page Footer, Last Page Footer, Summary
Background [height=0]
```

Qué representa: el árbol real del checkpoint 2.5. Los siete estilos están declarados antes de las bandas y los elementos reutilizan esos estilos mediante el atributo `style`.

Cómo verificarlo: expandir **Styles**, **Column Header** y **Detail 1** en **Outline**. El campo de precio debe referenciar **TextoPrecio**; el contador debe terminar exactamente en **X=555**.

D.3 — Documento PDF resultante, página por página

TEXT

INFORME: informe_concepto.pdf
PÁGINAS TOTALES: 2
REGISTROS PROCESADOS: 14

Página 1 de 2

- Cabecera del catálogo con estilos reutilizables.
- Registros 1 a 9.
- Los precios > 20 € usan TextoPrecio: rojo + negrita.
- Column Footer: Registros procesados: 9.

- Registros 10 a 14.
- Se mantienen los mismos estilos de tabla y el estilo condicional.
- Summary: Total de páginas: 2; Total de libros: 14; Subtotal precios: 270.05 €.
- Column Footer: Registros procesados: 14.
- Last Page Footer en la última página.

Qué representa: el PDF real del checkpoint 2.5. El estilo condicional no cambia la estructura de datos, pero sí la presentación de los precios superiores a 20 euros.

Cómo verificarlo: abrir el artefacto `M2-2.5-runtime`, localizar precios como 22.50 €, 21.00 €, 23.40 €, 20.80 € y 25.00 € y comprobar cuáles superan estrictamente el umbral de 20.00 € y aparecen destacados.

D.4 — Árbol de carpetas del proyecto tras completar el punto

TEXT

```

EditorialReports/
├── ECOSISTEMA.md                (documentación del ecosistema)
├── ENTORNO.md                  (documentación del entorno)
├── BANDAS.md                   (documentación de las bandas)
├── JRXML.md                    (documentación del formato JRXML)
├── TEXTO.md                    (documentación de elementos textuales)
├── CAMPOS.md                   (documentación de campos)
├── IMAGENES.md                (documentación de imágenes)
├── ESTILOS.md                  (documentación de estilos)
├── reports/
│   ├── informe_concepto.jrxml  (plantilla con 7 estilos)
│   └── informe_concepto.jasper (artefacto compilado)
├── resources/
│   ├── logo.png
│   ├── icono_disponible.png
│   ├── icono_no_disponible.png
│   └── portadas/
│       ├── Cien años de soledad.png
│       ├── Rayuela.png
│       └── Pedro Páramo.png
└── output/
    └── informe_concepto.pdf    (documento con estilos aplicados)

EditorialReportsJava/
├── lib/
│   └── README.md              (las dependencias reales se resuelven mediante Maven)
└── src/
    ├── GeneradorInformeConcepto.java
    ├── Libro.java
    └── CatalogoDataSource.java

```

Qué representa: el estado de los dos proyectos tras completar los doce pasos. La novedad respecto al punto anterior es el archivo ESTILOS.md en la raíz del proyecto EditorialReports.

Cómo verificarlo: expandir los nodos del panel Project Explorer y comparar con este esquema. Si el archivo ESTILOS.md no aparece, repetir el paso 13.

Errores comunes del ejercicio completo

Error	Causa	Solución
Duplicate default style al compilar	Existe más de un estilo con isDefault="true"	Dejar isDefault="true" en un único estilo y quitarlo en los demás
El estilo no se aplica al elemento	El atributo style no está declarado en el reportElement	Añadir style="NombreEstilo" al bloque reportElement
El estilo condicional no se aplica	La expresión de la condición no devuelve un valor booleano	Revisar la expresión y asegurarse de que devuelve true o false
El texto blanco de la cabecera no se ve	El estilo no tiene mode="Opaque" y el fondo no se rellena	Añadir mode="Opaque" al estilo de la cabecera
El estilo no se encuentra al compilar	El nombre del estilo en el atributo style no coincide con el nombre declarado	Verificar que el nombre es idéntico y respeta mayúsculas y minúsculas
El estilo hereda propiedades incorrectas	El atributo style apunta a un estilo inexistente	Verificar que el estilo padre está declarado antes del hijo
El color del estilo no se aplica al elemento	El elemento sobrescribe el color con un forecolor propio	Eliminar el forecolor del elemento para que herede el del estilo
El estilo condicional se aplica a todos los registros	La condición siempre devuelve verdadero	Revisar la expresión y asegurarse de que compara con el valor correcto
El estilo no se aplica a los elementos de una banda	El atributo style está declarado en el elemento pero no en la banda	Los estilos se aplican elemento por elemento, no por banda
El panel Outline no muestra la sección Styles	Los estilos no están declarados antes de las bandas	Mover las declaraciones de estilo a la posición correcta del JRXML

Reto resuelto paso a paso

Enunciado: añadir un estilo TextoDisponible que herede de TextoTabla y aplique un estilo condicional que muestre en verde el texto Sí y en gris el texto No. Aplicar el estilo al campo de disponibilidad y verificar el resultado en el PDF.

Paso 1. Hacer doble clic sobre el archivo informe_concepto.jrxml en el panel Project Explorer.

Paso 2. Hacer clic sobre la pestaña Source en la parte inferior del editor central.

Paso 3. Hacer clic al final de la línea que contiene `<style name="TextoPequeno" .../>` y pulsar Enter.

Paso 4. Escribir exactamente `<style name="TextoDisponible" style="TextoTabla">` y pulsar Enter.

Paso 5. Escribir exactamente `<conditionalStyle>` y pulsar Enter.

Paso 6. Escribir exactamente `<conditionExpression><![CDATA[{$f{disponible}.booleanValue()}]]></conditionExpression>` y pulsar Enter.

Paso 7. Escribir exactamente `<style forecolor="#006600" isBold="true"/>` y pulsar Enter.

Paso 8. Escribir exactamente `</conditionalStyle>` y pulsar Enter.

Paso 9. Escribir exactamente `<conditionalStyle>` y pulsar Enter.

Paso 10. Escribir exactamente `<conditionExpression><![CDATA[!$F{disponible}.booleanValue()]]></conditionExpression>` y pulsar Enter.

Paso 11. Escribir exactamente `<style forecolor="#888888" isItalic="true"/>` y pulsar Enter.

Paso 12. Escribir exactamente `</conditionalStyle>` y pulsar Enter.

Paso 13. Escribir exactamente `</style>` y pulsar Enter.

Paso 14. Pulsar Ctrl+S para guardar el archivo.

Paso 15. Pulsar Ctrl+Mayús+B para compilar el informe.

Paso 16. Hacer clic sobre la pestaña Design en la parte inferior del editor central.

Paso 17. Hacer clic sobre el nodo Detail 1 en el panel Outline.

Paso 18. Hacer clic sobre el Text Field que contiene la expresión `$F{disponible}.booleanValue() ? "Sí" : "No"` en el editor central.

Paso 19. Hacer clic sobre el desplegable Style en el panel Properties y seleccionar `TextoDisponible`.

Paso 20. Pulsar `Ctrl+S` para guardar el archivo.

Paso 21. Pulsar `Ctrl+Mayús+B` para compilar el informe.

Paso 22. Hacer clic con el botón derecho sobre `GeneradorInformeConcepto.java` y seleccionar `Run As > Java Application`.

Paso 23. Abrir el archivo `output/informe_concepto.pdf` y verificar que los libros disponibles muestran **Sí** en verde y negrita, y que los libros no disponibles muestran *No* en gris y cursiva.

Simulación ASCII del PDF tras el reto

TEXT

Portada	Título	Precio	Páginas	Año	Disp.	#
IMG	Cien años de sol.	19,95 €	471	1967	Sí (verde)	1
IMG	Rayuela	22,50 € (rojo)	736	1963	Sí (verde)	2
IMG	Doña Bárbara	16,95 €	400	1929	No (gris) cursiva	9

Resultado del reto: el estilo `TextoDisponible` aplica dos estilos condicionales. El primero resalta en verde y negrita los libros disponibles. El segundo muestra en gris y cursiva los libros no disponibles. Los libros *Doña Bárbara*, *Martín Fierro* y *El túnel* aparecen con el texto *No* en gris y cursiva.

Analogía final con el contexto de la editorial

Los estilos son la hoja de estilo del catálogo. El estilo por defecto es la tipografía base que se aplica a todas las páginas. Los estilos derivados son las variantes para títulos, subtítulos, cabeceras de tabla y notas. Los estilos condicionales son las reglas de composición que el editor aplica según el contenido: los precios altos se resaltan en rojo, los libros agotados se muestran en gris. La jerarquía de estilos refleja la jerarquía visual del documento y permite que un cambio en la tipografía base se propague a todas las páginas sin tener que rehacerlas una por una. El catálogo se compone aplicando estilos coherentes a cada tipo de contenido.

Resultado esperado

Al finalizar este punto, el alumno dispone de:

- El archivo `reports/informe_concepto.jrxml` con siete estilos declarados y aplicados a los elementos del informe.
- El estilo `TituloPrincipal` aplicado al título de la banda Title.
- El estilo `TextoTablaCabecera` aplicado a los encabezados de la banda Column Header.
- El estilo `TextoTabla` aplicado a los campos de la banda Detail.
- El estilo `TextoPrecio` con estilo condicional aplicado al campo del precio.
- El estilo `TextoPequeno` aplicado a los elementos del número de registro.
- El archivo `output/informe_concepto.pdf` con los estilos aplicados y los precios superiores a 20 euros resaltados en rojo.
- El archivo `ESTILOS.md` en la raíz del proyecto con la documentación de los estilos.
- Comprensión operativa de los estilos, de la herencia y de los estilos condicionales.

Conclusión y enlace al siguiente punto

El punto 2.5 ha sustituido formato repetido por una jerarquía de estilos locales y ha añadido comportamiento condicional al precio. El resultado es un informe más mantenible, en el que tipografía, color y jerarquía visual pueden modificarse desde definiciones centralizadas.

El punto 2.6, «Expresiones», utiliza esa base para introducir cálculos, comparaciones, métodos Java, parámetros y variables dentro del JRXML. El objetivo ya no será solo presentar datos, sino derivar información nueva a partir de ellos.

Punto 2.6 — Expresiones

PUNTO DE PARTIDA. Continúa con tu propio resultado de 2.5. Si te incorporas directamente en este punto, usa M2/2.5 como estado inicial. M2/2.6 contiene la solución completa y no debe utilizarse como punto de partida si quieres realizar la práctica sin ver la respuesta.

Parte A — Práctica visual

Paso 1: Crear el parámetro usuario VALIDADO

Acciones:

1. Abrir `reports/informe_concepto.jrxml` y seleccionar la pestaña **Design**.
2. En el panel **Outline**, localizar el nodo **Parameters**.
3. Hacer clic con el botón derecho sobre **Parameters** y seleccionar **Create Parameter**.
4. Seleccionar el nuevo parámetro y abrir la pestaña **Object** del panel **Properties**.
5. Escribir exactamente `usuario` en **Name**.
6. Seleccionar `java.lang.String` en **Class**.
7. Escribir exactamente `"Ana Martínez"` en **Default Value Expression**.
8. Desmarcar **Is For Prompting** si estuviera activado.
9. Pulsar `Ctrl+S`.

Verificación visual: el nodo **Parameters** del **Outline** muestra `usuario` y **Properties** indica la clase `java.lang.String` y el valor por defecto `"Ana Martínez"`.

Qué hace: crea un dato externo al registro actual que puede enviarse desde la aplicación Java o resolverse con su valor por defecto.

Por qué: permite demostrar la diferencia entre `$P{usuario}` y los campos `$F{...}` obtenidos desde `CatalogoDataSource`.

Error común: escribir `Ana Martínez` sin comillas en **Default Value Expression**. El compilador intenta interpretarlo como código Java. **Solución:** escribir una expresión Java de cadena: `"Ana Martínez"`.

Analogía: es como imprimir en el catálogo el nombre del operador que ha lanzado la tirada.

Paso 2: Crear la variable `PrecioConIVA` VALIDADO

Acciones:

1. En el panel **Outline**, hacer clic con el botón derecho sobre **Variables**.
2. Seleccionar **Create Variable**.
3. Seleccionar la nueva variable y abrir **Properties > Object**.
4. Escribir `PrecioConIVA` en **Name**.
5. Seleccionar `java.lang.Double` como tipo de la variable.
6. En **Variable Expression**, abrir el editor de expresiones.
7. Escribir exactamente `$F{precio} == null ? null : Double.valueOf($F{precio}.doubleValue() * 1.21d)`.
8. Confirmar el editor y pulsar `Ctrl+S`.

Verificación visual: **Variables** contiene `PrecioConIVA` y la expresión aparece en **Properties** sin error.

Qué hace: calcula un valor derivado del precio para cada registro.

Por qué: centraliza el cálculo del IVA y permite reutilizarlo desde un `textField` mediante `$V{PrecioConIVA}`.

Error común: declarar la variable como `java.lang.String` y devolver un `Double`. **Solución:** mantener `java.lang.Double` para que el tipo coincida con el resultado.

Analogía: es como añadir a cada ficha del catálogo una casilla calculada automáticamente a partir del precio base.

Paso 3: Mostrar el usuario en la banda Title VALIDADO

Acciones:

1. En la pestaña **Design**, seleccionar la banda **Title**.
2. Seleccionar el **Text Field** de fecha (`new java.util.Date()`) y cambiar únicamente **Width** de 150 a 110; mantener **X=215**, **Y=60**, **Height=20**.
3. Arrastrar un **Text Field** desde **Palette > Elements** hasta la zona derecha de la línea de fecha.
4. En **Properties** escribir 335 en **X**, 60 en **Y**, 220 en **Width** y 20 en **Height**.
5. Seleccionar el estilo **TextoPequeno**.
6. Seleccionar **Right** en **Horizontal Text Alignment**.
7. En **Text Field Expression** escribir exactamente "Usuario: " + `$P{usuario}`.
8. Guardar con **Ctrl+S**.

Verificación visual: el campo de fecha termina en **X=325** y el nuevo campo de usuario comienza en **X=335**, por lo que no se solapan; la expresión del nuevo campo utiliza `$P{usuario}`.

Qué hace: combina un literal con un parámetro del informe.

Por qué: demuestra que el informe puede recibir información que no forma parte del `JRDataSource`.

Error común: añadir el usuario sin reducir primero el ancho de la fecha, provocando solapamiento entre **X=335** y la fecha heredada. **Solución:** dejar la fecha en **X=215**, **Width=110** y después usar `$P{usuario}` en el campo **X=335**.

Analogía: es como añadir en la portada la firma del operador que ha generado el catálogo.

Paso 4: Ampliar la banda Detail para una segunda fila VALIDADO

Acciones:

1. Seleccionar **Detail 1** en **Outline**.
2. En **Properties** escribir 85 en **Band height**.
3. Mantener **Split Type = Prevent**.
4. Comprobar que los elementos existentes permanecen entre **y=5** y **y=40**.
5. Reservar la franja **y=60** . **80** para los campos calculados.

Verificación visual: la banda **Detail** mide 85 unidades y deja una segunda línea libre debajo de la fila principal.

Qué hace: añade espacio vertical para las nuevas expresiones sin mover la estructura principal del punto 2.5.

Por qué: cinco resultados calculados no caben en la misma fila sin reducir en exceso las columnas existentes.

Error común: mantener la altura en 60 y colocar campos en **y=60**. Los elementos quedan fuera de la banda. **Solución:** ampliar primero la banda a 85.

Analogía: es como añadir una segunda línea de anotaciones debajo de cada ficha del catálogo.

Paso 5: Añadir la categoría calculada del precio VALIDADO

Acciones:

1. Arrastrar un **Text Field** a **Detail**.
2. Escribir 0 en **X**, 60 en **Y**, 100 en **Width** y 20 en **Height**.
3. Seleccionar el estilo **TextoPequeno**.
4. En **Text Field Expression** escribir exactamente `$F{precio}.doubleValue() > 20.0d ? "Premium" : ($F{precio}.doubleValue() > 15.0d ? "Estándar" : "Económico")`.
5. Guardar el informe.

Verificación visual: el campo ocupa la primera columna de la segunda fila y su expresión contiene dos operadores ternarios.

Qué hace: clasifica cada libro en tres categorías de precio.

Por qué: demuestra operadores de comparación y un ternario anidado.

Error común: omitir los paréntesis del ternario interno. **Solución:** conservar la agrupación (... ? ... : ...).

Analogía: es como asignar automáticamente una banda comercial a cada título según su precio.

Paso 6: Añadir una expresión con métodos de String VALIDADO

Acciones:

1. Arrastrar otro **Text Field** a Detail.
2. Escribir 100 en X, 60 en Y, 135 en Width y 20 en Height.
3. Seleccionar TextoPequeno.
4. Escribir exactamente `$F{titulo}.length() > 30 ? "Título largo (" + $F{titulo}.length() + ")" : "Título corto"` en Text Field Expression.
5. Guardar.

Verificación visual: la expresión usa dos llamadas a `length()` y devuelve una cadena.

Qué hace: evalúa la longitud del título y construye un texto descriptivo.

Por qué: demuestra la invocación de métodos Java sobre un campo `String`.

Error común: escribir `length` sin paréntesis. **Solución:** usar `length()` porque se trata de un método.

Analogía: es como etiquetar automáticamente los títulos que pueden necesitar más espacio tipográfico.

Paso 7: Mostrar la variable PrecioConIVA con patrón numérico VALIDADO

Acciones:

1. Arrastrar un **Text Field** a Detail.
2. Escribir 235 en X, 60 en Y, 85 en Width y 20 en Height.
3. Seleccionar TextoPequeno y alineación **Right**.
4. Escribir `$V{PrecioConIVA}` en Text Field Expression.
5. Escribir exactamente ' IVA: ' #,##0.00 € en **Pattern**.
6. Activar **Blank When Null**.
7. Guardar.

Verificación visual: el campo referencia `$V{PrecioConIVA}` y el patrón contiene el literal IVA: sin imprimir comillas.

Qué hace: presenta el resultado de una variable con un formato numérico.

Por qué: separa el cálculo del formato visual del resultado.

Error común: utilizar comillas dobles dentro del patrón (" IVA: "). El PDF puede imprimir esas comillas. **Solución:** utilizar comillas simples de patrón: ' IVA: ' #,##0.00 €.

Analogía: es como calcular el precio en una hoja de trabajo y aplicar después el formato de moneda de la editorial.

Paso 8: Añadir una expresión de fecha VALIDADO

Acciones:

1. Arrastrar un **Text Field** a Detail.
2. Escribir 325 en X, 60 en Y, 150 en Width y 20 en Height.
3. Seleccionar TextoPequeno.
4. Escribir exactamente `$F{fechaPublicacion}.after(new java.util.GregorianCalendar(2000, 0, 1).getTime()) ? "Después de 2000" : "Hasta 2000"`.
5. Guardar.

Verificación visual: el campo ocupa la cuarta zona de la segunda fila y la expresión contiene `after( . . . )`.

Qué hace: compara la fecha del libro con el 1 de enero de 2000.

Por qué: demuestra una llamada a un método de `Date` y la creación de una fecha de referencia.

Error común: usar mes 1 pensando que es enero. **Solución:** en `GregorianCalendar`, enero es 0.

Analogía: es como separar automáticamente fondo histórico y novedades editoriales.

Paso 9: Combinar campo, parámetro y variable en una expresión VALIDADO

Acciones:

1. Arrastrar un último **Text Field** a Detail.
2. Escribir 480 en X, 60 en Y, 75 en Width y 20 en Height.
3. Seleccionar `TextoPequeno` y alineación **Right**.
4. En Text Field Expression escribir exactamente `"R" + $V{REPORT_COUNT} + " · " + $P{usuario}.substring(0, Math.min(3, $P{usuario}.length())) + " · " + $F{titulo}.substring(0, 1)`.
5. Guardar.

Verificación visual: la expresión contiene simultáneamente `$V{REPORT_COUNT}`, `$P{usuario}` y `$F{titulo}`.

Qué hace: construye un identificador contextual para cada registro, por ejemplo `R1 · Ana · C`.

Por qué: demuestra en una sola expresión la combinación de variable, parámetro, campo y métodos Java.

Error común: asignar un ancho que termine más allá de `columnWidth=555`. **Solución:** mantener `x=480`, `width=75`; `480 + 75 = 555`.

Analogía: es como imprimir una marca de control con el número de ficha, el operador y la inicial del libro.

Paso 10: Pasar el parámetro usuario desde Java VALIDADO

Acciones:

1. Abrir `EditorialReportsJava/src/GeneradorInformeConcepto.java`.
2. Localizar la línea `Map<String, Object> parametros = new HashMap<String, Object>();`.
3. En la línea siguiente escribir exactamente `parametros.put("usuario", "Ana Martínez");`.
4. Guardar con `Ctrl+S`.
5. Comprobar que Problems no muestra errores Java.

Verificación visual: el programa Java contiene el `put` inmediatamente después de crear el mapa de parámetros.

Qué hace: envía el valor `Ana Martínez` desde la aplicación al parámetro `$P{usuario}`.

Por qué: demuestra la comunicación real aplicación → informe; el valor por defecto queda como fallback para Preview.

Error común: usar la clave `Usuario` con mayúscula. **Solución:** la clave del mapa debe coincidir exactamente con el nombre `usuario` declarado en el JRXML.

Analogía: es como entregar a la imprenta una orden de trabajo con el nombre del operador que debe aparecer en el documento.

Paso 11: Compilar, ejecutar y verificar las tres páginas VALIDADO

Acciones:

1. Guardar el JRXML con `Ctrl+S`.
2. Compilar el informe y comprobar Problems.
3. Ejecutar `GeneradorInformeConcepto.java` como **Java Application** con el working directory del proyecto `EditorialReports`.

4. Observar Console.
5. Abrir `output/informe_concepto.pdf`.
6. Comprobar que Console informa Registros de ejemplo: 14.
7. Comprobar que Console informa Páginas del documento: 3.
8. Verificar que las páginas muestran Página 1 de 3, Página 2 de 3 y Página 3 de 3.
9. Verificar que cada registro contiene la segunda línea calculada.

Verificación visual: el PDF tiene tres páginas; la última muestra el registro 14, los totales y el Last Page Footer.

Qué hace: valida el conjunto completo de expresiones sobre datos reales.

Por qué: una expresión puede compilar y fallar durante el llenado; la ejecución completa confirma ambas fases.

Error común: esperar una sola página como en una simulación anterior. **Solución:** con Detail de 85 unidades y 14 registros, el checkpoint validado genera tres páginas.

Analogía: es como revisar la tirada completa y no solo la maqueta de una ficha.

Paso 12: Documentar las expresiones VALIDADO

Acciones:

1. Crear `EXPRESIONES.md` en la raíz de `EditorialReports`.
2. Añadir un apartado para el parámetro `usuario`.
3. Añadir un apartado para la variable `PrecioConIVA`.
4. Documentar categoría, longitud, IVA, antigüedad y contexto de registro.
5. Añadir una nota: `PAGE_COUNT` cuenta registros de la página actual y no el total de páginas.
6. Guardar el archivo.

Verificación visual: Project Explorer muestra `EXPRESIONES.md` junto a `ESTILOS.md` e `IMAGENES.md`.

Qué hace: deja una referencia de mantenimiento de las expresiones incorporadas en 2.6.

Por qué: las expresiones son código embebido en la plantilla y deben documentarse igual que el código Java.

Error común: documentar `PAGE_COUNT` como total de páginas. **Solución:** registrar que el total final se obtiene en este proyecto con `PAGE_NUMBER` evaluado a `Report`.

Analogía: es como conservar una ficha técnica de todas las fórmulas usadas en la maquetación.

Parte B — JRXML explicado y contrastado COMPLETADO

Las secciones nuevas o modificadas respecto a 2.5 son el parámetro `usuario`, la variable `PrecioConIVA`, el campo de usuario en Title y la segunda fila de la banda Detail. El resto del JRXML permanece acumulado desde el checkpoint 2.5.

XML

```
<parameter name="usuario" class="java.lang.String">
  <defaultValueExpression><![CDATA["Ana Martínez"]]></defaultValueExpression>
</parameter>

<variable name="PrecioConIVA" class="java.lang.Double">
  <variableExpression><![CDATA[
    ${precio} == null
      ? null
      : Double.valueOf(${precio}.doubleValue() * 1.21d)
  ]]></variableExpression>
</variable>

<!-- En Title -->
<textField>
  <reportElement x="335" y="60" width="220" height="20" uuid="28282828-2828-2828-2828-282828282828" style="TextoPequeno"/>
  <textElement textAlignment="Right"/>
  <textFieldExpression><![CDATA["Usuario: " + ${P{usuario}}]]></textFieldExpression>
</textField>

<!-- En Detail: la banda pasa a 85 -->
<band height="85" splitType="Prevent">
```

```

<!-- ... fila principal heredada de 2.5 ... -->

<textField>
  <reportElement x="0" y="60" width="100" height="20" uuid="29292929-2929-2929-2929-292929292929" style="TextoPequeno"/>
  <textFieldExpression><![CDATA[
    ${precio}.doubleValue() > 20.0d
    ? "Premium"
    : (${precio}.doubleValue() > 15.0d ? "Estándar" : "Económico")
  ]]></textFieldExpression>
</textField>

<textField>
  <reportElement x="100" y="60" width="135" height="20" uuid="30303030-3030-3030-3030-303030303030" style="TextoPequeno"/>
  <textFieldExpression><![CDATA[
    ${titulo}.length() > 30
    ? "Título largo (" + ${titulo}.length() + ")"
    : "Título corto"
  ]]></textFieldExpression>
</textField>

<textField pattern="' IVA: ' #,##0.00 €" isBlankWhenNull="true">
  <reportElement x="235" y="60" width="85" height="20" uuid="31313131-3131-3131-3131-313131313131" style="TextoPequeno"/>
  <textElement textAlignment="Right"/>
  <textFieldExpression><![CDATA[${V{PrecioConIVA}}]></textFieldExpression>
</textField>

<textField>
  <reportElement x="325" y="60" width="150" height="20" uuid="32323232-3232-3232-3232-323232323232" style="TextoPequeno"/>
  <textFieldExpression><![CDATA[
    ${fechaPublicacion}.after(new java.util.GregorianCalendar(2000, 0, 1).getTime())
    ? "Después de 2000"
    : "Hasta 2000"
  ]]></textFieldExpression>
</textField>

<textField>
  <reportElement x="480" y="60" width="75" height="20" uuid="33333333-3333-3333-3333-333333333333" style="TextoPequeno"/>
  <textElement textAlignment="Right"/>
  <textFieldExpression><![CDATA[
    "R" + ${REPORT_COUNT}
    + " · " + ${P{usuario}}.substring(0, Math.min(3, ${P{usuario}}.length()))
    + " · " + ${F{titulo}}.substring(0, 1)
  ]]></textFieldExpression>
</textField>
</band>

```

Explicación línea por línea

Línea 1	<code><parameter name="usuario" class="java.lang.String"></code> → declara el parámetro usuario.
Línea 2	<code><defaultValueExpression><![CDATA["Ana Martínez"]]></defaultValueExpression></code> → define el valor usado cuando la aplicación no envía el parámetro.
Línea 3	<code></parameter></code> → cierra la declaración del parámetro.
Línea 5	<code><variable name="PrecioConIVA" class="java.lang.Double"></code> → declara una variable de tipo Double.
Línea 6-10	<code><variableExpression>...</variableExpression></code> → calcula el precio con IVA y conserva null si el campo precio fuera nulo.
Línea 14-18	textField de Title → concatena el literal Usuario: con <code>\${P{usuario}}</code> y lo alinea a la derecha.
Línea 21	<code><band height="85" splitType="Prevent"></code> → amplía Detail a 85 unidades y hace que el motor intente evitar la primera división de cada ficha; si no cabe tras el desplazamiento, JasperReports puede permitir la división para evitar un ciclo.
Línea 24-33	primer textField calculado → clasifica el precio con comparaciones y ternarios.
Línea 35-44	segundo textField calculado → usa <code>String.length()</code> para etiquetar la longitud del título.
Línea 46-50	campo de IVA → muestra <code>\${V{PrecioConIVA}}</code> y aplica el patrón ' IVA: ' #,##0.00 €.
Línea 52-60	campo de antigüedad → compara una fecha mediante <code>Date.after(...)</code> .
Línea 62-71	campo contextual → combina <code>\${V{REPORT_COUNT}}</code> , <code>\${P{usuario}}</code> y <code>\${F{titulo}}</code> en una misma expresión.
Línea 72	<code></band></code> → cierra la banda Detail modificada.

Parte C — Código Java explicado línea por línea COMPLETADO

El cambio Java del punto 2.6 es pequeño pero importante: el mapa de parámetros deja de estar vacío y pasa el valor usuario al informe.

JAVA

```
import java.io.File;
import java.util.HashMap;
import java.util.Map;

import net.sf.jasperreports.engine.JasperCompileManager;
import net.sf.jasperreports.engine.JasperExportManager;
import net.sf.jasperreports.engine.JasperFillManager;
import net.sf.jasperreports.engine.JasperPrint;

public class GeneradorInformeConcepto {
    public static void main(String[] args) {
        try {
            String rutaJrxml = "reports/informe_concepto.jrxml";
            String rutaJasper = "reports/informe_concepto.jasper";
            String rutaPdf = "output/informe_concepto.pdf";

            JasperCompileManager.compileReportToFile(rutaJrxml, rutaJasper);

            Map<String, Object> parametros = new HashMap<String, Object>();
            parametros.put("usuario", "Ana Martínez");

            JasperPrint documento = JasperFillManager.fillReport(
                rutaJasper,
                parametros,
                new CatalogoDataSource(Libro.listaEjemplo()));

            JasperExportManager.exportReportToPdfFile(documento, rutaPdf);

            System.out.println("Informe generado en: " + new File(rutaPdf).getAbsolutePath());
            System.out.println("Paginas del documento: " + documento.getPages().size());
            System.out.println("Registros de ejemplo: " + Libro.listaEjemplo().size());
        } catch (Exception e) {
            e.printStackTrace();
            System.exit(1);
        }
    }
}
```

Explicación línea por línea

Línea 1	<code>import java.io.File;</code> → importa File para mostrar la ruta absoluta del PDF.
Línea 2	<code>import java.util.HashMap;</code> → importa la implementación del mapa de parámetros.
Línea 3	<code>import java.util.Map;</code> → importa la interfaz Map.
Línea 5-8	imports JasperReports → cargan compilación, llenado, exportación y JasperPrint.
Línea 10	<code>public class GeneradorInformeConcepto</code> → declara la clase ejecutable.
Línea 11	<code>public static void main(String[] args)</code> → declara el punto de entrada.
Línea 13-15	rutasy JRXML, Jasper y PDF → fijan las rutas relativas al working directory EditorialReports.
Línea 17	<code>JasperCompileManager.compileReportToFile(...)</code> → compila la plantilla.
Línea 19	<code>new HashMap<String, Object>()</code> → crea el mapa de parámetros.
Línea 20	<code>parametros.put("usuario", "Ana Martínez")</code> → envía el parámetro utilizado por <code>\$P{usuario}</code> .
Línea 22-25	<code>JasperFillManager.fillReport(...)</code> → llena el informe con el mapa y CatalogoDataSource.
Línea 27	<code>JasperExportManager.exportReportToPdfFile(...)</code> → exporta el JasperPrint a PDF.
Línea 29-31	<code>System.out.println(...)</code> → registra ruta, páginas y número de registros para la validación.
Línea 32-35	<code>catch + System.exit(1)</code> → imprime la traza y devuelve error al sistema si falla alguna fase.

Parte D — Validación del resultado y estructura del proyecto

D.1 — Vista de diseño esperada

TEXT

```
Detail 1 [height=85]
┌──────────┴──────────┐
│ Port. | Título | Precio | Págs. | Año | Disp. | # │
├──────────┴──────────┤
│ Categoría | Longitud | IVA | Antigüedad | Rn · usuario · inicial │
└──────────┴──────────┘
0 555
```

Qué representa: las dos filas de la banda Detail. Todos los elementos terminan como máximo en $x + width = 555$, por lo que permanecen dentro del ancho útil del informe.

Cómo verificarlo: seleccionar cada uno de los cinco campos nuevos y comprobar X, Y, Width y Height en Properties.

D.2 — Jerarquía del Outline

TEXT

```
informe_concepto
├── Parameters
│   └── usuario [java.lang.String]
├── Variables
│   ├── TotalPrecios [java.lang.Double, Sum]
│   └── PrecioConIVA [java.lang.Double]
├── Title [height=100]
│   └── textField → "Usuario: " + ${usuario}
├── Detail 1 [height=85, splitType=Prevent]
│   ├── (elementos acumulados de 2.5)
│   ├── textField → categoría
│   ├── textField → longitud del título
│   ├── textField → ${PrecioConIVA}
│   ├── textField → comparación de fecha
│   └── textField → ${} + ${} + ${}
└── ...
```

Qué representa: las novedades estructurales introducidas por 2.6.

Cómo verificarlo: expandir Parameters, Variables, Title y Detail 1 en Outline.

D.3 — PDF real validado por GitHub Actions

TEXT

```
INFORME: informe_concepto.pdf
PÁGINAS TOTALES: 3
REGISTROS PROCESADOS: 14

Página 1 de 3
  registros 1-6
  segunda fila de expresiones en cada ficha

Página 2 de 3
  registros 7-13
  segunda fila de expresiones en cada ficha

Página 3 de 3
  registro 14
  Total de páginas: 3
  Total de libros: 14
  Subtotal precios: 270.05 €
  Last Page Footer
```

Qué representa: el resultado real del checkpoint 2.6, no una estimación. El workflow E2E informó Páginas del documento: 3 y Registros de ejemplo: 14.

Cómo verificarlo: abrir el artefacto M2-2.6-runtime o ejecutar localmente el mismo generador.

D.4 — Árbol de carpetas al finalizar 2.6

TEXT

```
EditorialReports/
├── BANDAS.md
└── CAMPOS.md
```

```

├── ECOSISTEMA.md
├── ENTORNO.md
├── ESTILOS.md
├── EXPRESIONES.md
├── IMAGENES.md
├── JRXML.md
├── TEXTO.md
├── reports/
│   └── informe_concepto.jrxml
├── resources/
│   ├── logo.png
│   ├── icono_disponible.png
│   ├── icono_no_disponible.png
│   └── portadas/
├── output/
│   └── informe_concepto.pdf
├── EditorialReportsJava/
│   ├── pom.xml
│   └── src/
│       ├── Libro.java
│       ├── CatalogoDataSource.java
│       └── GeneradorInformeConcepto.java

```

Qué representa: el estado final acumulativo del Módulo 2.

Cómo verificarlo: comparar Project Explorer con el checkpoint M2/2.6 después de completar la práctica.

Errores comunes del ejercicio completo

Error	Causa	Solución
Variable not found: PrecioConIVA	La variable no está declarada o está después de un punto inválido del JRXML	Crear la variable desde Outline o colocarla en la zona de variables antes de las bandas
Parameter not found: usuario	Se escribió \$P{usuario} sin declarar el parámetro	Crear el parámetro usuario de tipo <code>java.lang.String</code>
El usuario aparece con el valor por defecto aunque se esperaba otro	La aplicación no ha añadido la clave al mapa	Ejecutar <code>parametros.put("usuario", valor)</code> antes de <code>fillReport</code>
El PDF imprime comillas alrededor de IVA:	El patrón usa comillas dobles como literal	Utilizar el patrón <code>'IVA: ' #,##0.00 €</code>
<code>NullPointerException</code> en <code>length()</code> o <code>after()</code>	El campo es nulo	Comprobar <code>null</code> antes de invocar métodos si la fuente puede devolver valores nulos
El último campo sale fuera de la página	<code>x + width</code> supera 555	Ajustar posiciones; en la solución <code>480 + 75 = 555</code>
Se utiliza <code>PAGE_COUNT</code> como total de páginas	Se ha confundido con <code>PAGE_NUMBER</code>	Recordar que <code>PAGE_COUNT</code> cuenta registros de la página actual
El PDF conserva una versión anterior	Se ejecuta un <code>.jasper</code> desactualizado	Recompilar el JRXML antes de llenar
El informe compila pero falla al llenar	La expresión es sintácticamente válida pero falla con un dato concreto	Revisar Console y la traza de la excepción
Actions no detecta el error Java	El programa captura la excepción y termina con código 0	Mantener <code>System.exit(1)</code> en el bloque catch

Reto resuelto paso a paso

Enunciado: añadir una variable Descuento que calcule el 5 % del precio y mostrar, en el Summary, el descuento acumulado teórico de todos los libros sin alterar la fila principal del catálogo.

Paso 1. Crear una variable Descuento de tipo `java.lang.Double` con la expresión `$F{precio} == null ? null : Double.valueOf($F{precio}.doubleValue() * 0.05d)`.

Paso 2. Crear una segunda variable TotalDescuento de tipo `java.lang.Double`, `calculation="Sum"`, con expresión `$V{Descuento}`.

Paso 3. Guardar y compilar el JRXML.

Paso 4. Aumentar la banda Summary de 95 a 120 unidades.

Paso 5. Arrastrar un Static Text al Summary y escribir Descuento teórico total:.

Paso 6. Arrastrar un Text Field junto al rótulo y usar `$V{TotalDescuento}`.

Paso 7. Aplicar el patrón `#,##0.00 €`.

Paso 8. Guardar, compilar y ejecutar `GeneradorInformeConcepto`.

Paso 9. Abrir el PDF y verificar que el valor del descuento total aparece después del subtotal de precios.

Paso 10. Confirmar que el informe continúa generándose en tres páginas y que los 14 registros siguen presentes.

Resultado del reto: el alumno diferencia una variable de cálculo por registro (Descuento) de una variable agregada (`TotalDescuento`) y comprueba que una variable puede depender de otra variable del informe.

Analogía final con el contexto de la editorial

Las expresiones son las fórmulas de la mesa de producción. Los campos traen los datos de la ficha del libro, los parámetros traen instrucciones de la orden de trabajo y las variables conservan resultados calculados por el informe. La expresión de categoría clasifica el producto, `length()` analiza el título, `after()` compara fechas y `PrecioConIVA` encapsula un cálculo repetible. El informe deja de ser una maqueta pasiva y empieza a interpretar los datos que recibe.

Resultado esperado

Al finalizar este punto, el alumno dispone de:

- El checkpoint acumulativo M2/2.6 basado en M2/2.5.
- El parámetro `usuario` declarado en el JRXML y enviado desde Java.
- La variable `PrecioConIVA` de tipo `Double`.
- Cinco expresiones calculadas en la segunda fila de `Detail`.
- Una expresión que combina simultáneamente `$F{}`, `$P{}` y `$V{}`.
- El archivo `EXPRESIONES.md` con la documentación del punto.
- Un PDF real de 3 páginas con 14 registros, validado end-to-end.
- Comprensión operativa de operadores, métodos Java, parámetros, variables y depuración de expresiones.

Conclusión y enlace al siguiente punto

El punto 2.6 cierra el Módulo 2 incorporando lógica de cálculo al informe acumulativo. El proyecto ya combina estructura de bandas, texto, campos tipados, imágenes, estilos y expresiones, y demuestra el ciclo completo desde JasperSoft Studio hasta un PDF real generado por Java.

El siguiente módulo deberá partir **exactamente del checkpoint M2/2.6**. No se reiniciará `EditorialReports` ni se volverá a una versión anterior del informe.

Validación end-to-end del módulo

La práctica no termina en una simulación documental. Los seis checkpoints se ejecutan en CI con JDK 8. Para cada uno se compila el proyecto Java, se resuelven las dependencias Maven, se compila el JRXML, se llena el informe, se genera un PDF y se comprueba la firma `%PDF-`. El identificador del run de cierre de esta revisión se registra en `VALIDACION_M2.md` después de ejecutar la matriz completa 2.1-2.6.